

Appunti del corso di Basi di dati

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis

Matteo Drago

Indice

1	Tecnologie per le basi di dati	1
1.1	Architettura transazionale di un DBMS	1
1.2	Transazione	2
1.2.1	Sintassi	2
1.2.2	Proprietà delle transazioni	3
1.3	Architettura di un DBMS	3
2	Linguaggio SQL	6
2.1	Clausola SELECT	7
2.2	Clausola FROM	9
2.3	Clausola WHERE	10
2.3.1	Operatore LIKE	11
2.3.2	Operatore SIMILAR TO	12
2.3.3	Operatore BETWEEN	13
2.3.4	Operatore IN	14
2.3.5	Operatore IS NULL	14
2.4	Clausola DISTINCT	15
2.5	Clausola JOIN	16
2.5.1	Clausola INNER JOIN	17
2.5.2	Clausola LEFT OUTER JOIN	18
2.5.3	Clausola RIGHT OUTER JOIN	19
2.5.4	Clausola FULL OUTER JOIN	19
2.6	Clausola AS	19
2.7	Clausola ORDER BY	20
2.8	Operazioni insiemistiche	22
2.8.1	Operatore UNION	22
2.8.2	Operatore INTERSECT	24
2.8.3	Operatore EXCEPT	25
2.9	Operatori aggregati	25
2.9.1	Operatore COUNT	25
2.9.2	Operatore SUM	26
2.9.3	Operatore MAX/MIN	27
2.9.4	Operatore AVG	27
2.10	Clausola GROUP BY	28
2.10.1	Clausola HAVING	29
2.11	Interrogazioni nidificate	30

2.11.1	Clausola WHERE	30
2.11.2	Clausola EXISTS	31
2.11.3	Clausola SELECT	32
2.11.4	Clausola FROM	32
3	Strutture fisiche e di accesso ai dati	34
3.1	Gestore del buffer	34
3.1.1	Buffer	35
3.1.2	Politica di gestione della memoria	35
3.1.3	Gestione delle pagine	35
3.1.4	Primitive per la gestione del buffer	36
3.1.4.1	Primitiva fix	36
3.1.4.2	Primitiva unfix	36
3.1.4.3	Primitiva setDirty	36
3.1.4.4	Primitiva force	36
3.1.4.5	Primitiva flush	37
3.2	Gestore dell'affidabilità	37
3.2.1	File di LOG	37
3.2.2	Regole di scrittura sul file di LOG	38
3.2.3	Guasti	39
3.3	Gestore dei metodi di accesso	41
3.3.1	Metodi di accesso	41
3.3.2	Organizzazione di una pagina	42
3.3.3	Operazioni sulle pagine	43
3.3.4	Strutture primarie per l'organizzazione dei file	43
3.3.4.1	Struttura sequenziale	43
3.3.4.1.1	Struttura sequenziale disordinata	43
3.3.4.1.2	Struttura sequenziale ad array	44
3.3.4.1.3	Struttura sequenziale ordinata	44
3.3.4.1.4	Indice	45
3.3.4.2	Struttura ad albero	48
3.3.4.2.1	Struttura	48
3.3.4.2.2	Vincoli di riempimento	49
3.3.4.2.3	Operazione: ricerca con chiave K	50
3.3.4.2.4	Operazione: inserimento	51
3.3.4.2.5	Operazione: cancellazione	52
3.3.4.3	Struttura ad accesso calcolato (hash)	53
3.3.4.3.1	Operazione: ricerca con chiave K	54
3.3.4.3.2	Collisioni	54
4	Esecuzione concorrente di transazioni	56
4.1	Tipologie di anomalie	56
4.1.1	Perdita di aggiornamento	57
4.1.2	Lettura inconsistente	57
4.1.3	Lettura sporca	58
4.1.4	Aggiornamento fantasma	58
4.1.5	Inserimento fantasma	59
4.2	Schedule	59
4.2.1	Schedule seriale	60

4.3	Tecniche di gestione della concorrenza	60
4.3.1	View-equivalenza	60
4.3.2	Conflict-equivalenza	65
4.3.3	Locking a due fasi stretto	70
4.3.4	Blocco critico (deadlock)	72
4.3.5	Gestione della concorrenza in SQL	73
5	Ottimizzazione di interrogazioni	74
5.1	Ottimizzazione dipendente dai metodi di accesso	74
5.1.1	Scansione sequenziale	75
5.1.2	Ordinamento	75
5.1.3	Accesso diretto tramite indice	76
5.1.4	Join	77
5.1.4.1	Nested loop join	77
5.1.4.2	Nested loop join con indice	78
5.1.4.3	Block nested loop join	79
5.1.4.4	Merge scan join	80
5.1.4.5	Hash-based join	80
5.2	Scelta del piano esecuzione	81
6	Laboratorio	82
6.1	Comando CREATE TABLE	82
6.1.1	Identificatori	83
6.1.2	Domini elementari	83
6.1.2.1	Tipo carattere	83
6.1.2.2	Tipo boolean	84
6.1.2.3	Tipo numerico esatto	84
6.1.2.4	Tipo numerico approssimato	85
6.1.2.5	Istanti temporali	85
6.1.2.6	Intervalli temporali	85
6.1.3	Domini definiti dall'utente	86
6.1.4	Vincoli intrarelazionali	86
6.1.5	Vincoli interrelazionali	87
6.2	Modifica degli schemi	88
6.2.1	Comando ALTER TABLE	89
6.3	Comando DELETE e DROP TABLE	89
6.4	Comando INSERT INTO e UPDATE	90
6.5	Selezione	90
6.5.1	Clausola SELECT	91
6.5.1.1	Target List	91
6.5.1.2	Concatenazione stringhe	91
6.5.2	Clausola FROM	92
6.5.2.1	Uso di alias	92
6.5.2.2	Target List	93
6.5.3	Clausola WHERE	93
6.5.3.1	Operatore LIKE	93
6.5.3.2	Operatore SIMILAR TO	94
6.5.3.3	Operatore BETWEEN	94
6.5.3.4	Operatore IN	94

	6.5.3.5 Operatore IS NULL	95
	6.5.3.6 Operatore ORDER BY	95
	6.5.3.7 Operatori di aggregazione	95
6.6	Esempi di interrogazioni	96
6.6.1	Interrogazione con raggruppamento: Group by	96
6.6.2	Interrogazione con JOIN	98
6.6.3	Interrogazioni con LEFT OUTER JOIN	98
6.6.4	Interrogazioni con RIGHT OUTER JOIN	98
6.6.5	Interrogazioni con FULL OUTER JOIN	99
6.7	Interrogazioni nidificate e insiemistiche	99
6.7.1	Clausola FROM	99
6.7.2	Clausola WHERE	99
	6.7.2.1 Operatore ANY/SOME	100
	6.7.2.2 Operatore ALL	100
	6.7.2.3 Operatore IN	101
	6.7.2.4 Operatore EXIST	101
6.7.3	Le viste	104
6.8	Indici	105
6.8.1	Creazione indice	106
6.8.2	Caso pratico	107
6.8.3	Tipi di indice	108
	6.8.3.1 B-tree	108
	6.8.3.2 Hash	108
	6.8.3.3 Multi-attributo	108
	6.8.3.4 Indici di espressioni	109
6.8.4	Costo e Pratiche d'uso	109
	6.8.4.1 Explain con due nodi	110
	6.8.4.2 Explain con due nodi – AND con un solo indice	111
	6.8.4.3 Esempi d'uso	111
6.8.5	Explain versione avanzata	113
6.9	Introduzione al controllo di concorrenza in SQL	114
6.9.1	Transazioni	114
	6.9.1.1 Controllo concorrenza funzionamento	115
6.9.2	Livelli di isolamento	116
	6.9.2.1 Sintassi	117
	6.9.2.2 Livello di isolamento: Read Committed	117
	6.9.2.3 Livello di isolamento Repeatable Read	119
	6.9.2.4 Livello di isolamento: Repeatable Read	120
	6.9.2.5 Serializable: avvertenze	121
	6.9.2.6 Cosa fare in caso di failure:	121
6.9.3	Lock Espliciti	122

Tecnologie per le basi di dati

Definizione: DBMS

Un **Database Management System** è un software in grado di gestire collezioni di dati che siano **grandi**, **condivise** e **persistenti**, assicurando la loro **affidabilità** e **privatezza**.

- ▷ **grandi**: dati devono essere organizzati in modo efficiente per poter essere gestiti in grandi quantità
- ▷ **condivise**: dati devono essere accessibili da più utenti contemporaneamente
- ▷ **persistenti**: dati devono essere memorizzati in modo permanente, anche dopo la chiusura del programma
- ▷ **affidabili**: dati devono essere protetti da errori e guasti del sistema
- ▷ **privati**: dati devono essere protetti da accessi non autorizzati

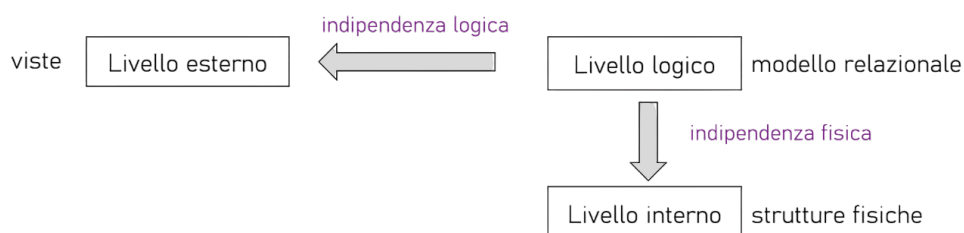
Un DBMS in quanto sistema software deve essere **efficiente** ed **efficace** e deve, inoltre, estendere le funzionalità del file system.

Definizione: base di dati

Una **base di dati** è una collezione di dati gestita da un DBMS.

1.1 Architettura transazionale di un DBMS

Un DBMS è costituito da tra 3 livelli:



Un DBMS basato sul **modello relazionale** è nella maggior parte dei casi anche un sistema transazionale: fornisce un meccanismo per la definizione ed esecuzione di transazioni.

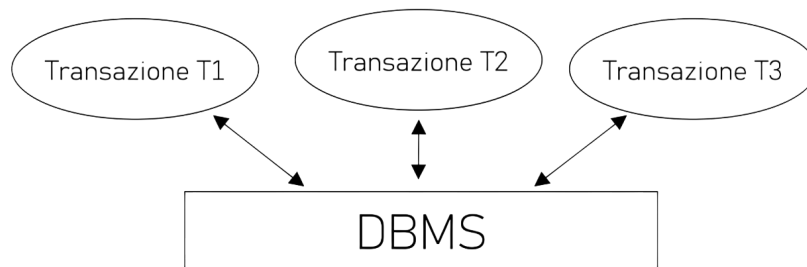


Figura 1.1: sistema transazionale

1.2 Transazione

Definizione: Transazione

Una **transazione** è un'unità di lavoro svolta da un programma applicativo (che interagisce con una base di dati) per la quale si vogliono garantire proprietà di correttezza, robustezza e isolamento.

Le caratteristiche principali sono:

- ▷ **atomicità**: una transazione deve essere eseguita completamente o non essere eseguita affatto
- ▷ **non sono ammesse esecuzioni parziali**

1.2.1 Sintassi

La sintassi per definire una transazione in SQL:

```
1 <transazione> -> begin transaction
2                   <programma>
3                   end transaction
4
5 <programma> -> < <istruzione> | commit work | rollback work >
6               { < <istruzione> | commit work | rollback work > }
```

La transazione va a buon fine all'esecuzione di un `commit work`, ma non ha alcun effetto se viene eseguito un `rollback work`.

Per essere ben formata, una transazione deve:

- **iniziare** con un `begin transaction`
- **terminare** con un `end transaction`
- **esecuzione** comportare il raggiungimento di un `commit` o di un `rollback work`, a seguito del quale non si eseguono altri accessi alla base di dati.

```
1 begin transaction;  
2   update CONTO set saldo = saldo - 1200  
3   where filiale = '005' and numero = 15;  
4   update CONTO set saldo = saldo + 1200  
5   where filiale = '005' and numero = 205;  
6   commit work;  
7 end transaction;
```

1.2.2 Proprietà delle transazioni

Una transazione ha 4 proprietà fondamentali:

1. **atomicità**: una transazione è un'unità di esecuzione indivisibile (viene eseguita completamente o non viene eseguita affatto).

Questo implica:

- ◇ se una transazione viene interrotta prima del `commit`, il lavoro fin qui eseguito deve essere disfatto ripristinando la situazione in cui si trovava la base di dati prima dell'inizio della transazione
- ◇ se una transazione viene interrotta dopo l'esecuzione del `commit` (`commit` eseguito con successo), il sistema deve assicurare che la transazione abbia effetto sulla base di dati

2. **consistenza**: esecuzione di una transazione non deve violare i vincoli di integrità.

Questo implica:

- ◇ verifica immediata:
 - fatta nel corso della transazione
 - viene abortita solo l'ultima operazione e il sistema restituisce all'applicazione una segnalazione d'errore
 - applicazione può reagire alla violazione
- ◇ verifica differita:
 - al `commit`, se un vincolo di integrità viene violato, la transazione viene abortita senza possibilità da parte dell'applicazione di reagire alla violazione

3. **isolamento**: esecuzione di una transazione deve essere indipendente dalla contemporanea esecuzione di altre transazioni

- ◇ `rollback` di una transazione non deve creare `rollback` a catena di altre transazioni che si trovano in esecuzione contemporaneamente
- ◇ sistema deve regolare l'esecuzione concorrente con meccanismi di controllo dell'accesso alle risorse

4. **persistenza**: effetto di una transazione che ha eseguito il `commit` non deve andare perso

- ◇ sistema deve essere in grado, in caso di guasto, di garantire gli effetti delle transazioni che al momento del guasto avevano già eseguito un `commit`

Un DBMS che gestisce transazioni dovrebbe garantire per ogni transazione che esegue tutte queste proprietà.

1.3 Architettura di un DBMS

L'architettura di un DBMS è composta da **moduli** che si occupano di gestire le diverse funzionalità del sistema.

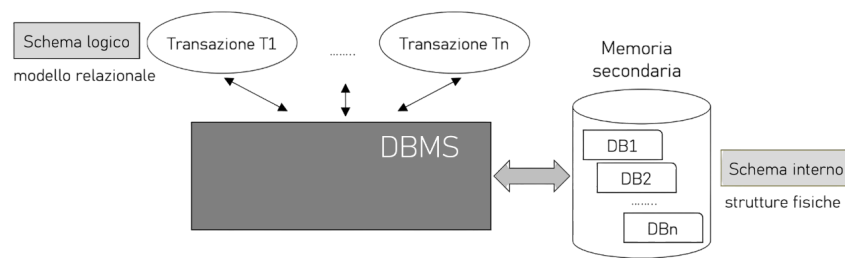


Figura 1.2: architettura di riferimento di un DBMS

Il modulo di gestione delle transazioni si occupa di trasformare una transazione in un insieme di operazioni sulla base di dati, garantendo le proprietà ACID.

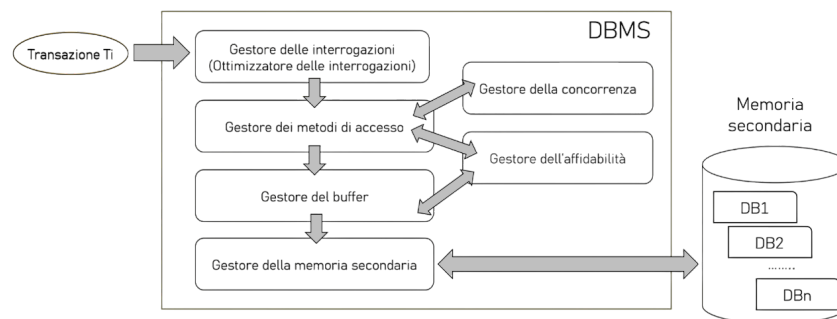
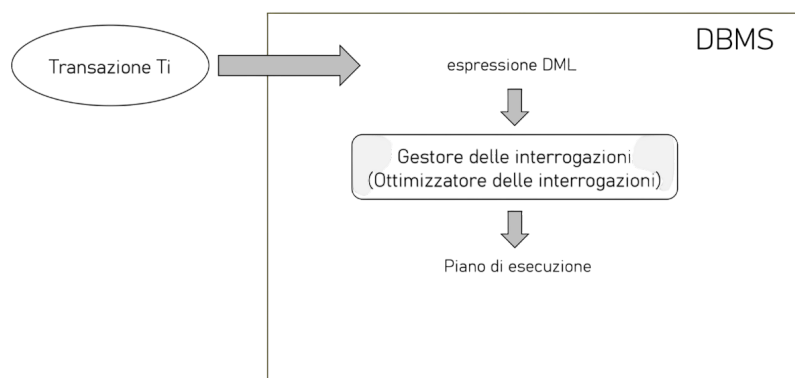


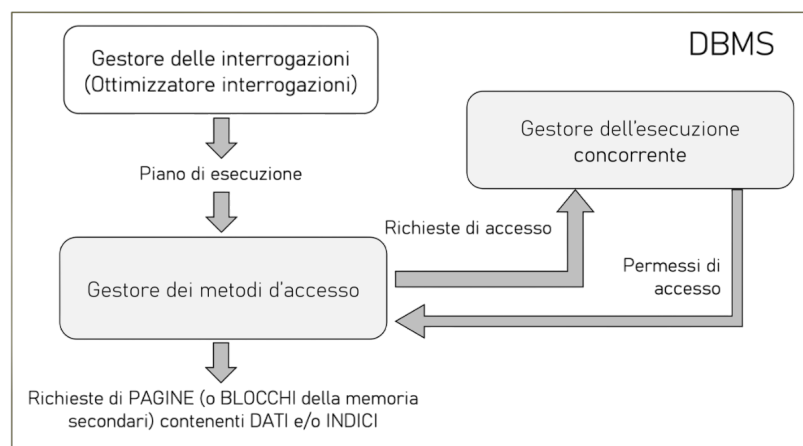
Figura 1.3: modulo di gestione delle transazioni di un DBMS

I singoli moduli che compongono questo modulo sono:

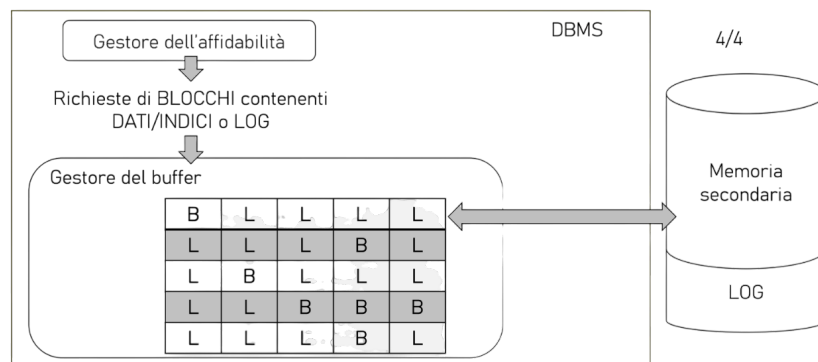
- ▷ **gestore delle interrogazioni:** riceve in ingresso una query SQL



- ▷ **gestore dei metodi di accesso:** determina quali sono le pagine della memoria secondaria di cui si ha bisogno, ma capisce anche se è possibile accederci



- ▷ **gestore dell'affidabilità**: carica in memoria principale le pagine della memoria secondaria di cui si ha bisogno e le mantiene in memoria finché non sono più necessarie



- ▷ **gestore del buffer**
- ▷ **gestore dell'esecuzione concorrente**
- ▷ **gestore dell'affidabilità**

I moduli che contribuiscono a garantire le proprietà ACID sono:

- **gestore dei metodi d'accesso**: consistenza
- **gestore dell'esecuzione concorrente**: atomicità e isolamento
- **gestore dell'affidabilità**: atomicità e persistenza

Linguaggio SQL

Il termine SQL in origine era l'acronimo di **S**tructured **Q**uery **L**anguage, ma a oggi è diventato un nome proprio.

Definizione: linguaggio SQL

Il linguaggio di interrogazione SQL permette di:

- ▷ **definire delle strutture dati e i vincoli di integrità**: **D**ata **D**efinition **L**anguage
- ▷ **manipolare dati** (inserimento, aggiornamento e cancellazione): **D**ata **M**anipulation **L**anguage
- ▷ **interrogare**: **D**ata **Q**uery **L**anguage

Esistono diverse versioni:

Nome informale	Nome ufficiale	Caratteristiche
SQL base	SQL-86	Costrutti base
	SQL-89	Integrità referenziale
SQL-2	SQL-92	Modello relazionale Vari costrutti nuovi 3 livelli: entry, intermediate, full
SQL-3	SQL:1999	Modello relazionale a oggetti Organizzato in diverse parti Trigger, funzioni esterne, ...
	SQL:2003	Estensioni del modello a oggetti Eliminazione di costrutti non usati Nuove parti: SQL/JRT, SQL/XML, ...
	SQL:2006	Estensione della parte XML
	SQL:2008	Lievi aggiunte (per esempio, trigger instead of)
	SQL:2011	Ricco supporto per dati temporali
	SQL:2016	Gestione del formato JSON

La parte di SQL dedicata alle dichiarazioni fa parte dei DML.

Le dichiarazioni vengono espresse in modo dichiarativo: viene specificato l'obiettivo e non la procedura per ottenerlo (algebra relazionale).

Una parte del DBMS, detta query optimizer, traduce l'interrogazione SQL nel linguaggio procedurale interno del sistema, che è nascosto all'utente.

SQL permette, quindi, di descrivere le interrogazioni in modo astratto e ad alto livello.

2.1 Clausola SELECT

La sintassi dell'istruzione SELECT è:

```
1 SELECT [DISTINCT]
2 [ * | expression [[ AS ] output_name ] [, ...] ]
3 [ FROM from_item [, ...] ]
4 [ WHERE condition ]
5 [ GROUP BY grouping_element [, ...] ]
6 [ HAVING condition [, ...] ]
7 [ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
8 [ ORDER BY expression [ ASC | DESC | USING operator ] ]
```

dove:

- ▷ expression: espressione che determina un attributo
- ▷ from_item: espressione che determina una sorgente per gli attributi
- ▷ condition: espressione booleana per selezionare i valori degli attributi
- ▷ grouping_element: espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato

La sintassi dell'istruzione è:

```
1 SELECT attributo
2 FROM tabella
3 [WHERE condizione]
```

SELECT, FROM e WHERE sono chiamate **clausole**.

La clausola SELECT estrae dal prodotto cartesiano delle tabelle, elencate nella clausola FROM, le righe che soddisfano la condizione specificata nella clausola WHERE.

Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola FROM ed, eventualmente, filtrata dalla clausola WHERE, le cui colonne dipendono dalla lista degli attributi nella clausola SELECT.

Interrogazione con SELECT

Consideriamo la seguente base di dati:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁: estrarre lo stipendio degli impiegati di cognome 'Rossi'

```
1 SELECT Stipendio
2 FROM Impiegato
3 WHERE Cognome = 'Rossi';
```

Il risultato dell'interrogazione è la seguente tabella:

Stipendio
45
80

Il simbolo * permette di indicare tutti gli attributi delle tabelle.

```
1 SELECT *
2 FROM tabella;
```

Interrogazione con SELECT *

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₂: estrarre lo stipendio degli impiegati di cognome 'Rossi'

```

1 SELECT *
2 FROM Impiegato
3 WHERE Cognome = 'Rossi';

```

Il risultato dell'interrogazione è la seguente tabella:

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano

2.2 Clausola FROM

La clausola FROM specifica le tabelle da cui estrarre i dati.

La sintassi dell'istruzione è:

```

1 FROM from_item [, ...]

```

dove from_item può essere una delle seguenti clausole:

- ▷ table_name [[AS] alias [(column_alias [, ...])]]: se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle.
Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati antepoendo il nome della tabella e un '.' (es. nomeTabella.nomeAttributo)
- ▷ (other_select) [AS] nomeRisultato [(column_alias [, ...])]: è una SELECT innestata.
Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
- ▷ from_item [NATURAL] join_type from_item [ON condition]: è la clausola JOIN.
Permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle.
join_type specifica il tipo di join

Interrogazione con FROM

Consideriamo le seguenti tabelle:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Q₃: estrarre i nomi e il cognome degli impiegati e le città in cui lavorano

```
1 SELECT Impiegato.Nome, Impiegato.Cognome, Dipartimento.Città
2 FROM Impiegato, Dipartimento
3 WHERE Impiegato.Dipart = Dipartimento.Nome
```

Il risultato dell'interrogazione è la seguente tabella:

Impiegato.Nome	Impiegato.Cognome	Dipartimento.Città
Mario	Rossi	Milano
Carlo	Bianchi	Torino
Giovanni	Verdi	Milano
Franco	Neri	Roma
Carlo	Rossi	Milano
Lorenzo	Gialli	Milano
Paola	Rosati	Milano
Marco	Franco	Torino

2.3 Clausola WHERE

La sintassi dell'istruzione è:

```
1 WHERE condition
```

La clausola WHERE ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori AND, OR e NOT:

- AND: seleziona le righe in cui tutti i predicati sono veri
- OR: seleziona le righe in cui almeno un predicato è vero
- NOT: seleziona le righe in cui il predicato è falso

Ciascun predicato semplice usa gli operatori =, <> (<> ⇔ ≠), <, >, <=, >= per confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante oppure un'altra espressione.

Interrogazione con WHERE

Consideriamo le seguenti tabelle:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Q₄: estrarre i nomi e il cognome degli impiegati che lavorano nell'ufficio 20 del dipartimento Amministrazione

```

1 SELECT Nome, Cognome
2 FROM Impiegato
3 WHERE Ufficio = 20 AND Dipart = 'Amministrazione';

```

Il risultato dell'interrogazione è la seguente tabella:

Nome	Cognome
Giovanni	Verdi

2.3.1 Operatore LIKE

Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore LIKE che consente di verificare se una stringa rispetta un certo "pattern" (oppure ILIKE per **case insensitive**).

I caratteri speciali per il confronto sono:

- ◇ `_`: confronto con un carattere arbitrario
- ◇ `%`: confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari

Interrogazione con LIKE

Consideriamo le seguenti tabelle:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Q₅: estrarre gli impiegati che hanno un cognome con una "o" in seconda posizione e finisce con una "i"

```

1 SELECT *
2 FROM Impiegato
3 WHERE Cognome LIKE '_o%i';

```

Il risultato dell'interrogazione è la seguente tabella:

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Amministrazione	75	40	Venezia

2.3.2 Operatore SIMILAR TO

L'operatore `SIMILAR TO` è un `LIKE` più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).

La sintassi delle espressioni regolari è:

- `_`: carattere qualsiasi
- `%`: 0 o più caratteri qualsiasi
- `*`: ripetizione del precedente match 0 o più volte
- `+`: ripetizione del precedente match 1 o più volte
- `n,m`: ripetizione del precedente match almeno n e non più di m volte
- `[...]`: elenco di caratteri ammissibili

Interrogazione con SIMILAR TO

Consideriamo la seguente tabella:

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Q₆: estrarre gli studenti con cognome che inizia con 'A' o 'B', o 'D', o 'N' e finisce con 'a'

```
1 SELECT cognome, nome, citta
2 FROM Studente
3 WHERE cognome SIMILAR TO '[ABDN]{1}%a';
```

Il risultato dell'interrogazione sono le righe evidenziate in giallo.

2.3.3 Operatore BETWEEN

L'operatore BETWEEN consente di verificare se un valore è compreso tra due valori estremi (inclusi).

La sintassi dell'istruzione è:

```
1 condition BETWEEN value1 AND value2
```

Interrogazione con BETWEEN

Consideriamo la seguente tabella:

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Q₇: estrarre tutti gli studenti che hanno matricola tra 'IN0002' e 'IN0004'

```
1 SELECT cognome, nome, citta
2 FROM Studente
3 WHERE matricola BETWEEN 'IN0002' AND 'IN0004';
```

Il risultato dell'interrogazione sono le righe evidenziate in giallo.

2.3.4 Operatore IN

Nella clausola WHERE può apparire l'operatore [NOT] IN per testare l'appartenenza di un valore ad un insieme.

La sintassi dell'istruzione è:

```
1 WHERE attributo [NOT] IN (valore, [, ...]);
```

Interrogazione con IN

Consideriamo la seguente tabella:

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Q₈: estrarre tutti gli studenti che hanno matricola nell'elenco 'IN0001', 'IN0003' e 'IN0005'

```
1 SELECT cognome, nome, matricola
2 FROM Studente
3 WHERE matricola IN ('IN0001', 'IN0003', 'IN0005');
```

Il risultato dell'interrogazione sono le righe evidenziate in giallo.

2.3.5 Operatore IS NULL

Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato IS NULL che ritorna true se l'attributo ha valore nullo (oppure IS NOT NULL):

- ▷ IS NULL: ritorna true se l'attributo ha valore nullo
- ▷ IS NOT NULL: ritorna true se l'attributo ha valore diverso da nullo

La sintassi dell'istruzione è:

```
1 WHERE attributo IS [NOT] NULL;
```

SQL-2 implementa la logica a 3 valori:

- ◇ predicato semplice restituisce il valore NULL quando un qualsiasi termine del predicato ha valore nullo
- ◇ predicato IS NULL è un'eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL

Interrogazione con IS NULL

Consideriamo la seguente tabella:

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

Q₉: estrarre tutti gli studenti che non hanno una città

```
1 SELECT cognome, nome, citta
2 FROM Studente
3 WHERE citta IS NULL;
```

Il risultato dell'interrogazione sono le righe evidenziate in giallo.

2.4 Clausola DISTINCT

La rimozione delle righe duplicate è un'operazione costosa, perciò, SQL le mantiene di default. Per togliere i duplicati si può utilizzare la parola chiave DISTINCT dopo SELECT:

```
1 SELECT DISTINCT colonna
2 FROM tabella
3 WHERE condizione;
```


Interrogazione con DISTINCT

Consideriamo la seguente tabella:

Persona	CodFiscale	Nome	Cognome	Città
	RSSMRA55B21T234J	Mario	Rossi	Verona
	BNCCLR69T30H745Z	Carlo	Bianchi	Roma
	RSSGNN41A31B344C	Giovanni	Rossi	Verona
	RSSPRT75C12F205V	Pietro	Rossi	Milano

Q₁₀: estrarre le città delle persone in cui il cognome è Rossi

```
1 SELECT Città
2 FROM Persona
3 WHERE Cognome = 'Rossi';
```

L'output di questa interrogazione è la tabella:

Città
Verona
Verona
Milano

Per togliere i duplicati si può utilizzare la parola chiave DISTINCT dopo SELECT:

```
1 SELECT DISTINCT Città
2 FROM Persona
3 WHERE Cognome = 'Citta';
```

L'output di questa interrogazione è la tabella:

Città
Verona
Milano

2.5 Clausola JOIN

In SQL, le interrogazioni con la clausola JOIN si possono definire in due modi:

- ▷ **senza l'uso di nuova sintassi:** specifica il legame tra le tabelle coinvolte a livello della clausola WHERE
- ▷ **con l'uso di sintassi riservata** a livello di clausola FROM

La clausola FROM ammette come argomento:

```
1 tabella [[AS] name] [, ...]
```

Si è visto che se sono presenti due o più nomi di tabelle si esegue il prodotto cartesiano tra tutte le tabelle e lo schema del risultato può contenere tutti gli attributi del prodotto cartesiano.

Il prodotto cartesiano di due o più tabelle è un `CROSS JOIN`.

A partire da SQL-2, esistono altri tipi di `JOIN`:

- `INNER JOIN`: restituisce solo le righe che soddisfano la condizione di join
- `LEFT [OUTER] JOIN`: restituisce
 - ◊ tutte le righe della tabella di destra e sinistra che soddisfano la condizione di join
 - ◊ `null` per le righe della tabella di destra che non soddisfano la condizione di join
- `RIGHT [OUTER] JOIN`: restituisce
 - ◊ tutte le righe della tabella di destra e sinistra che soddisfano la condizione di join
 - ◊ `null` per le righe della tabella di sinistra che non soddisfano la condizione di join
- `FULL [OUTER] JOIN`: restituisce
 - ◊ tutte le righe di entrambe le tabelle
 - ◊ `null` per le righe che non soddisfano la condizione di join

In generale, la sintassi dell'istruzione è:

```
1 tabella [NATURAL] join_type table_name [ ON condition [, ...]]
```

▷ `condition`: espressione booleana che seleziona le tuple del join da aggiungere al risultato

Le tuple selezionate possono essere poi filtrate con la condizione della clausola `WHERE`.

Prodotto Cartesiano

► Formulazione con `JOIN`:

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I
      CROSS JOIN Reparto AS R;
```

► Formulazione con `'`:

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I, Reparto AS R;
```

cognome	nome
Rossi	Acquisti
Verdi	Acquisti
Rossi	Vendite
Verdi	Vendite

2.5.1 Clausola `INNER JOIN`

La sintassi dell'istruzione è:

```
1 tabella [ INNER ] JOIN tabella ON join_condition [, ...]
```

Rappresenta il tradizionale θ join dell'algebra relazionale.

Combina ciascuna riga r_1 di $table_1$ con ciascuna riga di $table_2$ che soddisfa la condizione della clausola `ON`.

Interrogazione con INNER JOIN

```

1 SELECT I.cognome, R.nomeRep, R.sede
2 FROM Impiegato AS I
3 INNER JOIN Reparto AS R ON I.nomeRep = R.nomeRep;

```

cognome	nomerep	sede
Rossi	Acquisti	Verona
Verdi	Vendite	Verona

2.5.2 Clausola LEFT OUTER JOIN

La sintassi dell'istruzione è:

```

1 tabella LEFT [ OUTER ] JOIN tabella ON join_condition [, ...]

```

Si esegue un INNER JOIN.

Poi, per ciascuna riga r_1 di $table_1$ che non soddisfa la condizione con qualsiasi riga di $table_2$, si aggiunge una riga al risultato con i valori di r_1 e si assegna NULL agli altri attributi.

Interrogazione con LEFT [OUTER] JOIN

```

1 INSERT INTO Reparto (nomerep, sede, telefono)
2 VALUES ('Finanza', 'Padova', '02 8028888');
3
4 SELECT R.nomeRep, I.cognome
5 FROM Reparto AS R
6 LEFT OUTER JOIN Impiegato AS I ON I.nomerep = R.nomerep;

```

nomerep	cognome
Acquisti	Rossi
Vendite	Verdi
Finanza	

Nota

Il LEFT OUTER JOIN non è simmetrico!

Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

2.5.3 Clausola RIGHT OUTER JOIN

La sintassi dell'istruzione è:

```
1 tabella RIGHT [ OUTER ] JOIN tabella ON join_condition [, ...]
```

Interrogazione con RIGHT [OUTER] JOIN

```
1 SELECT I.cognome , R.nomeRep
2 FROM Impiegato I
3 RIGHT OUTER JOIN Reparto AS R ON I. nomerep = R. nomerep;
4
```

nomerep	cognome
Acquisti	Rossi
Vendite	Verdi
Finanza	

2.5.4 Clausola FULL OUTER JOIN

La sintassi dell'istruzione è:

```
1 tabella FULL [ OUTER ] JOIN tabella ON join_condition [, ...]
```

È equivalente a:

```
1 INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN
```

Non è equivalente a CROSS JOIN.

2.6 Clausola AS

SQL permette di associare un nome alternativo (alias) alle tabelle che compaiono nella clausola FROM.

Il nome viene usato per far riferimento alla tabella nel contesto dell'interrogazione per diversi motivi:

- ▷ far riferimento alla tabella in modo compatto
- ▷ fare accesso più volte alla stessa tabella

Interrogazione con utilizzo di AS

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₁: estrarre tutti gli impiegati che hanno lo stesso cognome, ma nome diverso del dipartimento Produzione

```

1 SELECT I1.Cognome, I2.Cognome
2 FROM Impiegato I1, Impiegato I2
3  WHERE I1.Cognome = I2.Cognome AND
4        I1.Nome <> I2.Cognome AND
5        I2.Dipart = 'Produzione';

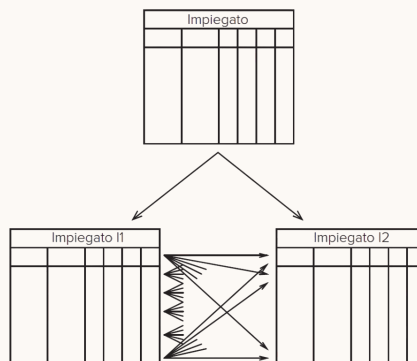
```

Questa interrogazione confronta ciascuna riga di Impiegato con tutte le righe di Impiegato associate al dipartimento Produzione.

Si può immaginare che al momento della definizione dell'alias vengano create due copie della tabella Impiegato, ciascuna associata ad una delle due variabili I1 e I2.

Le due copie contengono tutte le righe di Impiegato.

Sulla copia I2 vengono selezionate solo le righe del dipartimento 'Produzione'.



2.7 Clausola ORDER BY

Una relazione è un insieme di tuple non ordinato, però, sorge spesso il bisogno di costruire un ordine sulle righe.

SQL permette di specificare un ordinamento (ascendente o discendente) sulle righe dell'output grazie alla clausola ORDER BY che chiude l'interrogazione.

La sintassi dell'interrogazione è:

```

1 ORDER BY attributo [{ ASC | DESC }] [, ... ]

```

Le righe vengono ordinate per il primo attributo nell'elenco, a parità di valore del primo attributo si usa il secondo e così via in sequenza.

Di default l'ordinamento è ASC.

Interrogazione con ORDER BY

Consideriamo la seguente tabella:

Persone		
Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filippo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

Q₁₂: estrarre il nome, l'età e il reddito delle persone con meno di 30 anni in ordine alfabetico

```

1 SELECT Nome, Eta, Reddito
2 FROM Persone
3 WHERE Eta < 30
4 ORDER BY Eta;
```

Interrogazione con ORDER BY

Consideriamo la seguente tabella:

Automobile	Targa	Marca	Modello	NroPatente
	KB 574 WW	Fiat	Punto	VR 2030020Y
	GA 652 FF	Fiat	Panda	VR 2030020Y
	BJ 747 XX	Lancia	Ypsilon	PZ 1012436B
	ZB 421 JJ	Fiat	Uno	MI 2020030U

Q₁₃: estrarre il contenuto della tabella Automobile ordinato in base alla marca (discendente) e al modello

```

1 SELECT *
2 FROM Automobile
3 ORDER BY Marca DESC, Modello;
```

L'output di questa interrogazione è la tabella:

Targa	Marca	Modello	NroPatente
BJ 747 XX	Lancia	Ypsilon	PZ 1012436B
GA 652 FF	Fiat	Panda	VR 2030020Y
KB 574 WW	Fiat	Punto	VR 2030020Y
ZB 421 JJ	Fiat	Uno	MI 2020030U

2.8 Operazioni insiemistiche

SQL mette a disposizione degli operatori insiemistici:

```
1 SELECT
2   { UNION | INTERSECT | EXCEPT
3   [ALL] SELECT }
```

Gli operatori insiemistici, al contrario del resto del linguaggio, eseguono di default l'eliminazione dei duplicati, a meno che si usi ALL.

Sono sufficienti domini compatibili e lo stesso numero di attributi.

La corrispondenza si basa sulla posizione e se i nomi degli attributi sono diversi di solito il sistema usa il nome del primo operando.

2.8.1 Operatore UNION

Interrogazione con UNION

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₄: estrarre i nomi e i cognomi degli impiegati

```
1 SELECT Nome
2 FROM Impiegato
3 UNION
4 SELECT Cognome
5 FROM Impiegato
```

L'output di questa interrogazione è:

Nome
Mario
Carlo
Giovanni
Franco
Lorenzo
Paola
Marco
Rossi
Bianchi
Verdi
Neri
Gialli
Rosati

Interrogazione con UNION

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₅: estrarre i nomi e i cognomi degli impiegati eccetto quelli del dipartimento Amministrazione mantenendo i duplicati

```

1 SELECT Nome
2 FROM Impiegato
3 WHERE Diaprt <> 'Amministrazione'
4 UNION ALL
5 SELECT Cognome
6 FROM Impiegato
7 WHERE Diaprt <> 'Amministrazione';

```

L'output di questa interrogazione è:

Nome
Carlo
Franco
Carlo
Lorenzo
Marco
Bianchi
Neri
Rossi
Gialli
Franco

2.8.2 Operatore INTERSECT

Interrogazione con INTERSECT

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₆: estrarre i cognomi degli impiegati che sono anche nomi

```
1 SELECT Nome
2 FROM Impiegato
3 INTERSECT
4 SELECT Cognome
5 FROM Impiegato;
```

L'output di questa interrogazione è:

Nome
Franco

2.8.3 Operatore EXCEPT

Interrogazione con differenza

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₇: estrarre i nomi degli impiegati che non sono cognomi di qualche impiegato

```

1 SELECT Nome
2 FROM Impiegato
3 EXCEPT
4 SELECT Cognome
5 FROM Impiegato;
```

L'output di questa interrogazione è:

Nome
Mario
Carlo
Giovanni
Lorenzo
Paola
Marco

2.9 Operatori aggregati

Gli operatori di aggregazione permettono di valutare proprietà che dipendono da insiemi di tuple.

2.9.1 Operatore COUNT

L'operatore COUNT conta il numero di righe di una tabella o il numero di valori di uno o più attributi.

La sintassi dell'interrogazione è:

```

1 COUNT ( < * | [DISTINCT | ALL] ListaAttributi > )
```

- ▷ *: conta il numero di righe della tabella
- ▷ DISTINCT ListaAttributi: conta il numero di valori diversi degli attributi ListaAttributi

- ▷ ALL ListaAttributi: conta il numero di righe che hanno valori diversi da NULL negli attributi in ListaAttributi (default)

Interrogazione con COUNT

Consideriamo la seguente tabella:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₈: estrarre il numero di impiegati del dipartimento di Produzione

```
1 SELECT COUNT(*)
2 FROM Impiegato
3 WHERE Diaprt = 'Produzione';
```

L'output di questa interrogazione è: 2.

Interrogazione con COUNT

Consideriamo la seguente tabella:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₁₉: estrarre il numero di valori diversi dell'attributo Stipendio fra tutte le righe di Impiegato

```
1 SELECT COUNT(DISTINCT Stipendio)
2 FROM Impiegato;
```

L'output di questa interrogazione è: 6.

2.9.2 Operatore SUM

L'operatore SUM restituisce la somma dei valori posseduti dall'espressione.

La sintassi dell'interrogazione è:

```
1 SUM ( < * | [DISTINCT | ALL] ListaAttributi > )
```

Interrogazione con SUM

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₂₀: estrarre la somma degli stipendi del dipartimento Amministrazione

```

1 SELECT SUM(Stipendio)
2 FROM Impiegato
3 WHERE Dipart = 'Amministrazione';

```

L'output di questa interrogazione è: 125.

2.9.3 Operatore MAX/MIN

Gli operatori MIN e MAX restituiscono rispettivamente il massimo e il minimo dei valori posseduti dall'espressione.

La sintassi dell'interrogazione è:

```

1 <MAX | MIN> ( [DISTINCT | ALL] ListaAttributi )

```

2.9.4 Operatore AVG

L'operatore AVG restituisce la media dei valori.

La sintassi dell'interrogazione è:

```

1 AVG ( [DISTINCT | ALL] ListaAttributi )

```

Interrogazione con MIN, MAX, AVG

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₂₁: estrarre gli Stipendi minimi, massimi e medi fra quelli di tutti gli impiegati

```
1 SELECT MIN(Stipendio), MAX(Stipendio), AVG(Stipendio)
2 FROM Impiegato;
```

L'output di questa interrogazione è: 36, 80, 50.625.

Nota

- ▷ SUM e AVG ammettono come argomento solo espressioni che rappresentano valori numerici o intervalli di tempo
- ▷ MAX e MIN ammettono come argomento solo espressioni su cui è definito un ordinamento (anche stringhe)
- ▷ DISTINCT elimina i duplicati
- ▷ ALL trascura solo i valori nulli

ATTENZIONE

```
1 SELECT Cognome, Nome, MAX(I.Stipendio)
2 FROM Impiegato AS I
3 JOIN Dipartimento AS D ON I.Dipart = D.Nome
4 WHERE D.Citta = 'Milano';
```

La seguente interrogazione non è corretta!

Gli operatori aggregati non sono un meccanismo di selezione, ma solo funzioni che restituiscono un valore quando sono applicate ad un insieme.

Cognome e Nome restituiscono un valore per ogni tupla selezionata, mentre MAX restituisce un valore unico per tutto l'insieme.

Per risolvere questa situazione serve il meccanismo di **raggruppamento**.

2.10 Clausola GROUP BY

Gli operatori aggregati possono essere applicati separatamente a sottoinsiemi di righe.

La clausola GROUP BY permette di specificare come dividere le tabelle in sottoinsiemi.

La clausola permette di specificare un insieme di attributi e l'interrogazione raggrupperà le righe che possiedono gli stessi valori per questo insieme di attributi.

Interrogazione con GROUP BY

Consideriamo la seguente tabella:

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Q₂₂: estrarre la somma degli stipendi di tutti gli impiegati dallo stesso dipartimento

```
1 SELECT Dipart, SUM(Stipendio)
2 FROM Impiegato
3 GROUP BY Dipart;
```

In un'interrogazione che fa uso della clausola GROUP BY può comparire come argomento della SELECT solo un sottoinsieme degli attributi usati nella clausola GROUP BY, oppure, operatori di aggregazione.

2.10.1 Clausola HAVING

È possibile applicare delle selezioni ai gruppi generati tramite la clausola GROUP BY.

La clausola HAVING permette di specificare delle condizioni che devono essere soddisfatte dai gruppi generati dopo l'esecuzione del raggruppamento tramite GROUP BY.

La sintassi dell'interrogazione è:

```
1 SELECT Dipart, SUM(Stipendio) AS SommaStipendi
2 FROM Impiegato
3 GROUP BY Dipart
4 HAVING SUM(Stipendio) > 100;
```

Dopo aver eseguito i raggruppamenti, vengono mantenuti solo i gruppi che soddisfano la condizione specificata nella clausola HAVING.

Dipart	Stipendio
Amministrazione	45
Amministrazione	40
Amministrazione	40
Produzione	36
Produzione	46
Distribuzione	45
Direzione	80
Direzione	73

1. Raggruppamento



Dipart	sum(Stipendio)
Amministrazione	125
Produzione	82
Distribuzione	45
Direzione	153

2. Calcolo operatore sui ciascun gruppo



Dipart	SommaStipendi
Amministrazione	125
Direzione	153

3. Selezione con HAVING

OSSERVAZIONE

- ▷ clausola WHERE applica delle selezioni su singole righe **prima** dell'eventuale raggruppamento tramite GROUP BY
- ▷ clausola HAVING applica delle selezioni su gruppi generati a **seguito** dell'esecuzione del raggruppamento tramite GROUP BY

```
1 SELECT Dipart
2 FROM Impiegato
3 WHERE Ufficio = 20 --seleziona righe raggruppate
4 GROUP BY Dipart --forma gruppi con righe selezionate
5 HAVING AVG(Stipendio) > 25; --seleziona gruppi del
6 risultato
7 \end{lstlisting}
```

2.11 Interrogazioni nidificate

Un'interrogazione nidificata è un'interrogazione che contiene al suo interno altre interrogazioni.

La nidificazione può avvenire sia nella clausola SELECT, che nella clausola FROM, che nella clausola WHERE.

Il caso tipico è la nidificazione nella clausola WHERE, dove un predicato contiene un confronto tra un valore (espressione su una singola riga) ed il risultato di un'interrogazione SQL.

2.11.1 Clausola WHERE

Se in un predicato si confronta un attributo o una espressione che produce un valore singolo con il risultato di un'interrogazione, sorge il problema di disomogeneità dei termini del confronto. I normali operatori di confronto (=, <>, <, >, <=, >=) possono essere estesi con le parole chiave ANY e ALL:

- ▷ ANY: riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **almeno** uno degli elementi restituiti dall'interrogazione
- ▷ ALL: riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **tutti** gli elementi restituiti dall'interrogazione

Interrogazione con WHERE

Consideriamo la seguente tabella:

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Q₂₃: estrarre i dipartimenti in cui non lavorano persone di cognome Rossi

```

1 SELECECT Nome
2 FROM Dipartimento
3   WHERE Nome <> ALL (
4     SELECT Dipart
5     FROM Impiegato
6     WHERE Cognome = 'Rossi'
7   );

```

Questo equivale a:

```

1 SELECT Nome
2 FROM Dipartimento
3 EXCEPT
4 SELECT Dipart
5 FROM Impiegato
6 WHERE Cognome = 'Rossi';

```

ATTENZIONE!

Per le interrogazioni nidificate viste fino ad ora, l'interrogazione può essere eseguita una volta sola ed il risultato ottenuto può essere salvato in una tabella temporanea usata per eseguire ciascun confronto con le righe dalla tabella esterna.

Talvolta però l'interrogazione nidificata fa riferimento al contesto dell'interrogazione che la racchiude attraverso l'uso di una variabile (**passaggio di binding**).

L'interrogazione nidificata dovrà essere **valutata separatamente per ciascuna riga** prodotta dalla valutazione della query esterna.

2.11.2 Clausola EXISTS

L'operatore logico EXISTS riceve come parametro un'interrogazione nidificata e restituisce il valore vero solo se l'interrogazione fornisce un risultato non vuoto.

Questo operatore può essere usato in modo significativo solo quando si ha un passaggio di binding tra la query esterna e quella interna.

Interrogazione con EXISTS

Consideriamo la seguente tabella:

Persona	CodFiscale	Nome	Cognome	Città
	RSSMRA55B21T234J	Mario	Rossi	Verona
	BNCCLR69T30H745Z	Carlo	Bianchi	Roma
	RSSGNN41A31B344C	Giovanni	Rossi	Verona
	RSSPRT75C12F205V	Pietro	Rossi	Milano

Q₂₄: estrarre le persone che hanno degli omonimi (stesso nome e cognome, ma codice fiscale diverso)

```

1 SELECT *
2 FROM Persona AS P
3 WHERE EXISTS (
4     SELECT *
5     FROM Persona P1
6     WHERE P1.Nome = P.Nome AND
7           P1.Cognome = P.Cognome AND
8           P1.CodFiscale <> P.CodFiscale
9 );

```

2.11.3 Clausola SELECT

Le interrogazioni nidificate possono essere specificate anche come elemento della clausola SELECT.

Nota

Spesso l'uso di query nidificate nella SELECT sostituiscono l'uso del JOIN.

Interrogazione con SELECT

Q₂₅: estrarre per ogni cantante il numero di canzoni di cui è autore

```

1 SELECT DISTINCT Nome, NumCanzoni = (
2     SELECT COUNT(*)
3     FROM Autore AS A
4     WHERE A.Nome = C.Nome
5 )
6 FROM Cantante AS C;

```

2.11.4 Clausola FROM

Le interrogazioni nidificate possono essere utilizzate nella clausola FROM come sorgente dati su cui eseguire l'interrogazione.

È possibile creare una catena di interrogazioni, dove ciascuna interrogazione utilizza il risultato della precedente come una tabella di base.

Interrogazione con FROM

Q₂₆: estrarre per ogni cantante che è anche autore di canzoni il numero di canzoni di cui è rispettivamente interprete e autore

```
1 SELECT DISTINCT C.Nome, C.NumCantate, A.NumScritte
2 FROM (
3     SELECT Nome, COUNT(*) AS NumCantate
4     FROM Cantante
5 ) AS C
6 JOIN (
7     SELECT Nome, COUNT(*) AS NumScritte
8     FROM Autore
9 ) AS A ON C.Nome = A.Nome;
```

Strutture fisiche e di accesso ai dati

Le basi di dati risiedono nella **memoria secondaria** perchè sono:

- ▷ **grandi**: non possono essere contenute completamente in memoria centrale
- ▷ **persistenti**: hanno un tempo di vita che non è limitato all'esecuzione dei programmi che le usano

Caratteristiche della memoria secondaria

Le caratteristiche principali della memoria secondaria sono:

- **non utilizzabile direttamente dai programmi**
- **dati organizzati in blocchi** (grandezza dipende dal sistema operativo)
- **operazioni possibili**:
 - ◊ lettura intero blocco
 - ◊ scrittura intero blocco
- **costo delle operazioni di accesso alla memoria secondaria è maggiore al costo per accedere ai dati in memoria centrale**

3.1 Gestore del buffer

L'interazione tra memoria secondaria e memoria centrale avviene tramite il **gestore del buffer**, mediante il trasferimento di uno o più **blocchi** della memoria secondaria in **pagine** dedicate della memoria centrale (buffer).

Precisazione

La dimensione delle pagine in memoria centrale sono solitamente un multiplo della dimensione dei blocchi in memoria secondaria.

Per semplificare, si assume che siano uguali:

$$1 \text{ pagina} \approx 1 \text{ blocco}$$

3.1.1 Buffer

Il **buffer** è una zona della memoria condivisa dalle applicazioni e una gestione ottimale del buffer consente di ottenere buone prestazioni nell'accesso ai dati.

Quando uno stesso dato viene utilizzato più volte in tempi ravvicinati il buffer evita l'accesso alla memoria secondaria.

Compito del gestore del buffer

Il gestore del buffer si occupa di caricare e salvare i dati dalla memoria secondaria alla memoria centrale.

3.1.2 Politica di gestione della memoria

L'obiettivo del gestore del buffer è di **ottimizzare gli accessi** attraverso le politiche di gestione della memoria:

◇ lettura di un blocco:

- ▷ blocco presente in una pagina del buffer
 - ⇒ non viene eseguita la lettura in memoria secondaria
 - ⇒ viene restituito un puntatore alla pagina del buffer
- ▷ blocco non presente in una pagina del buffer
 - ⇒ viene cercata una pagina libera
 - ⇒ viene caricato il blocco nella pagina
 - ⇒ viene restituito il puntatore alla pagina del buffer

- ◇ **scrittura di un blocco:** se viene effettuata una richiesta di scrittura di un blocco già caricato in una pagina del buffer, il gestore del buffer può decidere di **differire la scrittura** su memoria secondaria in un secondo momento

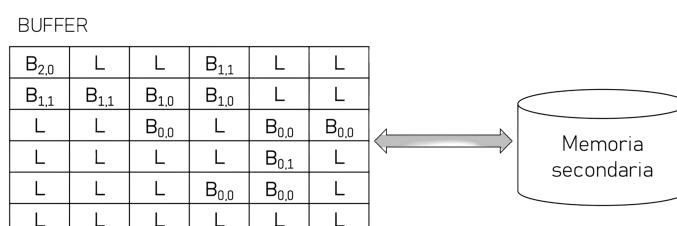
Per poter aumentare la velocità di accesso ai dati, le politiche di lettura e scrittura si basano sul **principio di località**.

Definizione: principio di località

Il **principio di località** si basa sul fatto che i dati referenziati di recente hanno maggiore probabilità di essere referenziati nuovamente in futuro.

3.1.3 Gestione delle pagine

Il contenuto del buffer lo possiamo immaginare come una matrice.



Per ogni pagina del buffer si memorizza:

- ▷ **blocco B** (L se libero) indicando il file e il numero di blocco (offset)
- ▷ **variabili di stato** ($B_{i,j}$):
 - **contatore i**: indica il numero di transazioni
 - **bit di stato j**: indica se la pagina è stata modificata (1) o no (0)

3.1.4 Primitive per la gestione del buffer

3.1.4.1 Primitiva fix

La **primitiva fix** viene utilizzata dalle transazioni per richiedere l'accesso a un blocco e restituisce un puntatore alla pagina contenente il blocco richiesto.

Le azioni della primitiva sono:

- ▷ **ricerca del blocco**:
 - ◊ blocco viene trovato
 - ⇒ viene restituito un puntatore alla pagina contenente il blocco
 - ◊ blocco non viene trovato:
 - viene scelta una pagina libera
 - ⇒ viene salvata in memoria secondaria (primitiva flush)
 - ⇒ viene caricato il blocco in P aggiornando file e numero di blocco
 - non ci sono pagine libere
 - ⇒ viene rubata una pagina a un'altra transazione (primitiva flush)
 - ⇒ viene sospesa la transazione inserendola in una coda di attesa
 - ⇒ se si libera una pagina il gestore si comporterà come al punto precedente

⇒ **effetto della modifica**: quando una transazione accede a una pagina per la prima volta il contatore si incrementa

3.1.4.2 Primitiva unfix

La **primitiva unfix** viene usata dalle transazioni per indicare che la transazione ha terminato di usare il blocco.

⇒ **effetto della modifica**: decremento del contatore i

3.1.4.3 Primitiva setDirty

La **primitiva setDirty** viene utilizzata dalle transazioni per indicare al gestore del buffer che il blocco della pagina è stato modificato.

⇒ **effetto della modifica**: bit di stato j passa da 0 a 1

3.1.4.4 Primitiva force

La **primitiva force** viene utilizzata per salvare in memoria secondaria in **modo sincrono** il blocco contenuto nella pagina.

Questa primitiva viene usata dal gestore dell'affidabilità.

⇒ **effetto della modifica**: viene salvato in memoria secondaria il blocco e il bit di stato j posto a 0

3.1.4.5 Primitiva flush

La **primitiva flush** viene utilizzata per salvare i blocchi in memoria secondaria in **modo asincrono**.

Questa primitiva viene usata dal gestore del buffer.

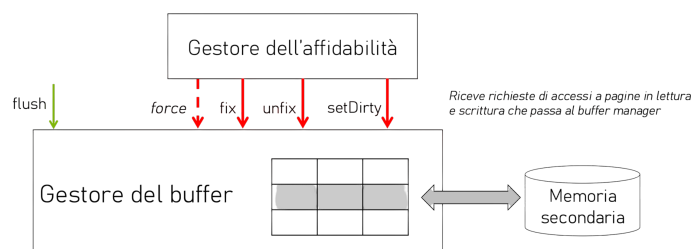
⇒ **effetto della modifica**: viene liberata la pagina "dirty" e il bit di stato j viene posto a 0

3.2 Gestore dell'affidabilità

Il gestore del buffer differisce le operazioni di scrittura e questo potrebbe rappresentare un pericolo.

Il **gestore dell'affidabilità** si occupa di garantire:

- ▷ **atomicità**: transazioni non devono essere incomplete
- ▷ **persistenza**: effetti di ciascuna transazione non conclusa con un commit devono essere mantenuti in modo permanente



3.2.1 File di LOG

Il funzionamento del gestore dell'affidabilità si basa sull'utilizzo del **file di LOG**.

Definizione: file di LOG

Il **file di LOG** è un archivio persistente su cui vengono registrate tutte le operazioni eseguite dal DBMS.

Per il corretto funzionamento, il gestore dell'affidabilità deve disporre di un dispositivo di **memoria stabile** (resistente ai guasti).

Il file di LOG viene memorizzato in memoria stabile e al suo interno vengono memorizzati:

- ▷ **record di transizione**: record che descrivono informazioni sulle transazioni
 - ◇ **B(T)**: **begin** della transazione
 - ◇ **C(T)**: **commit** della transazione
 - ◇ **A(T)**: **abort** della transazione
 - ◇ **I(T, O, AS)**: **insert** di un oggetto O da parte di una transazione T con il suo after state AS (stato dell'oggetto dopo l'operazione)

- ◇ $D(T, O, AS)$: **delete** di un oggetto O da parte della transizione T con il suo before state BS (stato dell'oggetto prima dell'operazione)
- ◇ $U(T, O, BS, AS)$: **update** di un oggetto O da parte della transizione T con il suo before state BS e il suo after state AS
- ▷ **record di sistema**: record che descrivono informazioni sulle operazioni eseguite dal DBMS
 - **DUMP**: copia completa e consistente della base di dati.
 - $CK(T_1, \dots, T_n)$: indica che all'esecuzione del checkpoint, le transazioni attive erano $T_1, \dots, T_n$
 - * elenco delle transazioni che non hanno ancora espresso la loro scelta relativamente all'esito finale
 - * operazione svolta periodicamente affinché i dati delle transazioni che hanno eseguito il commit siano registrati in modo permanente
 - * il protocollo di checkpoint prevede i seguenti passi:
 1. sospensione delle operazioni di scrittura, commit e abort delle transazioni
 2. esecuzione della primitiva force sulle pagine "dirty" di transazioni che hanno eseguito il commit
 3. scrittura sincrona (primitiva force) sul file di LOG del record di checkpoint con gli identificatori delle transazioni attive
 4. ripresa delle operazioni di scrittura, commit e abort delle transazioni

I record di transazione salvati nel LOG consentono di eseguire in caso di ripristino le seguenti operazioni:

- ▷ **UNDO**: viene usato per disfare un'azione su un oggetto O .
 È sufficiente ricopiare in O il valore di BS .
 L'insert/delete viene disfatto cancellando/inserendo O
- ▷ **REDO**: serve per rifare un'azione su un oggetto O .
 È sufficiente ricopiare in O il valore AS .
 L'insert/delete viene rifatto inserendo/cancellando O

Gli inserimenti controllano sempre l'esistenza di O prima di eseguire l'operazione.

Per queste operazioni vale la **proprietà di idempotenza**.

Definizione: proprietà di idempotenza

$$\begin{aligned} \text{UNDO}(\text{UNDO}(A)) &= \text{UNDO}(A) \\ \text{REDO}(\text{REDO}(A)) &= \text{REDO}(A) \end{aligned}$$

3.2.2 Regole di scrittura sul file di LOG

Il gestore dell'affidabilità deve rispettare le seguenti regole di scrittura sul file di LOG:

- ⇒ **regola Write Ahead Logging**: record di LOG devono essere scritti sul LOG precedente all'esecuzione delle corrispondenti operazioni sulla base di dati (garantisce la possibilità di fare sempre UNDO)
- ⇒ **regola di commit-precedenza**: record di LOG devono essere scritti sul LOG prima dell'esecuzione del commit della transazione (garantisce la possibilità di fare sempre REDO)

Queste regole consentono di salvare i blocchi delle pagine “dirty” in modo totalmente asincrono rispetto al commit delle transazioni.

3.2.3 Guasti

Una transazione T sceglie in modo atomico l'esito di `commit` nel momento in cui scrive nel file di LOG in modo sincrono (primitiva `force`) il suo record di `commit` $C(T)$.

⇒ se avviene un guasto prima della scrittura del record di `commit` → **UNDO**

⇒ se avviene un guasto dopo la scrittura del record di `commit` → **REDO**

Esistono due tipologie di guasto:

- ▷ **guasto di sistema**: perdita del contenuto della memoria centrale.

Viene risolto con la tecnica della **ripresa a caldo**:

1. accede al file di LOG e si ripercorre all'indietro il LOG fino al più recente record di checkpoint
2. si inizializza:
 - ◊ insieme UNDO con tutte le transazioni da disfare (presenti nel checkpoint)
 - ◊ insieme REDO con tutte le transazioni da rifare ($\emptyset$)
3. si ripercorre in avanti il LOG
 - per ogni record $B(T)$ incontrato si aggiunge T all'insieme UNDO
 - per ogni record $C(T)$ incontrato si sposta T dall'insieme UNDO all'insieme REDO
4. si ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni UNDO risalendo fino alla prima azione della transazione più vecchia
5. si ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

- ▷ **guasto di dispositivo**: perdita di parte o tutto il contenuto della base di dati in memoria secondaria.

Viene risolto con la tecnica della **ripresa a freddo**:

1. si accede al DUMP della base di dati e si ricopia selettivamente la parte deteriorata della base di dati
2. si accede al LOG risalendo al record di DUMP
3. si ripercorre in avanti il LOG rieseguendo tutte le operazioni relative alla parte deteriorata comprese le azioni di `commit` e `abort`
4. si applica una ripresa a caldo

Esercizio: file di LOG

Consideriamo il seguente file di LOG:

$B(T_1), B(T_2), U(T_2, O_1, B_1, A_1), I(T_1, O_2, A_2), B(T_3), C(T_1), B(T_4),$
 $U(T_3, O_2, B_3, A_3), U(T_4, O_3, B_4, A_4), CK(T_2, T_3, T_4), C(T_4), B(T_5),$
 $U(T_3, O_3, B_5, A_5), U(T_5, O_4, B_6, A_6), D(T_3, O_5, B_7), A(T_3), C(T_5),$
 $I(T_2, O_6, A_8)$ **guasto**

Che passi svolge la ripresa a caldo?

1. risale il LOG fino all'ultimo record di checkpoint $CK(T_2, T_3, T_4)$
2. inizializza UNDO e REDO

- ▷ UNDO = $\{T_2, T_3, T_4\}$ (inizializzato con le transazioni del checkpoint)
- ▷ REDO = $\emptyset$
- 3. ripercorre in avanti il LOG dal CheckPoint fino al guasto e si aggiungono o si spostano le transazioni tra UNDO e REDO
 - ▷ UNDO = $\{T_2, T_3, \cancel{T_4}, \cancel{T_5}\}$ (inizializzato con le transazioni del checkpoint)
 - ▷ REDO = $\{T_4, T_5\}$
- 4. ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni UNDO risalendo fino alla prima azione della transazione più vecchia

DELETE(O_6) (disfare insert)

$O_5 := B_7$ (disfare delete)

$O_3 := B_5$ (disfare update)

$O_2 := B_3$ (disfare update)

$O_1 := B_1$ (disfare update)

- 5. ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

$O_3 := A_4$ (rifare update)

$O_4 := A_6$ (rifare update)

Esercizio: file di LOG

Consideriamo il seguente file di LOG:

$B(T_1), B(T_2), B(T_3), I(T_1, O_1, A_1), D(T_2, O_2, B_2), B(T_4),$
 $U(T_4, O_3, B_3, A_3), U(T_1, O_4, B_4, A_4), C(T_2), CK(T_1, T_3, T_4), B(T_5), B(T_6),$
 $U(T_5, O_5, B_5, A_5), A(T_3), CK(T_1, T_4, T_5, T_6), B(T_7), C(T_4),$
 $U(T_7, O_6, B_6, A_6), U(T_6, O_3, B_7, A_7), B(T_8), A(T_7)$ **guasto**

Che passi svolge la ripresa a caldo?

1. risale il LOG fino all'ultimo record di checkpoint $CK(T_1, T_4, T_5, T_6)$
2. inizializza UNDO e REDO
 - ▷ UNDO = $\{T_1, T_4, T_5, T_6\}$ (inizializzato con le transazioni del checkpoint)
 - ▷ REDO = $\emptyset$
3. ripercorre in avanti il LOG dal checkpoint fino al guasto e si aggiungono o si spostano le transazioni tra UNDO e REDO
 - ▷ UNDO = $\{T_1, \cancel{T_4}, T_5, t_6, t_7, t_8\}$ (inizializzato con le transazioni del checkpoint)
 - ▷ REDO = $\{T_4\}$
4. ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni

UNDO risalendo fino alla prima azione della transazione più vecchia

$O_3 := B_7$ (disfare update)

$O_6 := B_6$ (disfare update)

$O_5 := B_5$ (disfare update)

$O_4 := B_4$ (disfare update)

DELETE(O_1) (disfare insert)

5. ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

$O_3 := A_3$ (rifare update)

3.3 Gestore dei metodi di accesso

Il **gestore dei metodi di accesso** esegue il piano di esecuzione prodotto dall'ottimizzatore e produce sequenze di richieste di accessi ai blocchi della base di dati presenti in memoria secondaria.

Le richieste vengono inviate al gestore del buffer che si occupa di caricare i blocchi necessari in pagine di memoria centrale.

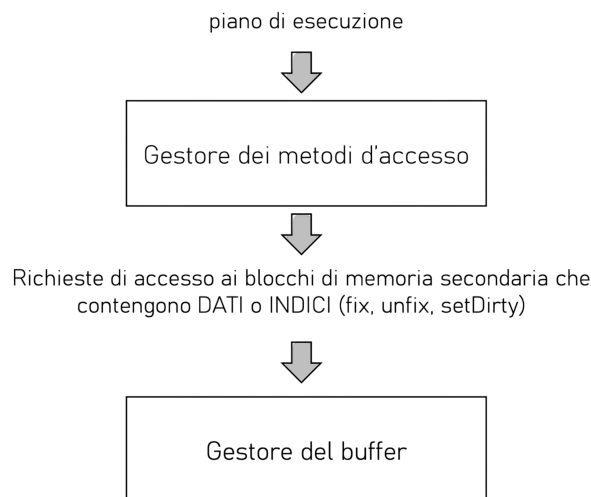


Figura 3.1: gestore dei meotodi di accesso

3.3.1 Metodi di accesso

I metodi di accesso sono **moduli software** (es. scansione sequenziale, accesso via indice, ordinamento, varie implementazioni del join) che implementano gli algoritmi di accesso e manipolazione dei dati organizzati in specifiche strutture fisiche.

Ogni metodo di accesso conosce:

- ◇ **organizzazione in tuple** (o dei record di indice) nei blocchi dati (indice) salvati in memoria secondaria
- ◇ **organizzazione fisica interna delle pagine** sia quando contengono dati, sia quando contengono strutture fisiche di accesso (indici)

3.3.2 Organizzazione di una pagina

All'interno di una pagina sono presenti:

- ▷ **informazioni utili:** dati veri e propri (tuple della tabella)
- ▷ **informazioni di controllo:** consentono di accedere alle informazioni utili (dizionario, bit di parità, altre informazioni del file system o della specifica struttura fisica)

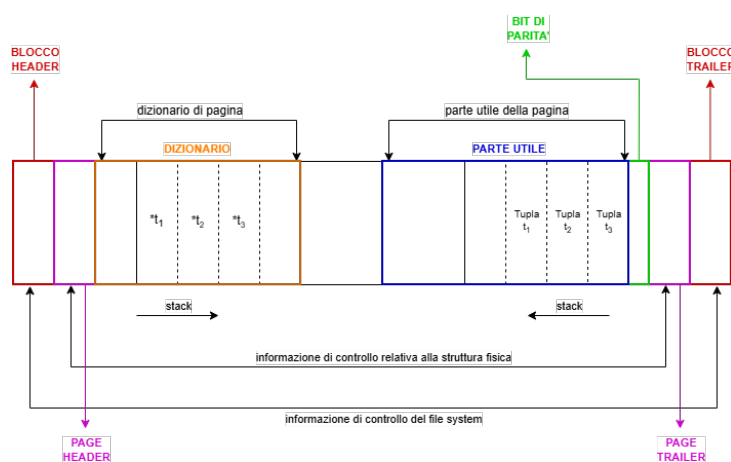


Figura 3.2: organizzazione di una pagina

- ⇒ **blocco header/trailer:** contengono informazioni di controllo utilizzate dal file system
- ⇒ **pagina header/trailer:** contengono le informazioni di controllo relative alla specifica struttura fisica
- ⇒ **dizionario di pagina:** contiene i puntatori a ciascun dato utile elementare contenuto nella pagina
- ⇒ **bit di parità:** verifica che l'informazione contenuta nella pagina sia valida
- ⇒ **parte utile:** contiene i dati

Nota

Dizionari di pagina e parte utile crescono come stack contrapposti lasciando libera la parte centrale in uno spazio contiguo.

In caso di **tuple di lunghezza fissa**, il dizionario non è necessario: si memorizza la dimensione delle tuple e l'offset del punto iniziale.

In caso di **tuple di lunghezza variabile**, il dizionario memorizza l'offset di ogni tupla presente nel blocco e di ogni attributo di ogni tupla.

Molti gestori delle pagine non consentono la separazione di una tupla su più pagine:

$$\text{dimensione massima tupla} = \text{dimensione massima area disponibile su un blocco}$$

Alcuni gestori delle pagine consentono di distribuire una tupla su più pagine, altrimenti va gestito il caso di tuple memorizzate su più pagine.

3.3.3 Operazioni sulle pagine

Le principali operazioni che si possono eseguire sulle pagine sono:

- ▷ **inserimento/aggiornamento di una tupla:**
 - ◊ esiste spazio contiguo sufficiente: inserimento
 - ◊ non esiste spazio contiguo sufficiente, ma spazio sufficiente: riorganizzazione dello spazio + inserimento
 - ◊ non esiste spazio sufficiente: interazione con il file system per allocare nuovi blocchi per il file
- ▷ **cancellazione contenuto della pagina**
- ▷ **accesso a una tupla:** avviene tramite il valore della chiave o l'offset nel dizionario
- ▷ **accesso a un attributo di una tupla:** identificato in base all'offset e alla lunghezza del campo dopo aver identificato la tupla tramite la chiave o il suo offset

3.3.4 Strutture primarie per l'organizzazione dei file

La struttura primaria di un file stabilisce il criterio di disposizione delle tuple all'interno del file presente in memoria di massa:

- ▷ **sequenziale:** colloca le tuple in maniera consecutiva sulla base di un criterio specifico (es. ordine di inserimento)
- ▷ **ad accesso calcolato:** colloca le tuple in posizioni determinate sulla base dell'esecuzione di un algoritmo di hash
- ▷ **ad albero:** utilizzate come strutture secondarie (indici)

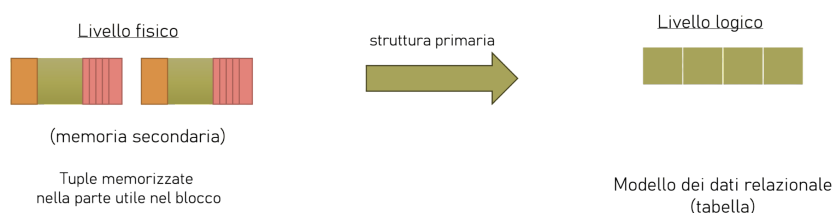


Figura 3.3: strutture primarie per l'organizzazione dei file

3.3.4.1 Struttura sequenziale

Un file è costituito da blocchi "logicamente" consecutivi e le tuple vengono inserite nei blocchi rispettando una sequenza.

3.3.4.1.1 Struttura sequenziale disordinata

Le tuple sono disposte in base al loro ordine di inserimento, senza alcun criterio specifico. Questa è la struttura più semplice e più diffusa.

⇒ **inserimento:**

- ◇ operazione molto efficiente
- ◇ sufficiente mantenere un riferimento all'ultimo blocco
- ◇ richiede solo un accesso in memoria secondaria
- ◇ maggiore costo in caso di verifica dei vincoli di integrità

⇒ **ricerca**

- ◇ operazione non efficiente
- ◇ richiede ogni volta una scansione sequenziale che consideri tutti i record, con un costo lineare nel numero di blocchi
- ◇ ricerca può essere fatta usando strutture secondarie (indici)

⇒ **eliminazione**

- ◇ facile una volta individuate le tuple coinvolte
- ◇ si marcano le tuple come "cancellate" senza riorganizzazione locale

⇒ **modifica**

- ◇ facile una volta individuate le tuple coinvolte
- ◇ si procede in loco se possibile, cioè quando non c'è stata nessuna modifica alla dimensione della tupla
- ◇ si procede con una cancellazione e un inserimento in fondo al file

Eliminazioni e modifiche possono portare a un uso non ottimale dello spazio, siccome, si devono effettuare riorganizzazioni periodiche.

3.3.4.1.2 Struttura sequenziale ad array

Le tuple sono disposte come in un array e la loro posizione dipende dal valore assunto in ciascuna tupla da un campo detto **indice**.

Questa struttura è utilizzabile soltanto quando le tuple di una tabella hanno una dimensione fissa, infatti non è quasi mai usata nei DBMS reali perchè difficilmente sono soddisfatte le condizioni per la sua applicazione.

Ad ogni file viene assegnato un numero n di blocchi contigui e ciascun blocco è dotato di un numero m di posizioni disponibili per le tuple (array bidimensionale $n \times m$).

Ogni tupla è dotata di un valore numerico i che rappresenta l'indice della tupla in i -esima posizione nell'array.

⇒ **inserimento**

- iniziale: l'indice i viene ottenuto incrementando un contatore
- successivo: in una posizione libera oppure alla fine del file

⇒ **cancellazione**: creano delle posizioni libere

3.3.4.1.3 Struttura sequenziale ordinata

È un file sequenziale dove le tuple sono ordinate secondo una chiave di ordinamento (diversa dalla chiave primaria).

Questa struttura rende efficienti le operazioni che hanno bisogno dell'ordinamento utilizzato, ad esempio:

- elenco ordinato

- range query
- operazioni aggregate

In caso di aggiornamento è necessario mantenere l'ordinamento, quindi servono periodiche riorganizzazioni.

⇒ **inserimento**

1. individuare il blocco B che contiene la tupla che precede, nell'ordine della chiave, la tupla da inserire
2. se B contiene spazio sufficiente per la nuova tupla allora si inserisce la tupla in B
3. altrimenti si aggiunge un nuovo blocco (overflow page) alla struttura e si inserisce la tupla nel nuovo blocco e si aggiusta la catena di puntatori

⇒ **scansione sequenziale ordinata secondo la chiave**: è sufficiente seguire i puntatori

⇒ **cancellazione di una tupla**

1. individuare il blocco B che contiene la tupla da cancellare
2. cancellare la tupla da B
3. aggiustare la catena di puntatori

⇒ **riorganizzazione**: si assegnano le tuple ai blocchi in base a opportuni coefficienti di riempimento, riaggiustando i puntatori

Esempio: struttura sequenziale

Il seguente esempio mostra una struttura sequenziale ordinata secondo la chiave Filiale.

chiave ordinamento
↓

	Filiale	Conto	Cliente	Saldo	
	A	102	Rossi	1000	
Blocco 1	B	110	Rossi	3020	
	B	198	Bianchi	500	
	E	17	Neri	345	
	E	102	Verdi	1200	
Blocco 2	E	113	Bianchi	200	
	H	53	Neri	120	
	F	78	Verdi	3400	



3.3.4.1.4 Indice

Per aumentare le prestazioni degli accessi alle tuple memorizzate nelle strutture fisiche (file sequenziale), si introducono strutture ausiliarie dette strutture di accesso ai dati (indici). Queste strutture velocizzano l'accesso casuale via chiave di ricerca, cioè un insieme di attributi utilizzati nell'indice della ricerca.

Ci sono due tipi di indici su file sequenziali:

- ▷ **indice primario**: la chiave di ordinamento del file sequenziale coincide con la chiave di ricerca dell'indice.
Ogni record dell'indice primario contiene una coppia $\langle v_i, p_i \rangle$, dove:

- v_i : valore della chiave di ricerca
- p_i : puntatore al primo record nel file sequenziale con chiave v_i

Esistono due varianti dell'indice primario:

- **indice denso**: per ogni occorrenza della chiave presente nel file esiste un corrispondente record nell'indice
- **indice sparso**: solo per alcune occorrenze della chiave presenti nel file esiste un corrispondente record nell'indice, tipicamente una per blocco

Le operazioni che si possono eseguire su un indice primario sono:

- ◇ **ricerca di una tupla con chiave di ricerca K**
 - se l'indice è denso $K \rightarrow$ è presente nell'indice
 1. scansione sequenziale dell'indice per trovare il record (K, p_k)
 2. accesso al file attraverso il puntatore p_k
- ⇒ **costo: 1 accesso indice + 1 accesso blocco dati**

Esempio: ricerca con indice primario denso

Un esempio di ricerca con indice primario denso è il seguente:

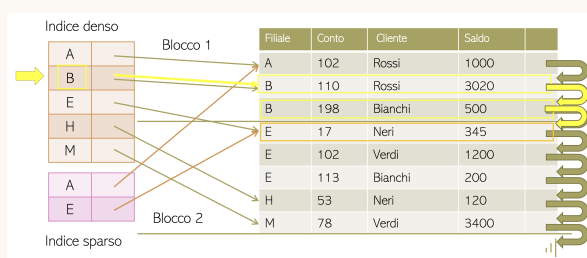


Figura 3.4: ricerca dei conti della filiale B

- se l'indice è sparso $K \rightarrow$ potrebbe non essere presente nell'indice
 1. scansione sequenziale dell'indice fino al record (K', p_k) , dove K' è il valore più grande che sia minore o uguale a K
 2. accesso al file attraverso il puntatore p_k e scansione del file (blocco corrente) per trovare le tuple con chiave K
- ⇒ **costo: 1 accesso indice + 1 accesso blocco dati**

Esempio: ricerca con indice primario sparso

Un esempio di ricerca con indice primario sparso è il seguente:

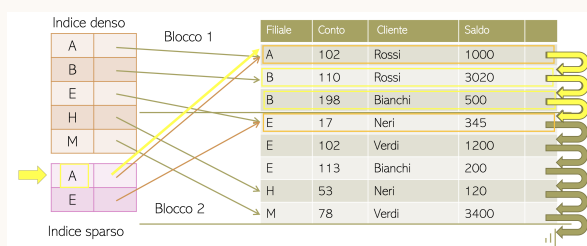


Figura 3.5: ricerca dei conti della filiale B

◇ **inserimento di un record nell'indice**

- **indice è denso**: inserimento nell'indice avviene solo se la tupla inserita nel file ha un valore di chiave K che non è già presente
- **indice è sparso**: inserimento avviene solo quando, per effetto dell'inserimento di una nuova tupla, si aggiunge un blocco dati alla struttura (negli altri casi rimane invariato)

Gli indici sono costosi, quindi, bisogna fare un compromesso sul numero di indici da mantenere e il numero di attributi ad accesso sequenziale.

◇ **cancellazione di un record nell'indice**

- **indice è denso**: cancellazione nell'indice avviene solo se la tupla cancellata nel file è l'ultima tupla con valore di chiave K
- **indice è sparso**: cancellazione nell'indice avviene solo quando K è presente nell'indice e il corrispondente blocco viene eliminato
Nel caso in cui il blocco sopravvive, va sostituito K nel record dell'indice con il primo valore K' presente nel blocco.

▷ **indice secondario**: la chiave di ordinamento e la chiave di ricerca sono diverse.

Ogni record dell'indice secondario contiene una coppia $\langle v_i, p_i \rangle$, dove:

- v_i : valore della chiave di ricerca
- p_i : puntatore al bucket di puntatori che individuano nel file sequenziale tutte le tuple con chiave v_i

Gli indici secondari sono sempre densi perchè non si può sfruttare l'ordinamento delle tuple.

Le operazioni che si possono eseguire su un indice secondario sono:

◇ **ricerca di una tupla con chiave di ricerca K**

1. scansione sequenziale dell'indice per trovare il record (K, p_K)
2. accesso al bucket B di puntatori attraverso il puntatore B
3. accesso al file attraverso i puntatori del bucket B

⇒ **costo: 1 accesso indice + 1 accesso al bucket + n accessi alle pagine dati** (n è il numero di puntatori nel bucket B)

Esempio: ricerca con indice secondario

Un esempio di ricerca con indice secondario è il seguente:

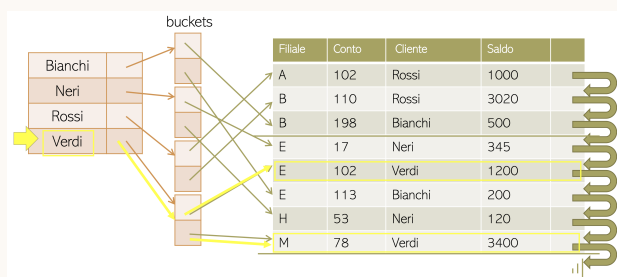


Figura 3.6: ricerca dei conti della filiale B

- ◇ **inserimento e cancellazione**: si effettuano gli stessi passi dell'inserimento e della cancellazione in un indice primario denso, con in più l'aggiornamento dei bucket

3.3.4.2 Struttura ad albero

Quando l'indice aumenta di dimensioni non si può più gestire in memoria centrale, di conseguenza deve essere gestito in memoria secondaria.

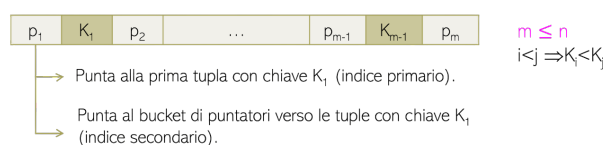
Per migliorare le prestazioni degli accessi a un indice di grandi dimensioni, si utilizza il B+ tree, in cui:

- ▷ ogni nodo viene memorizzato in una pagina della memoria secondaria
- ▷ legami tra nodi = puntatori a pagina
- ▷ sono presenti molti figli e di conseguenza ha pochi livelli
- ▷ albero bilanciato: la lunghezza dei percorsi che collegano la radice ai nodi foglia è costante
- ▷ inserimenti e cancellazioni non alterano le prestazioni dell'accesso ai dati: l'albero si mantiene bilanciato

3.3.4.2.1 Struttura

La struttura di un B+ tree con fan-out = n (n figli per nodo) è la seguente:

- ▷ **nodo foglia**



Può contenere fino a $n - 1$ valori ordinati di chiave di ricerca e fino a n puntatori ai dati.

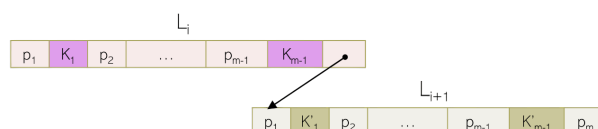
Ogni nodo foglia ha un vincolo di ordinamento, cioè:

- chiavi dentro a un nodo foglia sono ordinate
- nodi foglia sono ordinati tra di loro.

Dati due nodi foglia L_i e L_j con $i < j$, allora:

$$\forall k_i \in L_i; \quad \forall k_s \in L_j; \quad k_t < k_s$$

Il puntatore p_m del nodo L_i punta al nodo foglia L_{i+1} (se esiste):



Una variante del B+ tree è quella in cui i nodi foglia contengono direttamente le tuple. In questo caso si ha una struttura fisica integrata.

- ▷ **nodo intermedio**: è una sequenza di $m' \leq n$ valori ordinati di chiave (può contenere fino a n puntatori a nodo).

Ogni nodo chiave K_i è seguita da un puntatore p_i :



3.3.4.2.2 Vincoli di riempimento

Il fan-out è un parametro che fornisce il limite massimo, ma bisogna anche fornire un limite minimo per evitare che i nodi siano troppo vuoti e sono chiamati **vincoli di riempimento**:

- ◇ **vincoli per i nodi foglia**: ogni nodo foglia può contenere un numero di valori di chiavi (# chiavi) vincolato

$$\left\lceil \frac{n-1}{2} \right\rceil \leq \# \text{ chiavi} \leq n-1$$

- ◇ **vincoli per i nodi intermedi**: ogni nodo intermedio può contenere un numero di puntatori (# puntatori) vincolato

$$\left\lceil \frac{n}{2} \right\rceil \leq \# \text{ puntatori} \leq n$$

Esempio: utilizzo del B+-tree

Consideriamo ad esempio un B+-Tree con fan-out = 4 con contenuto:

A, B, D, E, F, G, L, M, N, P

Il fan-out ci dice che per:

- **nodi foglia**: vincoli di riempimento sono

$$2 \leq \# \text{chiavi} \leq 3$$

- **nodi intermedi**: vincoli di riempimento sono

$$2 \leq \# \text{puntatori} \leq 4$$

- **radice**: unico nodo per cui i vincoli di riempimento **minimi** non si applicano

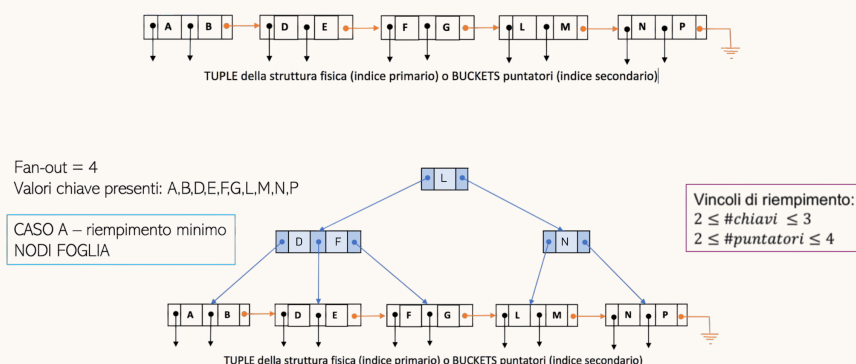


Figura 3.7: Esempio di B+-Tree con vincolo di riempimento minimo

La radice L punta:

- ◇ a sinistra: al sottoalbero con chiave $K < "L"$
- ◇ a destra: al sottoalbero con chiave $K \geq "L"$

I figli puntano alle tuple con chiave uguale a quella del nodo foglia.

In questo caso l'albero per chiavi desiderate rispetta il vincolo di riempimento minimo.

Per rispettare il vincolo di riempimento massimo bisogna fare un altro albero:

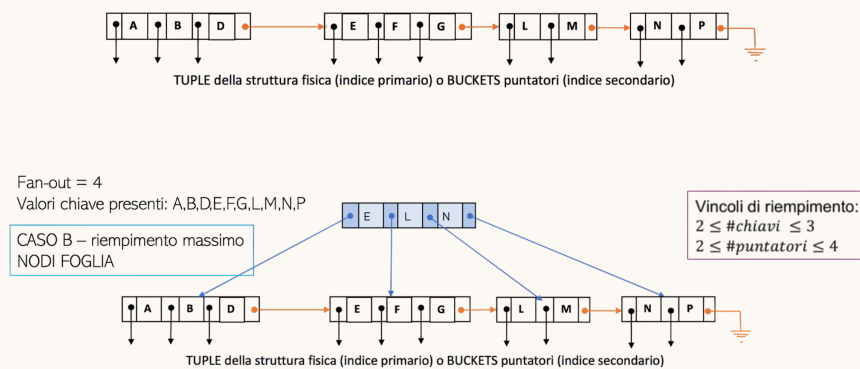


Figura 3.8: Esempio di B+-Tree con vincolo di riempimento massimo

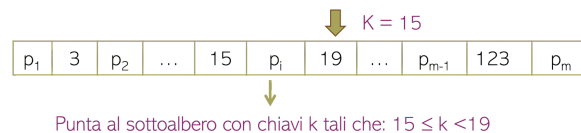
Le etichette per la radice sono assegnate secondo:

- sinistra: Chiavi comprese tra: " E " $\leq K < "L"$
- centro: Chiavi comprese tra " L " $\leq K < "N"$
- destra: Chiavi $K \geq "N"$

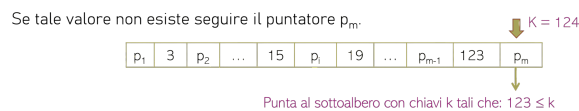
3.3.4.2.3 Operazione: ricerca con chiave K

Per effettuare la ricerca di una tupla con chiave di ricerca K si effettuano i seguenti passi:

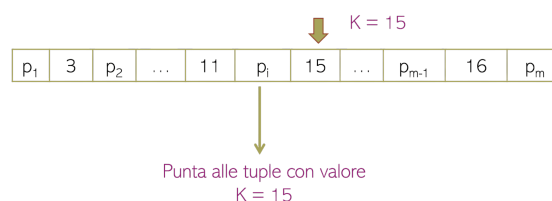
1. cercare nel nodo radice il più piccolo valore di chiave maggiore di K
 - ▷ se il valore esiste (value = k_i) → seguire il puntatore p_i



- ▷ se il valore non esiste → seguire puntatore p_m

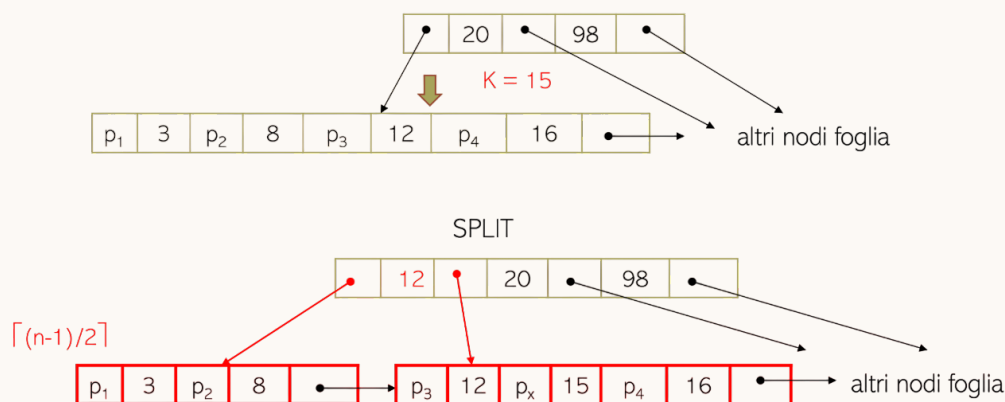


2. cercare il valore K (se nodo raggiunto = nodo foglia) e seguire il corrispondente puntatore verso le tuple o riprendere il passo 1 (se non esistono)



Esempio: split di un nodo foglia

Lo split di un nodo foglia con fan-out = 5 è il seguente:



Se si aggiungesse la chiave 15 tra il 12 e il 16 si violerebbe il vincolo di riempimento massimo: # chiavi = 5, quindi, si esegue lo split del nodo foglia.

3.3.4.2.5 Operazione: cancellazione

Per cancellare una tupla in un B+ tree con chiave K bisogna mantenere i vincoli di riempimento, quindi, vengono eseguiti i seguenti passi:

1. **ricercare il nodo foglio NF che contiene K**

2. **cancellare K e il suo puntatore**

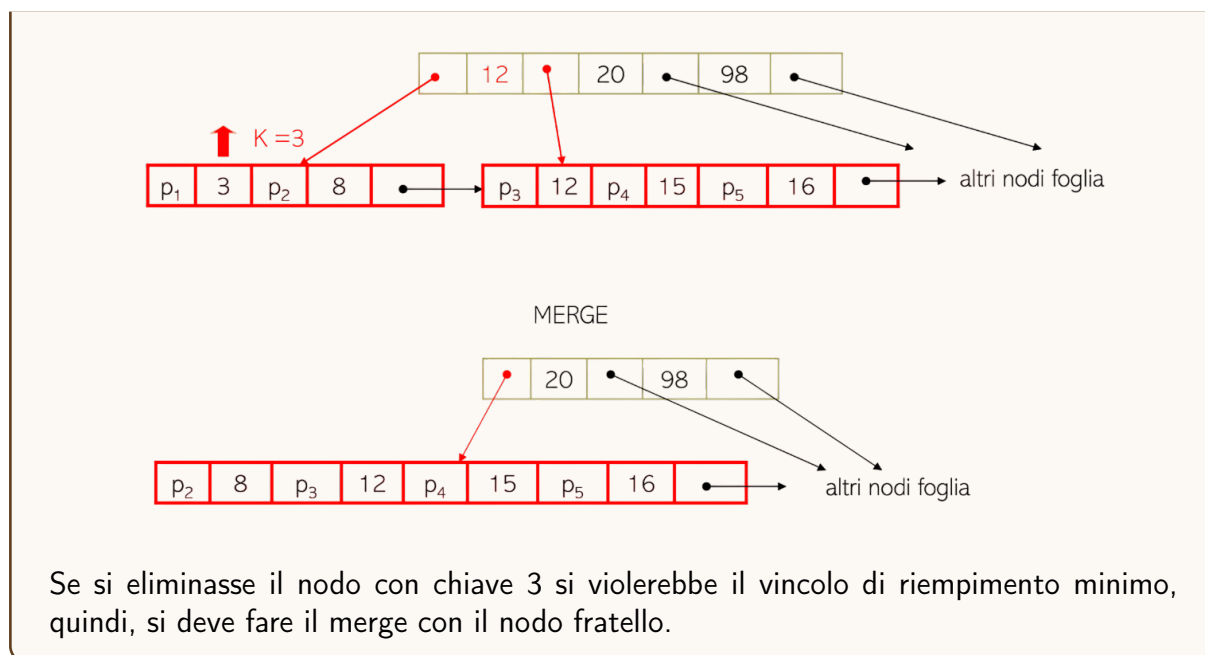
Potrebbe violare il vincolo di riempimento minimo, in quel caso bisogna fare il merge di NF con un fratello.

Per fare il merge, se nel nodo da unire esistono $\lceil \frac{n-1}{2} \rceil - 1$ valori chiavi, si procede:

- (a) individuare il nodo fratello
- (b) generare un unico nodo foglia
- (c) aggiornare il padre:
 - ▷ se non c'è un nodo foglia fratello che unito rispetta il vincolo di riempimento massimo, si fa prima il merge e poi lo split
 - ▷ se anche il padre viola il vincolo di riempimento massimo, si propaga il merge fino ad arrivare alla radice

Esempio: merge di un nodo foglia

Un esempio di merge di un nodo foglia con fan-out = 5, in cui si vuole eliminare la tupla con chiave K = "3" è il seguente:



3.3.4.3 Struttura ad accesso calcolato (hash)

Questa struttura garantisce un **accesso associativo** alle tuple, cioè la locazione fisica dei dati dipende dal valore assunto dalla chiave.

Si potrebbe utilizzare un array indicizzato dalla chiave, ma questo non è pratico perchè i valori della chiave sono tanti, quindi, si utilizza una **funzione di hash** che mappa i valori di chiave in indirizzi di memorizzazione delle tuple:

$$h : K \rightarrow B$$

dove:

- ▷ K : dominio delle chiavi
- ▷ B : dominio degli indirizzi di memorizzazione

Il vantaggio di questa struttura è che il costo della ricerca è costante e uguale a un accesso alla memoria secondaria.

Lo svantaggio è quello delle **collisioni**, cioè valori di chiavi diversi possono produrre lo stesso indirizzo.

Una funzione di hash **buona** minimizza il numero di collisioni.

Definizione: fattore riempimento

Il **fattore riempimento** definisce la frazione di spazio fisico utilizzata da un blocco.

Siano:

- ▷ T : numero di tuple
- ▷ F : fattore blocco (numero di tuple che ci stanno in un blocco)
- ▷ f : fattore di riempimento

si ha che il numero di blocchi è definito come:

$$\#B = \left\lceil \frac{T}{F \cdot f} \right\rceil$$

Il DBMS alloca un numero di bucket di puntatori uguale al numero di blocchi $\#B$.

Si definiscono due funzioni:

◇ **funzione di folding**: trasforma i valori di chiave in numeri interi positivi

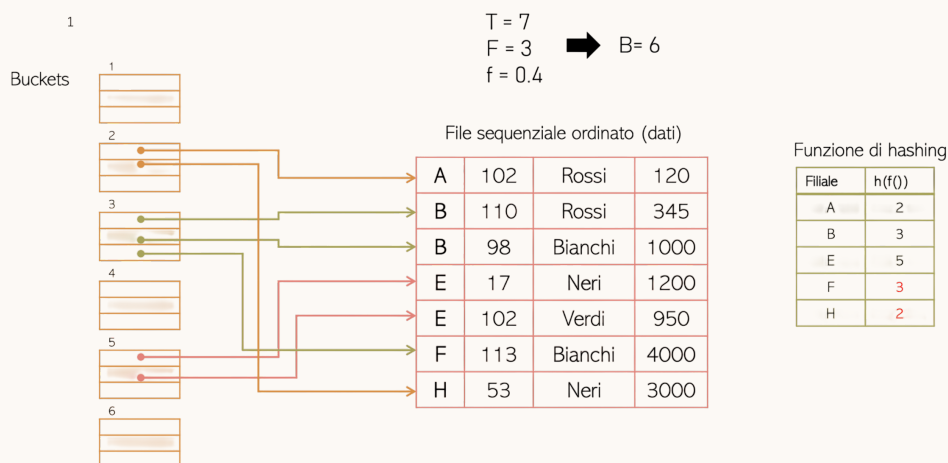
$$f : K \rightarrow \mathbb{Z}^+$$

◇ **funzione di hashing**:

$$h : \mathbb{Z}^+ \rightarrow B$$

Esempio: tabella di hash

Un esempio di tabella di hash è il seguente:



3.3.4.3.1 Operazione: ricerca con chiave K

Per effettuare la ricerca di una tupla con chiave di ricerca K si effettuano i seguenti passi:

1. calcolare $b = h(f(K))$ (costo zero)
2. accedere al bucket b (costo: 1 accesso alla pagina di memoria secondaria)
3. accedere alle n tuple attraverso i puntatori del bucket (costo: m accessi a pagine di memoria secondaria, con $m \leq n$)

L'inserimento e la cancellazione hanno costo simile alla ricerca.

3.3.4.3.2 Collisioni

La struttura ad accesso calcolato funziona se i buckets conservano un basso coefficiente di riempimento.

Infatti, il problema delle strutture ad accesso calcolato è la gestione delle collisioni.

Definizione: collisione

Una **collisione** si verifica, quando, dati due valori di chiave K_1 e K_2 con $K_1 \neq K_2$, risulta:

$$h(f(K_1)) = h(f(K_2))$$

La probabilità che uno stesso bucket riceva t chiavi su n inserimenti è:

$$p(t) = \binom{n}{t} \cdot \left(\frac{1}{B}\right)^t \cdot \left(1 - \frac{1}{B}\right)^{n-t}$$

dove:

▷ B : numero totale di bucket

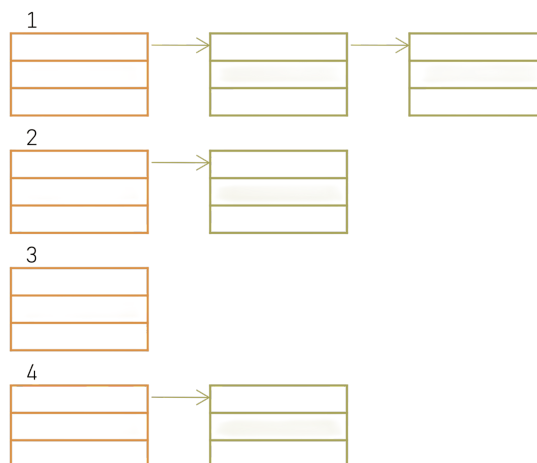
La probabilità di avere più di F collisioni, con F il fattore blocco, cioè il numero di puntatori nel bucket, è:

$$p_K = 1 - \sum_{i=0}^F p(i)$$

Un numero eccessivo di collisioni porta alla saturazione del bucket corrispondente.

Per gestirlo si introducono dei **bucket di overflow** collegati al bucket originale, che contengono i puntatori alle tuple che hanno causato la saturazione del bucket originale.

Le prestazioni della ricerca peggiorano perchè bisogna accedere ai bucket di overflow.



Esecuzione concorrente di transazioni

L'unità di misura usata per valutare il carico di lavoro di un DBMS è il **transazioni al secondo** (tps).

Per gestire con prestazioni accettabili il carico di lavoro tipico delle applicazioni gestionali è di 100 o 1000 tps.

L'esecuzione concorrente di transazioni senza controllo può generare **anomalie** o **problemi di correttezza**.

Bisogna, quindi, introdurre dei meccanismi di controllo nell'esecuzione delle transazioni per evitare tali anomalie.

4.1 Tipologie di anomalie

Le principali tipologie di anomalie sono:

- ▷ **perdita di aggiornamento**: effetti di una transazione concorrente vengono persi (riscritti)
- ▷ **lettura inconsistente**: accessi successivi alla stessa risorsa ritornano risultati diversi
- ▷ **lettura sporca**: viene letto un dato che rappresenta uno stato intermedio di un'altra transazione
- ▷ **aggiornamento fantasma**: osservazione di uno stato intermedio che viola i vincoli d'integrità
- ▷ **inserimento fantasma**: valutazioni diverse di operazioni di aggregazione producono risultati diversi

Notazione

- ◇ t_i : transazione t_i
- ◇ $r_i(x)$: operazione di lettura eseguita dalla transazione t_i sulla risorsa x
- ◇ $w_i(x)$: operazione di scrittura eseguita dalla transazione t_i sulla risorsa x

4.1.1 Perdita di aggiornamento

Nella **perdita di aggiornamento** gli effetti di una transazione concorrente vengono persi.

Esempio: perdita di aggiornamento

Consieriamo le seguenti transazioni:

$$t_1 : bot; r_1(x); x = x + 1; w_1(x); commit; eot$$

$$t_2 : bot; r_2(x); x = x + 1; w_2(x); commit; eot$$

Simuliamo l'esecuzione concorrente di t_1 e t_2 :

Operazioni eseguite da t_1	Segmento di memoria centrale di t_1	Memoria secondaria [buffer]	Segmento di memoria centrale di t_2	Operazioni eseguite da t_2
Stato iniziale		$x = 2$	Stato iniziale	
bot		2		
$r_1(x)$	2	[2]		
$x=x+1$	3	[2]		bot
	3	[2]	2	$r_2(x)$
	3	[2]	3	$x=x+1$
	3	[3]	3	$w_2(x)$
	3		3	commit
	3			eot
$w_1(x)$	3	[3]		
commit	3	3		
eot		3		

Il valore finale di x è 3, mentre una qualsiasi esecuzione seriale avrebbe prodotto per x il valore 4!

4.1.2 Lettura inconsistente

Nella **lettura inconsistente** gli accessi successivi ad uno stesso dato all'interno di una transazione ritornano valori diversi.

Esempio: lettura inconsistente

Consieriamo le seguenti transazioni:

$$t_1 : bot; r_1(x); r'_1(x); commit; eot$$

$$t_2 : bot; r_2(x); x = x + 1; w_2(x); commit; eot$$

Simuliamo l'esecuzione concorrente di t_1 e t_2 :

Operazioni eseguite da t1	Segmento di memoria centrale di t1	Memoria secondaria [buffer]	Segmento di memoria centrale di t2	Operazioni eseguite da t2
Stato iniziale		x = 2	Stato iniziale	
bot		2		
r ₁ (x)	2	[2]		
	2	[2]		bot
	2	[2]	2	r ₂ (x)
	2	[2]	3	x=x+1
	2	[3]	3	w ₂ (x)
	2	3	3	commit
	2	3		eot
r ₁ '(x)	3	[3]		
commit	3	3		
eot		3		

Il valore di x è diventato 3, ma la transazione t₁ non ha modificato x!

4.1.3 Lettura sporca

Nella **lettura sporca** viene letto un dato che rappresenta uno stato intermedio nell'evoluzione di una transazione.

Esempio: lettura inconsistente

Consieriamo le seguenti transazioni:

$t_1 : bot; r_1(x); commit; eot$

$t_2 : bot; r_2(x); x = x + 1; w_2(x); \dots abort;$

Simuliamo l'esecuzione concorrente di t_1 e t_2 :

Operazioni eseguite da T1	Segmento di memoria centrale di T1	Memoria secondaria [buffer]	Segmento di memoria centrale di T2	Operazioni eseguite da T2
Stato iniziale		x = 2	Stato iniziale	
bot		2		
		2		bot
		[2]	2	r ₂ (x)
		[2]	3	x=x+1
		[3]	3	w ₂ (x)
r ₁ (x)	3	[3]	3	abort
commit	3	2		
eot		2		

Il valore di x letto da t₁ è 3, ma la transazione t₂ non ha ancora eseguito il commit!

4.1.4 Aggiornamento fantasma

L'**aggiornamento fantasma** consiste nell'osservazione di uno stato intermedio che non soddisfa i vincoli d'integrità.

Esempio: lettura inconsistente

Consieriamo le seguenti transazioni:

Risorse : x, y, z

Vincoli : $x + y + z = 100$

t_1 : *bot*; $r_1(y)$; $r_1(x)$; $r_1(z)$; $s = x + y + z$; *eot*

t_2 : *bot*; $r_2(y)$; $y = y + 10$; $r_2(z)$; $z = z - 10$; $w_2(y)$; $w_2(z)$; *commit*; *eot*

Simuliamo l'esecuzione concorrente di t_1 e t_2 :

Operazioni eseguite da T1	Segmento di memoria centrale di T1	Memoria secondaria [buffer]	Segmento di memoria centrale di T2	Operazioni eseguite da T2
Stato iniziale		(x,y,z)=20,20,60	Stato iniziale	
bot		20,20,60		
$r_1(y)$	-20,-	20,[20],60		
	-20,-	20,[20],60		bot
	-20,-	20,[20],60		$r_2(y)$
	-20,-	20,[20],60		$y = y + 10$
	-20,-	20,[20],60		$r_2(z)$
	-20,-	20,[20],60		$z = z - 10$
	-20,-	20,[30],60		$w_2(y)$
	-20,-	20,[30],[50]		$w_2(z)$
	-20,-	20,30,50		<i>commit</i>
	-20,-	20,30,50		<i>eot</i>
$r_1(x)$	20,20,-	[20],30,50		
$r_1(z)$	20,20,50	[20],30,[50]		
$s=x-y-z$	20,20,50	[20],30,[50]		
<i>eot</i>		20,30,50		

S = 90 → VIOLAZIONE VINCOLO

4.1.5 Inserimento fantasma

L'**inserimento fantasma** si verifica quando avvengono valutazioni diverse di valori aggregati a causa di inserimenti intermedi.

La transazione che valuta un valore aggregato relativo a tutti gli elementi che soddisfano un predicato di selezione (es. voto medio degli studenti) può subire l'anomalia di inserimento se il valore aggregato viene valutato due volte e tra la prima e la seconda valutazione viene inserito un nuovo studente.

4.2 Schedule

Definizione: schedule

Lo **schedule** è una sequenza di operazioni di lettura e scrittura eseguite su risorse della base di dati da diverse transazioni concorrenti.

Lo schedule, quindi, rappresenta una possibile esecuzione concorrente di diverse transazioni:

$S : r_1(x) \quad r_2(x) \quad w_1(z) \quad w_2(x)$

Per evitare anomalie, si stabilisce di accettare solo gli schedule equivalenti a uno schedule seriale.

4.2.1 Schedule seriale

Definizione: schedule seriale

Uno **schedule seriale** è uno Schedule dove le operazioni di ogni transazione compaiono in una sequenza senza essere inframmezzate da operazioni di altre transazioni.

$$\begin{aligned} S_1 : r_1(x) \quad r_2(x) \quad w_2(x) \quad w_1(x) &= \text{NON è SERIALE} \\ S_2 : r_1(x) \quad w_1(x) \quad r_2(x) \quad w_2(x) &= \text{è SERIALE} \end{aligned}$$

Definizione: schedule serializzabile

Lo **schedule serializzabile** è uno schedule equivalente a uno schedule seriale.

Due schedule si definiscono equivalenti se **producono gli stessi effetti** sulla base di dati.

4.3 Tecniche di gestione della concorrenza

Per le seguenti tecniche di gestione della concorrenza, si assume l'ipotesi di **commit-proiezione** (transazioni hanno esito noto).

Di conseguenza, vengono tolti dagli schedule tutte le operazioni delle transazioni che non vanno a buon fine.

4.3.1 View-equivalenza

Definizione: relazione "LEGGE_DA"

Dato uno schedule S , si dice che un'operazione di lettura $r_i(x)$ (che compare in S), **LEGGE_DA** un'operazione di scrittura $w_i(x)$ (che compare in S), se $w_j(x)$ **precede** $r_i(x)$ in S e non è alcuna operazione $w_k(x)$ tra le due.

$$\begin{aligned} S_1 : r_1(x) \quad r_2(x) \quad w_2(x) \quad w_1(x) \quad \text{LEGGE_DA}(S_1) &= \emptyset \\ S_2 : r_1(x) \quad w_1(x) \quad r_2(x) \quad w_2(x) \quad \text{LEGGE_DA}(S_2) &= \{((r_2(x), w_1(x)))\} \end{aligned}$$

Definizione: scritture finali

Dato uno schedule S si dice che un'operazione di scrittura $w_i(x)$ (che compare in S) è una **scrittura finale** se è l'ultima operazione di scrittura della risorsa x in S .

$$\begin{aligned} S_1 : r_1(x) \quad r_2(x) \quad w_2(x) \quad w_1(x) \quad \text{LEGGE_DA}(S_1) &= \emptyset \\ S_2 : r_1(x) \quad w_1(x) \quad r_2(x) \quad w_2(x) \quad \text{LEGGE_DA}(S_2) &= \{((r_2(x), w_1(x)))\} \end{aligned}$$

Definizione: view-equivalenza

Due schedule S_1 e S_2 sono view-equivalenti ($s_1 \sim s_2$) se possiedono le stesse relazioni LEGGE_DA e SCRITTURE_FINALI.

Definizione: viwe-serializzabilità

Uno schedule S è viwe-serializzabile (VSR) se esiste uno schedule seriale s' tale che $S \sim s'$.

Attenzione

Gli schedule seriale S' si generano considerando tutte le possibili permutazioni delle transazioni (non delle operazioni delle transazioni) che compaiono in S . inoltre, non si deve cambiare l'ordine delle operazioni di ciascuna transazione perchè ciò significherebbe modificare la transazione.

Gli schedule corrispondenti alle anomalie di:

- ◇ perdita di aggiornamento
- ◇ letture inconsistenti
- ◇ aggiornamenti fantasmi

non sono view-serializzabili.

Inoltre, l'algoritmo per il test di view-equivalenza tra due schedule è di complessità lineare. mentre per il test di view-serializzabilità di uno schedule è di complessità esponenziale.

In conclusione si è osservato che la view-serializzabilità richiede l'ipotesi di commit-proiezione, e algoritmi di complessità troppo elevata, quindi, non è applicabile nei sistemi reali.

Esercizio: perdita di aggiornamento

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x); w_1(x)$$

$$t_2 : r_2(x); w_2(x)$$

Eseguite con lo schedule:

$$S_{pa} = r_1(x); r_2(x); w_2(x); w_1(x);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $LEGGE_DA(S_{pa}) = \emptyset$
- $SCRITTURE_FINALI(S_{pa}) = \{w_1(x)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_{pa} \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = r_1(x); w_1(x); r_2(x); w_2(x);$
 - $\text{LEGGE_DA}(S_{12}) = \{(r_2(x), w_1(x))\}$
 - $\text{SCRITTURE_FINALI}(S_{12}) = \{w_1(x)\}$

Osserviamo che $S_{pa} \not\approx_V S_{12}$

- $S_{21} = r_2(x); w_2(x); r_1(x); w_1(x);$
 - $\text{LEGGE_DA}(S_{21}) = \{(r_1(x), w_2(x))\}$
 - $\text{SCRITTURE_FINALI}(S_{21}) = \{w_1(x)\}$

Osserviamo che $S_{pa} \not\approx_V S_{21}$

Avendo provato per tutte le possibili permutazioni si può dire che lo schedule S_{pa} **non** è view-serializzabile.

Esercizio: lettura inconsistente

Consideriamo le seguenti transazioni:

$$\begin{aligned} t_1 &: r_1(x); w_1(x); r_1(y); w_1(y) \\ t_2 &: r_2(z); r_2(x); w_2(x); w_2(z) \end{aligned}$$

Eseguite con lo schedule:

$$S_1 = r_1(x); w_1(x); r_2(z); r_1(y); w_1(y); r_2(x); w_2(x); w_2(z);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\text{LEGGE_DA}(S_1) = \{(r_2(x), w_1(x))\}$
- $\text{SCRITTURE_FINALI}(S_1) = \{w_2(x), w_1(y), w_2(z)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_1 \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = t_1 t_2$
- $S_{21} = t_2 t_1$

Al posto di creare tutte le permutazioni degli schedule seriali, cerchiamo di osservare le relazioni LEGGE_DA e SCRITTURE_FINALI per capire se S_1 è view-equivalente ad S_{12} o S_{21} .

Dalla presenza di $w_2(x)$ nelle scritture finali di S_1 si nota che $t_1 < t_2$ perchè $w_2(x)$ è l'ultima scrittura di x in S_1 , e questa operazione viene eseguita solo da t_2 e di conseguenza va eseguita dopo t_1 . Da ciò sappiamo che sicuramente $S_1 \not\approx_V S_{21}$ e ci rimane da controllare soltanto se $S_1 \approx_V S_{12}$:

$$S_{12} = r_1(x); w_1(x); r_1(y); w_1(y); r_2(z); r_2(x); w_2(x); w_2(z);$$

- $\text{LEGGE_DA}(S_{12}) = \{(r_2(x), w_1(x))\}$
- $\text{SCRITTURE_FINALI}(S_{12}) = \{w_2(x), w_1(y), w_2(z)\}$

Osserviamo che $S_1 \approx_V S_{12}$ e quindi S_1 è view-serializzabile.

Esercizio:

Consideriamo le seguenti transazioni:

$$\begin{aligned} t_1 &: r_1(x); w_1(x); r_1(y); w_1(y) \\ t_2 &: r_2(z); r_2(x); w_2(x); w_2(z) \end{aligned}$$

Eseguite con lo schedule:

$$S_1 = r_1(x); w_1(x); r_2(z); r_1(y); w_1(y); r_2(x); w_2(x); w_2(z);$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\text{LEGGE_DA}(S_1) = \{(r_2(x), w_1(x))\}$
- $\text{SCRITTURE_FINALI}(S_1) = \{w_2(x), w_1(y), w_2(z)\}$

Bisogna generare tutti i possibili schedule seriali per determinare se esiste uno schedule seriale S' tale che $S_1 \approx_V S'$. Gli schedule seriali sono:

- $S_{12} = t_1 t_2$
- $S_{21} = t_2 t_1$

Al posto di creare tutte le permutazioni degli schedule seriali, cerchiamo di osservare le relazioni LEGGE_DA e SCRITTURE_FINALI per capire se S_1 è view-equivalente ad S_{12} o S_{21} .

Dalla presenza di $w_2(x)$ nelle scritture finali di S_1 si nota che $t_1 < t_2$ perchè $w_2(x)$ è l'ultima scrittura di x in S_1 , e questa operazione viene eseguita solo da t_2 e di conseguenza va eseguita dopo t_1 . Da ciò sappiamo che sicuramente $S_1 \not\approx_V S_{21}$ e ci rimane da controllare soltanto se $S_1 \approx_V S_{12}$:

$$S_{12} = r_1(x); w_1(x); r_1(y); w_1(y); r_2(z); r_2(x); w_2(x); w_2(z);$$

- $\text{LEGGE_DA}(S_{12}) = \{(r_2(x), w_1(x))\}$
- $\text{SCRITTURE_FINALI}(S_{12}) = \{w_2(x), w_1(y), w_2(z)\}$

Osserviamo che $S_1 \approx_V S_{12}$ e quindi S_1 è view-serializzabile.

Esercizio: view_serializzabilita

Consideriamo le seguenti transazioni:

$$\begin{aligned} t_1 &: r_1(x); w_1(x); w_1(y) \\ t_2 &: r_2(y); r_2(x) \\ t_3 &: w_3(x); r_3(y); w_3(y) \end{aligned}$$

$$S_2 = r_1(x); w_1(x); w_3(x); r_2(y); r_3(y); w_3(y); w_1(y); r_2(x)$$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $\text{LEGGE_DA}(S_2) = \{(r_2(x), w_3(x))\}$

- $SCRITTURE_FINALI(S_2) = \{w_3(x), w_1(y)\}$

Siccome ci sono 6 possibili permutazioni vediamo se è possibile scartarne alcune a priori.

- L'insieme $SCRITTURE_FINALI(S_2)$ implica:
 - $t_1 < t_3$ perchè $w_3(x)$ è l'ultima scrittura di x in S_2 e questa operazione viene eseguita solo da t_3 e di conseguenza va eseguita dopo t_1
 - $t_3 < t_1$ perchè $w_1(y)$ è l'ultima scrittura di y in S_2 e questa operazione viene eseguita solo da t_1 e di conseguenza va eseguita dopo t_3

Si osserva che $t_1 < t_3$ e $t_3 < t_1$ è una contraddizione, cioè non esiste uno schedule seriale che le rispetti entrambe e quindi si può concludere che S_2 **non è view-serializzabile**.

- L'insieme $LEGGE_DA(S_2)$ implica:
 - $t_3 < t_2$ perchè $r_2(x)$ (in t_2) legge da $w_3(x)$ (in t_3) e quindi $(r_2(x), w_3(x)) \in LEGGE_DA$
 - $t_1 < t_3$ per garantire $(r_1(x), w_3(x)) \notin LEGGE_DA$
 - $t_2 < t_3$ per garantire $(r_2(x), w_3(x)) \in LEGGE_DA$
 - $t_2 < t_1$ per garantire $(r_2(x), w_1(x)) \notin LEGGE_DA$
 - $t_3 < t_1$ per garantire $(r_3(x), w_1(x)) \notin LEGGE_DA$

Si osserva che $t_1 < t_3$ e $t_3 < t_1$ è una contraddizione, quindi si può concludere che S_2 **non è view-serializzabile**.

Esercizio: view_serializzabilita_2

Consideriamo le seguenti transazioni:

$t_1 : r_1(x); w_1(x); w_1(y); w_1(z)$

$t_2 : r_2(x); w_2(x)$

$t_3 : r_3(x); w_3(y); w_3(x)$

$t_4 : r_4(z)$

$t_5 : w_5(x); w_5(y); r_5(z)$

eseguite con lo schedule:

$S_3 = r_1(x); r_2(x); w_2(x); r_3(x); r_4(z); w_1(x); w_3(y); w_3(x); w_1(y);$
 $w_5(x); w_1(z); w_5(y); r_5(z)$

Questo schedule è view serializzabile?

Calcoliamo le relazioni:

- $LEGGE_DA(S_3) = \{(r_3(x), w_2(x)), (r_5(z), w_1(z))\}$
- $SCRITTURE_FINALI(S_3) = \{w_5(x), w_5(y), w_1(z)\}$

Le permutazioni di tutte le transazioni sono 5! e quindi è necessario cercare di scartarne alcune:

- Considerando $SCRITTURE_FINALI(S_3)$ si ha che:
 - $w_5(x) \Rightarrow t_2 < t_5, t_1 < t_5, t_3 < t_5$
 - $w_5(y) \Rightarrow t_3 < t_5, t_1 < t_5$

Da queste informazioni non si rilevano contraddizioni, quindi si procede con l'analisi di $LEGGE_DA(S_3)$.

- Considerando $LEGGE_DA(S_3)$ si ha che:

- $(r_3(x), w_2(x)) \Rightarrow t_2 < t_3$
- $(r_5(z), w_1(z)) \Rightarrow t_1 < t_5$

Da queste informazioni non si rilevano contraddizioni, quindi si procede osservando cosa **non compare** nell'insieme $LEGGE_DA(S_3)$:

- $(r_1(x), w_2(x)) \notin LEGGE_DA(S_3) \Rightarrow t_1 < t_2$
- $(r_1(x), w_3(x)) \notin LEGGE_DA(S_3) \Rightarrow t_1 < t_3$
- $(r_1(x), w_5(x)) \notin LEGGE_DA(S_3) \Rightarrow t_1 < t_5$
- $(r_2(x), w_1(x)) \notin LEGGE_DA(S_3) \Rightarrow t_2 < t_1$ Questo è in contraddizione con $t_1 < t_2$ e si può concludere che S_3 **non è view-serializzabile**.

4.3.2 Conflict-equivalenza

Definizione: conflitto

Dato uno schedule S , si dice che una coppia di operazioni (a_i, a_j) , dove a_i e a_j compaiono nello schedule S , rappresentano un conflitto se:

- appartengono a transazioni diverse ($i \neq j$)
- entrambe operano sulla stessa risorsa
- almeno una di esse è un'operazione di scrittura
- a_i compare in S prima di a_j

Definizione: conflict-equivalenza

Due schedule S_1 e S_2 sono conflict-equivalenti ($S_1 \equiv S_2$) se possiedono le stesse operazioni e gli stessi conflitti, cioè se le operazioni in conflitto sono nello stesso ordine nei due schedule.

Definizione: conflict-serializzabile

Uno schedule S è conflict-serializzabile (CSR) se esiste uno schedule seriale S' tale che $S \equiv S'$.

L'algoritmo per il test di conflict-serializzabilità di uno schedule è di complessità lineare. Dato il seguente schedule S :

- ◊ si costruisce il grafo dei conflitti GN, A dove:
 - ▷ $N = t_1 \dots t_n$ è l'insieme delle transazioni di S
 - ▷ $(t_i, t_j) \in A$ se esiste almeno un conflitto (a_i, a_j) in S
- ◊ se il grafo è aciclico, allora S è conflict-serializzabile

Dimostrazione

1. se S è CSR, cioè esiste uno schedule S seriale tale che $S \prec S_c$ allora il suo grafo è aciclico.

Ipotesi: le transazioni di S_r sono ordinate:

$$t_1, t_2, \dots, t_n$$

Poichè $S \prec S_r$, allora S_r ha tutti i conflitti di S con operazioni esattamente nello stesso ordine.

Quindi nel grafo dei conflitti di S_r , ci possono essere solo archi (i, j) con $i < j$, e questo vuol dire che non possono esserci cicli poichè un ciclo richiederebbe un arco (i, j) con $j > i$ che è impossibile.

Siccome $S \prec S_r$, allora hanno lo stesso grafo dei conflitti e, quindi, anche il grafo dei conflitti di S è aciclico.

2. se il grafo S è aciclico, allora S è CSR cioè esiste uno schedule S_r seriale tale che $S \prec S_r$ e, quindi, S e S_r hanno gli stessi conflitti e, quindi, lo stesso grafo.

Ipotesi: il grafo dei conflitti di S è aciclico e, quindi, esiste un ordinamento topologico, cioè un'enumerazione dei nodi tale che il grafo contiene solo archi (i, j) con $i < j$.

Utilizzando questo ordinamento topologico possiamo definire un ordine delle transazioni che produce lo stesso grafo e, quindi, siccome le transazioni sono ordinate S ha uno schedule seriale.

Richiede comunque l'ipotesi di commit-proiezione.

Esempio: perdita di aggiornamento

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x); w_1(x)$$

$$t_2 : r_2(x); w_2(x)$$

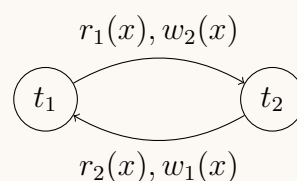
Eseguite con lo schedule:

$$S_{pa} : r_1(x); r_2(x); w_2(x); w_1(x);$$

L'insieme dei conflitti è:

$$CONFLITTI(S_{pa}) = \{(r_1(x), w_2(x)), (r_2(x), w_1(x)), (w_2(x), w_1(x))\}$$

Generiamo il grafo dei conflitti:



Siccome il grafo ha un ciclo S_{pa} **non è conflict-serializzabile**.

Esempio: lettura inconsistente

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x); r'_1(x)$$

$$t_2 : r_2(x); w_2(x)$$

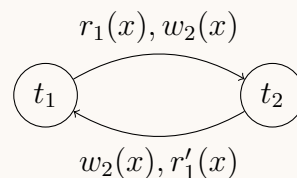
Eseguite con lo schedule:

$$S_{pa} : r_1(x); r_2(x); w_2(x); r'_1(x);$$

L'insieme dei conflitti è:

$$CONFLITTI(S_{pa}) = \{(r_1(x), w_2(x)), (w_2(x), r'_1(x))\}$$

Generiamo il grafo dei conflitti:

**Teorema: conflict-serializzabile**

La conflict-serializzabilità è una condizione sufficiente, ma non necessaria per la view-serializzabilità, cioè:

$$CSR \subset VSR$$

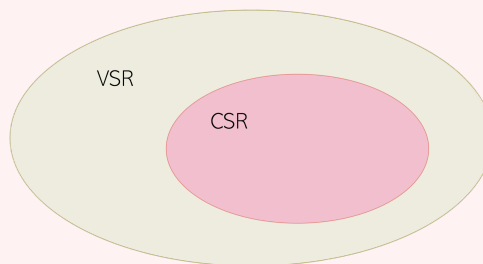


Figura 4.1: relazione tra conflict-serializzabilità e view-serializzabilità

Dimostrazione $S \text{ è } CSR \Rightarrow S \text{ è } VSR$ (se S non è CSR , allora non si può dire nulla sulla VSR).

▷ **dimostrazione che $CSR \subset VSR$:** basta trovare un controesempio

$$S \in VSR \text{ ma } S \notin CSR$$

Consideriamo:

$$S = r_1(x); w_2(x); w_1(x); w_3(x)$$

Gli insiemi sono:

$$\diamond \text{ LEGGE_DA}(S) = \emptyset$$

$$\diamond \text{SCRITTURE_FINALI}(S) = \{w_3(x)\}$$

Quindi questo schedule è VSR perché esiste uno schedule S_r tale che $S_r \equiv S$:

$$S_r = r_1(x); w_2(x); w_1(x); w_3(x)$$

che ha gli stessi insiemi:

$$\diamond \text{LEGGE_DA}(S_r) = \emptyset$$

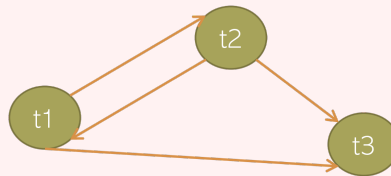
$$\diamond \text{SCRITTURE_FINALI}(S_r) = \{w_3(x)\}$$

Ora dimostriamo che questo schedule non è CSR.

Il suo insieme dei conflitti è:

$$\begin{aligned} \text{CONFLITTI}(S) &= \{(r_1(x), w_2(x)), (r_1(x), w_3(x)), (w_2(x), w_1(x)), (w_2(x), w_3(x))\} \\ &= \{(w_1(x), w_3(x))\} \end{aligned}$$

Costruiamo il grafico dei conflitti:



- ▷ **dimostrazione che CSR $\Rightarrow$ VSR:** se $S_1 \equiv_C S_2$ e dimostriamo $s_1 \equiv_V S_2$
- S_1 e S_2 devono avere lo stesso insieme di SCRITTURE_FINALI, altrimenti ci sarebbero due scritture sulla stessa risorsa da parte di transazioni in un ordine diverso e questo produrrebbe un conflitto diverso
 - devono avere lo stesso insieme di LEGGE_DA, altrimenti ci sarebbe un conflitto, composto da una lettura e una scrittura, diverso e, quindi, siccome il conflitto è diverso, allora S_1 e S_2 non sono conflit-equivalenti

Quindi se S :

- ▷ è CSR $\Rightarrow S$ è VSR
- ▷ non è CSR $\Rightarrow S$ non è VSR

Esempio:

Consideriamo il seguente schedule:

$$S_1 = r_1(x); w_1(x); r_2(z); r_1(y); w_1(y); r_2(x); w_2(x); w_2(z);$$

Verifichiamo se S_1 è VSR, per farlo vediamo se prima è CSR controllando l'insieme dei conflitti:

$$\text{CONFLITTI}(S_1) = \{(r_1(x), w_2(x)), (w_1(x), r_2(x)), (w_1(x), w_2(x))\}$$

Costruiamo il grafo dei conflitti:



Siccome il grafo è aciclico, allora S_1 è CSR e quindi S_1 è VSR.

Esempio:

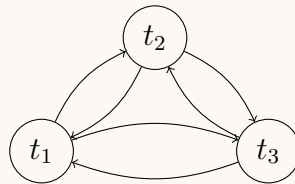
Consideriamo lo schedule:

$$S_2 = r_1(x); w_1(x); w_3(x); r_2(y); r_3(y); w_3(y); w_1(y); r_2(x);$$

Verifichiamo se S_2 è VSR, per farlo vediamo se prima è CSR controllando l'insieme dei conflitti:

$$\begin{aligned} alignedCONFLITTI(S_2) = \{ & (r_1(x), w_3(x)), (w_1(x), w_3(x)), (w_1(x), r_2(x)), \\ & (w_3(x), r_2(x)), (r_2(y), w_3(y)), (r_2(y), w_1(y)), \\ & (r_3(y), w_1(y)), (w_3(y), w_1(y)) \} \end{aligned}$$

Costruiamo il grafo dei conflitti:



Il grafo è ciclico, quindi S_2 **non è conflict-serializzabile** e di conseguenza S_2 **non è view-serializzabile**.

Questo schedule lo avevamo verificato anche nell'esercizio ?? e avevamo concluso che S_2 non è view-serializzabile, e questo conferma che S_2 non è conflict-serializzabile perchè $CSR \subset VSR$.

Esempio:

Consideriamo lo schedule:

$$\begin{aligned} S_3 = & r_1(x); r_2(x); w_2(x); r_3(x); r_4(z); w_1(x); w_3(y); w_3(x); w_1(y); \\ & w_5(x); w_1(z); w_5(y); r_5(z); \end{aligned}$$

Verifichiamo se S_3 è VSR, per farlo vediamo se prima è CSR controllando l'insieme dei conflitti:

$$\begin{aligned} CONFLITTI(S_3) = \{ & (r_1(x), w_2(x)), (r_1(x), w_3(x)), (r_1(x), w_5(x)), \\ & (r_2(x), w_1(x)), (r_2(x), w_3(x)), (r_2(x), w_5(x)), \\ & (w_2(x), r_3(x)), (w_2(x), w_1(x)), (w_2(x), w_3(x)), \\ & (w_2(x), w_5(x)), (r_3(x), w_1(x)), (r_3(x), w_5(x)), \\ & (r_4(z), w_1(z)), (w_1(x), w_3(x)), (w_1(x), w_5(x)), \\ & (w_3(y), w_1(y)), (w_3(y), w_5(y)), (w_3(x), w_5(x)), \\ & (w_1(y), w_5(y)), (w_1(z), r_5(z)) \} \end{aligned}$$

Il grafo dei conflitti è ciclico, allora S_3 **non è conflict-serializzabile** e di conseguenza S_3 **non è view-serializzabile**.

Questo schedule lo avevamo verificato anche nell'esercizio ?? e avevamo concluso che S_3 non è view-serializzabile.

4.3.3 Locking a due fasi stretto

Il locking a due fasi (2PL, Two-Phase Locking) è il metodo applicato nei sistemi reali per la gestione dell'esecuzione concorrente delle transazioni perchè non richiede di conoscere in anticipo l'esito delle transazioni.

Il locking a due fasi è un sottoinsieme della conflict-serializzabilità.

Gli aspetti che caratterizzano il locking a due fasi sono i seguenti:

- ▷ **meccanismo di base per la gestione dei lock**: si basa sull'introduzione di alcune **primitive di lock** che consentono alle transazioni di bloccare le risorse sulle quali vogliono agire con operazioni di lettura e scrittura. Le primitive di lock sono le seguenti:

- ◊ $r_lock_K(x)$: richiesta di un lock **condiviso** da parte della transazione t_K sulla risorsa x per eseguire una lettura
- ◊ $w_lock_K(x)$: richiesta di un lock **esclusivo** da parte della transazione t_K sulla risorsa x per eseguire una scrittura
- ◊ $unlock_K(x)$: richiesta da parte della transazione t_K di liberare la risorsa x da un precedente lock

Le transazioni devono seguire le seguenti regole per applicare le primitive di lock:

- ogni **lettura** deve essere preceduta da un r_lock e seguita da un $unlock$. Sono ammessi più r_lock contemporanei sulla stessa risorsa (lock condiviso)
- ogni **scrittura** deve essere preceduta da un w_lock e seguita da un $unlock$. **Non** sono ammessi più w_lock (oppure w_lock e r_lock) contemporanei sulla stessa risorsa (lock esclusivo)

Se una transazione segue le precedenti regole, allora si dice **ben formata** rispetto al locking.

- ▷ **politica di concessione dei lock sulle risorse**: il gestore dei lock (o gestore della concorrenza) mantiene per ogni risorsa x le seguenti informazioni:

- ◊ **stato**: $s(x) \in \{\text{libero}, r_lock, w_lock\}$
- ◊ **transazioni in r_lock** : $c(x) = (t_1, \dots, t_n)$, dove $t_1, \dots, t_n$ hanno un r_lock su x

Il comportamento del gestore dei lock è descritto dalla seguente tabella:

Stato x	LIBERO		R_LOCK		W_LOCK	
Richiesta	Esito	Operazioni	Esito	Operazioni	Esito	Operazioni
$r_lock_K(x)$	OK	$s(x) = r_lock$ $c(x) = \{K\}$	OK	$c(x) = c(x) \cup \{K\}$	Attesa	-
$w_lock_K(x)$	OK	$s(x) = w_lock$	if $ c(x) =1$ and $K \in c(x)$ then Esito OK Operazioni $s(x) = w_lock$ else Attesa		Attesa	Operazioni -
$unlock_K(x)$	Errore	-	OK	$c(x) = c(x) - \{K\}$ if $c(x) = \emptyset$ then $s(x) = \text{Libero}$ verifica coda	OK	$c(x) = \emptyset$ $s(x) = \text{Libero}$ verifica coda

- ▷ **regola che garantisce la serializzabilità**: una transazione **dopo aver rilasciato un lock non può acquisirne altri**. Questa regola è visualizzata dal seguente grafico:

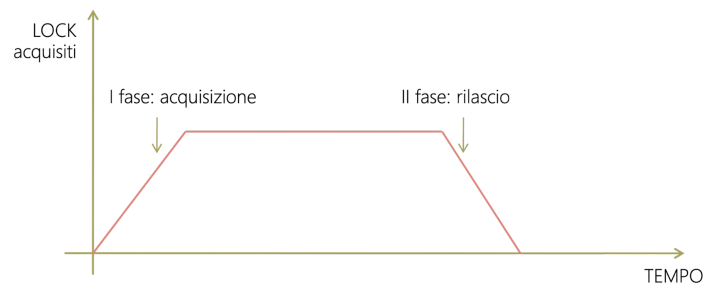


Figura 4.2: Locking a due fasi

Esempio: perdita di aggiornamento

Consideriamo le seguenti transazioni:

$$t_1 : r_1(x); w_1(x)$$

$$t_2 : r_2(x); w_2(x)$$

Eseguite con lo schedule:

$$S_{pa} : r_1(x); r_2(x); w_2(x); w_1(x);$$

1. $r_1(x)$ richiede un $r_lock_1(x)$
2. $r_2(x)$ richiede un $r_lock_2(x)$
3. $w_2(x)$ richiede un $w_lock_2(x)$, ma lo stato $s(x) = r_lock$, con $c(x) = \{t_1, t_2\}$, quindi $w_2(x)$ per ottenere $w_lock_2(x)$ deve attendere che t_1 rilasci $r_lock(x)$ ma se t_1 rilascia $r_lock(x)$ e quindi inizierebbe la sua fase discendente impedendogli di acquisire altri lock e di conseguenza $w_lock_1(x)$ non è più possibile per la serializzabilità del 2PL

Di conseguenza questo schedule non è 2PL.

Il locking a due fasi stretto (Strict 2PL) è una variante del locking a due fasi che introduce una condizione aggiuntiva per rimuovere la necessità dell'ipotesi di commit-proiezione. La condizione aggiuntiva è la seguente: Una transazione può rilasciare i lock solo quando ha eseguito correttamente un **commit** o un **rollback**.

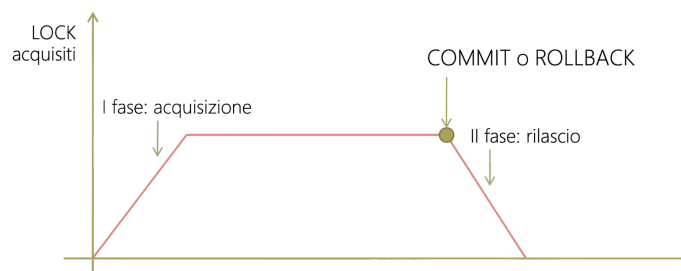


Figura 4.3: Locking a due fasi stretto

4.3.4 Blocco critico (deadlock)

Il blocco critico è una situazione di blocco che si verifica nell'esecuzione di transazioni concorrenti quando:

- due transazioni hanno bloccato delle risorse:
 - t_1 ha bloccato r_1
 - t_2 ha bloccato r_2
- inoltre t_1 è in attesa sulla risorsa r_2 e t_2 è in attesa sulla risorsa r_1

Se il numero medio di tuple per tabella è n e la granularità del lock (cioè il numero di risorse che possono essere bloccate) è la tupla, la probabilità che si verifichi un lock tra due transazioni è:

$$P(\text{deadlock di lunghezza } 2) = \frac{1}{n^2}$$

Esempio: deadlock

Un esempio di deadlock utilizzando il locking a due fasi stretto è il seguente:

t1	M.C.	Database			M.C.	t2
		c(x)	s(x)	Val		
bot		∅	L	2		
$r_lock_1(x)$		{1}	RL	2		
$r_1(x)$	2	{1}	RL	2		
$x = x + 1$	3	{1}	RL	2		
	3	{1}	RL	2		bot
	3	{1,2}	RL	2		$r_lock_2(x)$
	3	{1,2}	RL	2	2	$r_2(x)$
	3	{1,2}	RL	2	3	$x = x + 1$
	3	{1,2}	RL	2	3	$w_lock_2(x) \rightarrow$ ATTESA
$w_lock_1(x) \rightarrow$ ATTESA	3	{1,2}	RL	2	3	

BLOCCO CRITICO

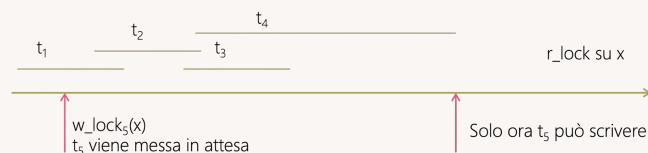
Le tecniche per risolvere i deadlock sono le seguenti:

- ◊ **Timeout:** se una transazione è in attesa su una risorsa da troppo tempo viene abortita
- ◊ **Prevenzione:**
 - Una transazione blocca tutte le risorse a cui deve accedere in lettura o scrittura in una sola volta (o blocca tutto o non blocca nulla)
 - Ogni transazione t_i acquisisce un timestamp TS_i all'inizio della sua esecuzione. La transazione t_i può attendere una risorsa bloccata da t_j solo se vale una certa condizione sui timestamp, ad esempio $TS_i < TS_j$, altrimenti t_i viene abortita e fatta ripartire con lo stesso timestamp
- ◊ **Rilevamento:** Si esegue un'analisi periodica della tabella di lock per rilevare la presenza di un deadlock che corrisponde ad un ciclo nel grafo delle relazioni di attesa. Questa tecnica è la più utilizzata nei sistemi reali.

Inoltre la scelta di gestire i lock in lettura come lock condivisi introduce il problema della **starvation** che tuttavia nel contesto delle DMBS risulta poco probabile, ma comunque risolvibile con le tecniche precedenti (in particolare il rilevamento).

Definizione: starvation

Se una risorsa x viene costantemente bloccata da una sequenza di transazioni che acquisiscono r_lock su x , un'eventuale transazione in attesa di scrivere su x viene bloccata per un lungo periodo fino alla fine della sequenza di letture.



4.3.5 Gestione della concorrenza in SQL

Siccome risulta molto pesante per il sistema gestire l'esecuzione concorrente di transazioni con una conseguente diminuzione delle prestazioni, SQL prevede la possibilità di **rinunciare del tutto o in parte al controllo di concorrenza** per aumentare le prestazioni del sistema. È quindi possibile, a livello di una singola transazione, decidere di **tollerare alcune anomalie** di esecuzione concorrente. La seguente tabella mostra le possibili combinazioni di anomalie che si possono tollerare:

Livello di isolamento	Perdita di update	Lettura sporca	Lettura inconsistente	Update fantasma	Inserimento fantasma
serializable	✓	✓	✓	✓	✓
repeatable read	✓	✓	✓	✓	
read committed	✓	✓			
read uncommitted	✓				

Figura 4.4: Anomalie tollerate in SQL

Nota

- ▷ Tutti i livelli richiedono il 2PL stretto per le scritture
- ▷ **Serializable:** Richiede il 2PL stretto anche per le letture e applica il **lock sulle tabelle** per evitare l'inserimento fantasma
- ▷ **Repeatable read:** Applica il 2PL stretto anche per le letture, ma applica i **lock a livello di tupla** e non di tabella. Consente inserimenti e quindi non evita l'anomalia di inserimento fantasma. **Attenzione:** in PostgreSQL la repeatable read è implementata in modo più stringente che evita anche l'anomalia di inserimento fantasma
- ▷ **Read committed:** Richiede lock condivisi per le letture, ma non il 2PL stretto
- ▷ **Read uncommitted:** Non applica lock in lettura

Ottimizzazione di interrogazioni

Ogni interrogazione sottomessa al DBMS viene espressa in un linguaggio dichiarativo (es. SQL).

E' necessario, quindi, trovare un'equivalente espressione procedurale (es. algebra relazionale) per generare un piano di esecuzione.

L'espressione algebrica va ottimizzata rispetto alle caratteristiche del DBMS a livello fisico e dalla base di dati corrente.

Le fasi di ottimizzazione di una query consistono in:

1. **analisi lessicale e sintattica**: verifica che la query sia scritta correttamente
2. **ottimizzazione algebrica**: si basa sulle regole dell'algebra relazionale
 - anticipo delle selezioni
 - anticipo delle proiezioni
3. **ottimizzazione dipendente dai metodi di accesso**: si basa sulle statistiche e sulle strutture fisiche (profili delle tabelle)
4. **generazione del codice**: si genera un piano di esecuzione per ottenere il risultato della query



Figura 5.1: processo di ottimizzazione di una query

5.1 Ottimizzazione dipendente dai metodi di accesso

Le operazioni tipiche di accesso supportate dal DBMS sono:

- ▷ **scansione delle tuple di una tabella**
- ▷ **ordinamento delle tuple**

- ▷ accesso diretto tramite indice
- ▷ join con diverse implementazioni

5.1.1 Scansione sequenziale

L'operazione di scansione opera contestualmente altre operazioni:

- ◇ scansione + proiezione (senza eliminazione dei duplicati)
- ◇ scansione + selezione (rispetto a un predicato semplice)
- ◇ scansione + inserimento/cancellazione/modifica

⇒ **costo scansione sequenziale**: $NP(R)$ (numero di pagine della relazione R)

5.1.2 Ordinamento

L'ordinamento viene utilizzato per:

- ◇ ordinare il risultato di un'interrogazione (clausola ORDER BY)
- ◇ eliminare duplicati (clausola SELECT DISTINCT)
- ◇ raggruppare tuple (clausola GROUP BY)

L'ordinamento viene eseguito su memoria secondaria utilizzando l'algoritmo **Z-way-Sort-Merge**:

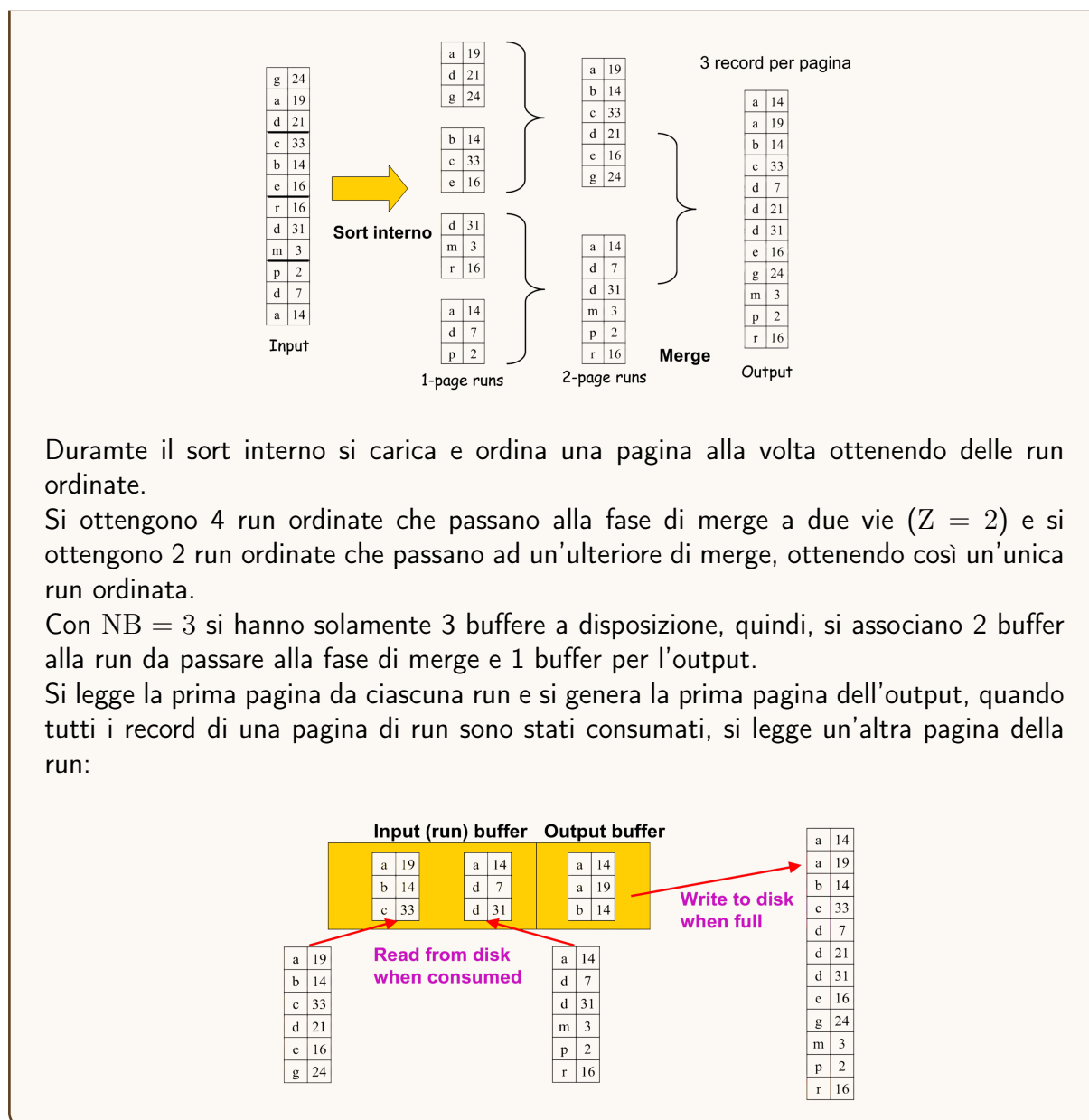
1. **sort interno**:
 - (a) si leggono una alla volta le pagine della tabella
 - (b) le tuple di ogni pagina vengono ordinate facendo uso di un algoritmo di sort interno (es. quick-sort)
 - (c) ogni pagina ordinata (run) viene scritta su memoria secondaria in un file temporaneo
2. **merge**: vengono unite le run in un'unica run

Esempio: algoritmo Z-way-Sort-merge

Supponiamo di dover ordinare un input che consiste di una tabella di NP pagine e di avere a disposizione sono NB buffer in memoria centrale ($NB < NP$).

Per semplicità, consideriamo il caso base a due vie ($Z = 2$) e supponiamo di avere solo 3 buffer in memoria centrale ($NB = 3$).

Si hanno 4 pagine composte da 3 tuple ciascuna e si vuole ordinare in base alla prima colonna:

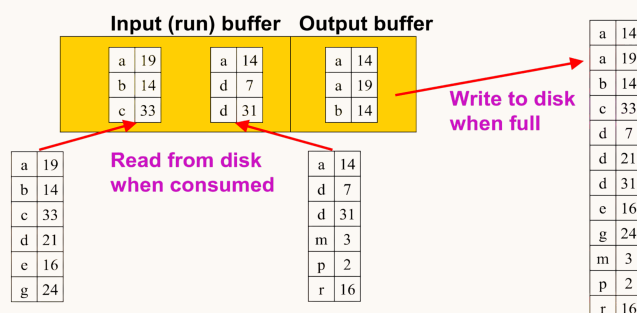


Durante il sort interno si carica e ordina una pagina alla volta ottenendo delle run ordinate.

Si ottengono 4 run ordinate che passano alla fase di merge a due vie ($Z = 2$) e si ottengono 2 run ordinate che passano ad un'ulteriore di merge, ottenendo così un'unica run ordinata.

Con $NB = 3$ si hanno solamente 3 buffer a disposizione, quindi, si associano 2 buffer alla run da passare alla fase di merge e 1 buffer per l'output.

Si legge la prima pagina da ciascuna run e si genera la prima pagina dell'output, quando tutti i record di una pagina di run sono stati consumati, si legge un'altra pagina della run:



⇒ **costo ordinamento** (con $Z = 2$ e $NB = 3$):

◇ fase di sort interno: si leggono $NP(R)$ pagine e si scrivono $NP(R)$ pagine

$$2 \cdot NP(R)$$

◇ fase di merge: si leggono $NP(R)$ pagine e si scrivono $NP(R)$ pagine:

$$2 \cdot NP(R)$$

Il numero di passi di merge è:

$$\lceil \log_2(NP(R)) \rceil$$

Il costo complessivo è:

$$2 \times NP(R) \times (\lceil \log_2(NP(R)) \rceil)$$

5.1.3 Accesso diretto tramite indice

Le interrogazioni che fanno uso dell'indice sono:

- ◇ **selezioni con condizione atomica di ugualianza** ($A = v$): richiede indici di hash o B+ tree
- ◇ **selezione con condizione di range** ($A \geq v_1 \wedge A \leq v_2$): richiede indice B+ tree
- ◇ **selezione con condizione costituita da una congiunzione di condizioni di ugualianza** ($A = v_1 \wedge B = v_2$): l'indice viene scelto in base alla condizione più selettiva, l'altra condizione viene verificata filtrando i record restituiti dall'indice
- ◇ **selezioni con condizione costituita da una disgiunzione di condizioni di ugualianza** ($A = v_1 \vee A = v_2$): gli indici possono essere usati in parallelo e poi fare un merge o unione dei risultati

5.1.4 Join

Il join è l'operazione più costosa da eseguire, quindi, l'ottimizzazione è particolarmente attento ad ottimizzare questa operazione, tanto da prevedere diversi algoritmi:

- ▷ **nested loop join**
- ▷ **merge scan join**
- ▷ **hash-based join**

Dal punto di vista logico è commutativo, ma dal punto di vista fisico c'è una differenza sostanziale tra $A \bowtie B$ e $B \bowtie A$ che cambia le prestazioni.

5.1.4.1 Nested loop join

Date 2 relazioni in input R e S tra cui è definito il predicato di join PJ, distinguiamo:

- **relazione esterna R**
- **relazione interna S**

L'algoritmo funziona come segue:

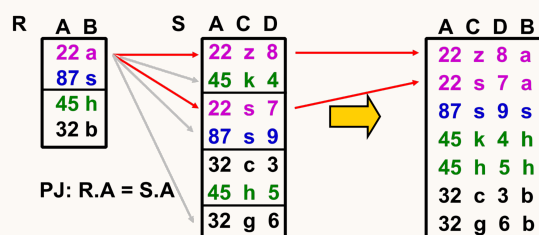
```

1  tupla t_R { --relazione esterna
2      \forall tupla t_S { --relazione interna
3          if (t_R, t_S) soddisfa PJ
4              aggiungi (t_R, t_S) al risultato
5          }
6      }

```

Esempio: algoritmo nested loop join

Consideriamo il seguente esempio di join tra le relazioni R e S:



R deve verificare per ogni riga di S che il predicato sia verificato.

⇒ **costo nested loop join** (consideriamo 1 pagina per caricare R e 1 pagina per caricare S):

- ◇ bisogna leggere R completamente una volta
 - ◇ per ogni riga di R ($NP(R) = \#RIGHER$) bisogna leggere completamente S
- Il costo complessivo è:

$$NP(R) + NR(R) \cdot NP(S)$$

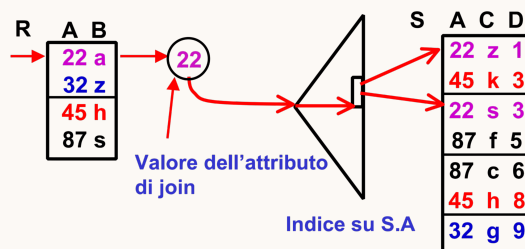
A seconda del numero di righe di R cambia il numero di volte che bisogna leggere $NP(R)$, quindi, la relazione esterna deve essere quella più piccola per minimizzare il costo.

5.1.4.2 Nested loop join con indice

Data una relazione esterna R e una interna S, si ha che la scansione completa di S può essere sostituita dalla scansione basata sull'indice costituito sugli attributi di join di S.

Esempio: algoritmo nested loop join con indice

Consieriamo il seguente esempio di nested loop join con indice tra le relazioni R e S:



L'indice su S.A permette di caricare solo i blocchi di S che contengono l'attributo $A = 22$.

⇒ **costo nested loop join** (usando B+ tree):

$$NP(R) + NR(R) \cdot \left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right)$$

La tabella R deve comunque essere caricata tutta ($NP(R)$) e rimane invariato anche il numero di volte che si esegue il ciclo esterno ($NR(R)$).

Ciò che cambia è il numero di blocchi/pagine di S che si devono caricare tutte le pagine di $\left(\left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right) \right)$.

L'idea è, invece, di caricare tutte le pagine di S si caricano solo quelle che contengono $S.A = R.A$

- ◇ $\frac{NR(S)}{VAL(A, S)}$: rappresenta la stima di quanti blocchi di S contengono A uguale al valore interessato.

Questa stima è anche selettività dell'attributo A.

- ◇ $VAL(A, S)$: numero di valori distinti di A che compaiono in S

Esempio:

Supponiamo che $A = \text{Nome_Regioni}$, quindi, il numero di possibili valori di A è 21.

Consideriamo:

- ◇ $NR(S) = 2100$
- ◇ numero di righe per ogni regione è:

$$\frac{NR(S)}{VAL(A, S)} = \frac{2100}{21} = 1000$$

quindi si ha l'assunzione che il dato sia uniforme, cioè che non siano più o meno lo stesso numero di pagine per ogni regione

- ◇ nel caso pessimo si ha un blocco/pagina per ogni riga

Quindi, il costo del nested loop join con indice è:

$$NP(R) + NR(R) \cdot \left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right)$$

$$NP(R) + NR(R) \cdot (\text{ProfonditàIndice} + 1000)$$

5.1.4.3 Block nested loop join

Il blocco nested loop join è una variante del nested loop in cui ciascuna pagina di S (tabella interna) viene letta per ogni pagina di R (tabella esterna), invece, che per ogni riga di R .

Lo pseudocodice è il seguente:

```

1  pagina P_R di R { --relazione esterna
2      \forall pagina P_S di S { --relazione interna
3          \forall tupla t_R di P_R {
4              \forall tupla t_S in P_S {
5                  if (t_R , t_S) soddisfa PJ
6                      aggiungi (t_R , t_S) al risultato
7              }
8          }
9      }
10 }
```

⇒ **costo block nested loop join**: dipende dallo spazio a disposizione nei buffer.

Supponiamo di avere 1 buffer per R e 1 buffer per S , allora:

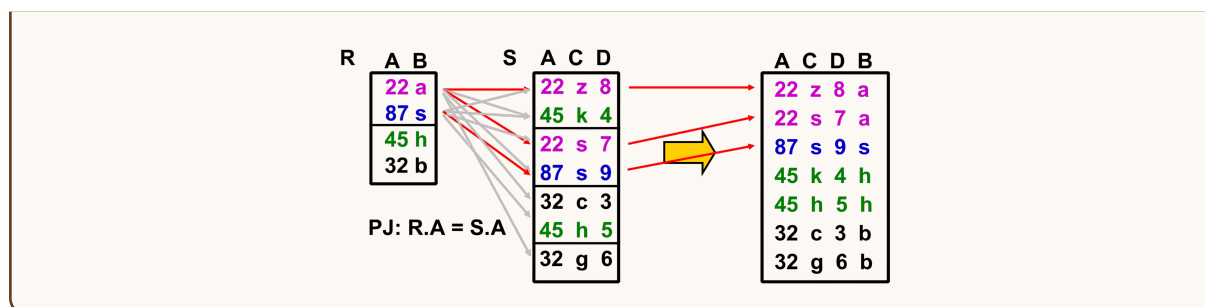
- ◇ bisogna leggere R completamente una volta, quindi $NP(R)$
- ◇ per ogni pagina di R ($NP(R)$) bisogna leggere completamente S quindi $NP(R) \cdot NP(S)$

Il costo totale è:

$$NP(R) + NP(R) \cdot NP(S)$$

Esempio: algoritmo block nested loop join

Consideriamo il seguente esempio di block nested loop join tra le relazioni R e S :



5.1.4.4 Merge scan join

Il merge scan join è applicabile quando entrambi gli insiemi di tuple in input sono ordinati sugli attributi di join e il join è un equi-join (predicato di join è una condizione di uguaglianza).

Ciò accade sia per entrambe le relazioni R e S vale almeno una delle seguenti condizioni:

- relazione è fisicamente ordinata sugli attributi di join (file sequenziale ordinato come struttura fisica)
- esiste un indice sugli attributi di join della tabella che consente una scansione ordinata delle tuple

La logica di questo algoritmo sfrutta il fatto che entrambi gli input sono ordinati per evitare di fare confronti inutili, e questo fa sì che il numero di letture totali sia:

$$NP(R) + NP(S)$$

se si accede sequenzialmente alle due relazioni ordinate fisicamente.

- con indici B+ tree su entrambe relazioni il costo può arrivare al massimo (con indici nel buffer) a:

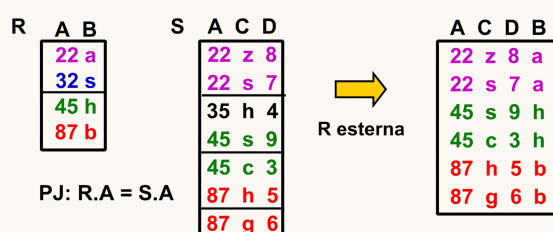
$$NR(R) + NR(S)$$

- con una relazione ordinata e un solo indice il costo è al massimo:

$$NP(R) + NR(S) \text{ (o viceversa)}$$

Esempio: algoritmo merge scan join

Consideriamo il seguente esempio di merge scan join tra le relazioni R e S:



5.1.4.5 Hash-based join

L'algoritmo di hash-based join è applicabile solo in caso di equi-join, non richiede né la presenza di indici né input ordinati e risulta particolarmente vantaggioso in caso di relazioni molto grandi.

L'idea sui casi si basa l'hash join è semplice: si suppone di avere a disposizione una funzione hash h , che viene applicata agli attributi di join delle due relazioni (es. $R.J$ e $S.J$).

- se t_R e t_S sono 2 tuple di R e S , allora è possibile che sia $t_R.J = t_S.J$ solo se $h(t_R.J) = h(t_S.J)$
- se, viceversa, $h(t_R.J)$ diverso $h(t_S.J)$, allora si ha $t_R.J$ diverso $t_S.J$

Da questa idea si hanno diverse implementazioni, che hanno in comune il fatto che R e S vengono partizionate sulla base dei valori della funzione di hash h , e che la ricerca di matching tuples avviene solo tra partizioni relative allo stesso valore di h .

Il costo di questo algoritmo è:

- costruzione dell'hash map:

$$NP(R) + NP(S)$$

- accesso alle matching tuples che nel caso pessimo può costare:

$$NP(R) \cdot NP(S)$$

5.2 Scelta del piano esecuzione

Data un'espressione ottimizzata in algebra i passi da eseguire sono:

1. si generano tutti i possibili piani di esecuzione (alberi) alternativi (o un sottoinsieme) ottenuti considerando le seguenti dimensioni
 - operatori alternativi applicabili per l'accesso ai dati: scan sequenziale o via indice
 - operatori alternativi applicabili nei nodi: nested loop join o hash based join
 - ordine delle operazioni da compiere (associatività)
2. si valuta con formule approssimate il costo di ogni alternativa in termini di accessi alla memoria secondaria richiesti (è una stima)
3. si sceglie l'albero con il costo stimato minimo

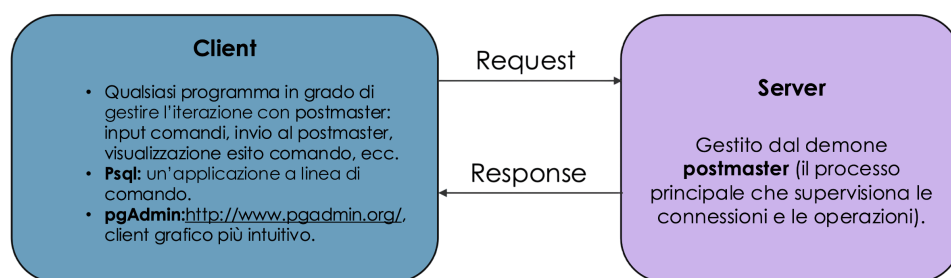
Nella valutazione delle stime di costo si tiene conto del profilo delle relazioni, solitamente memorizzato nel Data Directory e contenente per ogni relazione T :

- stima della cardinalità ($\#$ tuple): $CARD(T)$
- stima della dimensione di una tupla: $SIZE(T)$
- Stima della dimensione di ciascun attributo A : $VAL(A,T)$
- stima del numero di valori distinti per ogni attributo A : $MAX(A,T)$ e $MIN(A,T)$

Laboratorio

PostgreSQL è un **R**elational **D**ata **B**ase **M**anagement **S**ystem con alcune funzionalità orientate agli oggetti.

L'interazione tra un utente (programmatore/utente finale) e le basi di dati gestite da PostgreSQL avviene secondo il modello client-server.



6.1 Comando CREATE TABLE

Il comando CREATE TABLE definisce uno schema di relazione e ne crea un'istanza vuota, specificando attributi, domini e vincoli.

La sintassi è:

```

1 CREATE TABLE tabella(
2     attributo dominio [valoreDefault] {vincolo},
3     attributo dominio [valoreDefault] {vincolo},
4     {vincoloTabella}
5 );
  
```

dove:

- ▷ []: indicano parti opzionali
- ▷ [valoreDefault]: indica un eventuale valore di default
- ▷ {}: indica un elemento che può essere ripetuto 0 o più volte
- ▷ {vincolo}: indica eventuali (0 o più) vincoli per ogni attributo

Utilizzo di CREATE TABLE

```
1 CREATE TABLE Impiegato (  
2   Matricola CHARACTER(6) PRIMARY KEY,  
3   Nome VARCHAR(20) NOT NULL,  
4   Cognome VARCHAR(20) NOT NULL,  
5   Dipart VARCHAR(15),  
6   Stipendio NUMERIC(9) DEFAULT 0,  
7   FOREIGN KEY(Dipart)  
8     REFERENCES Dipartimento(NomeDip),      -- vincolo di tabella  
9   UNIQUE (Cognome, Nome)                    -- vincolo di tabella  
10 );
```

6.1.1 Identificatori

Definizione: identificatore

Un **identificatore** è una sequenza di caratteri che identifica un elemento del database (tabella, attributo, vincolo, ecc.).

Gli identificatori, in SQL, devono rispettare alcune regole:

- ◇ iniziare con una lettera (UTF-8) o un underscore (_)
- ◇ possono contenere lettere, numeri (0-9) o underscore (_)
- ◇ non possono essere parole chiave riservate a SQL

Nota

Gli identificatori **non sono case sensitive!** (idPersona = idpersona)

6.1.2 Domini elementari

Definizione: domini elementari

I domini elementari (predefiniti) sono:

- ▷ **caratteri**: singoli caratteri o stringhe anche di lunghezza variabile
- ▷ **bit**: bit singoli (booleani) o stringhe di bit
- ▷ **numeri esatti e approssimati**
- ▷ **istanti temporali**
- ▷ **intervalli temporali**

6.1.2.1 Tipo carattere

Permette di rappresentare singoli caratteri oppure stringhe.

Un carattere o stringa di caratteri sono rappresentati tra apici ('): 'a', 'Questa è una stringa'.

La sintassi è:

```
1 CHARACTER [VARYFING] [(lunghezza)]
```

dove:

- ▷ lunghezza: può essere fissa o variabile
 - ◇ CHARACTER: carattere singolo
 - ◇ CHARACTER(20): stringa di lunghezza fissa (20 caratteri).
Se la stringa è più corta di 20 caratteri, viene riempita con spazi a destra
 - ◇ CHARACTER VARYFING(20) o VARCHAR(20): stringa di lunghezza variabile (max. 20 caratteri)
 - ◇ TEXT: stringa di lunghezza variabile senza limite fissato

6.1.2.2 Tipo boolean

Permette di rappresentare valori booleani (true o false), più lo stato nullo.

La sintassi è:

```
1 BOOLEAN
```

La rappresentazione dei valori booleani può essere:

- TRUE o FALSE
- t o f
- 1 o 0
- yes o no
- NULL

6.1.2.3 Tipo numerico esatto

Permettono di rappresentare valori esatti (interi o con una parte decimale di lunghezza prefissata):

- ▷ **valori interi:**
 - ◇ SMALLINT: valori interi (2 bytes: da -32.768 (215) a +32.767 (215-1))
 - ◇ INTEGER: valori interi (4 bytes: da -2147483648 (231) to +2147483647 (231-1))
- ▷ **valori decimali:**
 - ◇ NUMERIC [(precisione [, scala])]:
 - precisione: numero totale di cifre significative (cifre a sinistra e a destra della virgola)
 - scala: numero di cifre dopo la virgola (vuoto = 0)
 - ◇ DECIMAL [(precisione [, scala])]: equivalente al precedente

Esempio: tipo numerico esatto

NUMERIC(5,2) permette di rappresentare valori come 100.01, 100.999 (arrotondato a 101.00), ma non valori come 1000.01

6.1.2.4 Tipo numerico approssimato

Permettono di rappresentare valori in virgola mobile:

- ◇ REAL: precisione di 6 cifre decimali
- ◇ DOUBLE PRECISION: precisione di 15 cifre decimali

Ciascun numero è rappresentato da una coppia di valori: **mantissa** (numero frazionario) + **esponente** (numero intero).

Il valore si ottiene moltiplicando la mantissa per la potenza di 10 con grado pari all'esponente:

- numero 0.17E16 corrisponde a $1,7 \times 10^{15}$
- numero -0.4E-6 corrisponde a -4×10^{-7}

6.1.2.5 Istanti temporali

Permettono di rappresentare istanti di tempo.

- ◇ DATE: istanti del tipo (anno, mese, giorno)
Si rappresenta tra apici (es. '2025-12-01') e lo standard ISO prescrive il formato YYYY-MM-DD
- ◇ TIME [(precisione)] [WITH TIME ZONE]: istanti del tipo (hour, minute, second)
 - precisione: numero cifre decimali per rappresentare frazioni del secondo
 - WITH TIME ZONE: permette di specificare anche [+/-] hour: minute, che rappresentano la differenza con l'ora di Greenwich, o nomeFusoOrario
- ◇ TIMESTAMP [(precisione)] [WITH TIME ZONE]: date + time

6.1.2.6 Intervalli temporali

Rappresenta un intervallo di tempo.

La sintassi è:

```
1 INTERVAL PrimaUnitaTempo [TO UltimaUNitaTempo]
```

dove:

- ▷ PrimaUnitàDiTempo e UltimaUnitàDiTempo: definiscono le unità di misura che devono essere utilizzate, dalla più precisa alla meno precisa

L'insieme delle unità di misura è diviso in due insiemi distinti:

- Year-Month
- Day-Hour-Minute-Second

6.1.3 Domini definiti dall'utente

Per creare un dominio si esegue il comando:

```
1 CREATE DOMAIN nome AS tipoBase [valoreDefault][vincolo]
```

dove:

- ▷ CREATE DOMAIN: definisce un dominio, utilizzabile in definizioni di relazioni, anche con vincoli e valori di default

Il dominio può essere definito a partire da un dominio elementare oppure da un altro dominio definito dall'utente.

Comando CREATE DOMAIN

```
1 CREATE DOMAIN giorniSettimana AS CHARACTER(3)
2 CHECK(VALUE IN('LUN', 'MAR', 'MER', 'GIO', 'VEN', 'SAB', 'DOM'))
3 ;
4 CREATE DOMAIN Voto AS SMALLINT DEFAULT NULL
5 CHECK (value >= 18 AND value <= 30);
```

6.1.4 Vincoli intrarelazionali

I vincoli intrarelazionali sono vincoli che si applicano a livello di relazione, cioè a una singola tabella.

I principali vincoli intrarelazionali sono:

- ◇ NOT NULL: valore nullo non è ammesso per l'attributo, quindi il valore deve sempre essere specificato, o si assegna un valore di default
- ◇ UNIQUE: definisce (super)chiavi

```
1 Nome VARCHAR(20),
2 Cognome VARCHAR(20),
3 UNIQUE(Nome, Cognome)
4 --impone che non ci siano due righe che abbiano uguale
5 --sia il nome che il cognome
```

```
1 Nome VARCHAR(20) UNIQUE,
2 Cognome VARCHAR(20) UNIQUE
3 --impone che non ci siano due righe che abbiano lo stesso nome
4 --o lo stesso cognome
```

- ◇ PRIMARY KEY: chiave primaria (una sola, implica NOT NULL)

```
1 matricola CHAR(6) PRIMARY KEY;
2 --se unico componente della chiave primaria
```

```

1 nome VARCHAR(20),
2 cognome VARCHAR(20),
3 PRIMARY KEY(nome, cognome)
4 --se la chiave primaria ha piu' attributi

```

- ◇ CHECK: specifica un vincolo generico che deve soddisfare le tuple della tabella. Viene soddisfatto se la sua espressione è vera o nulla

```

1 CREATE TABLE Impiegato(
2   matricola CHAR(6) PRIMARY KEY,
3   nome VARCHAR(20) NOT NULL,
4   cognome VARCHAR(20) NOT NULL,
5   qualifica VARCHAR(20),
6   stipendio NUMERIC(8,2) DEFAULT 500.00 NOT NULL
7   CHECK(stipendio >= 0.0),      --check di attributo
8   UNIQUE(cognome, nome),
9   CHECK(nome <> cognome)        --check di tabella
10 )

```

6.1.5 Vincoli interrelazionali

Un vincolo di integrità referenziale (interrelazionale) crea un legame tra i valori di un attributo (o di un insieme di attributi) A della tabella corrente (interna/slave) e i valori di un attributo (o di un insieme di attributi) B di un'altra tabella (esterna/master).

Il vincolo impone che:

- in ogni tupla della tabella interna, il valore di A, se diverso dal valore nullo, sia presente tra i valori di B nella tabella esterna
- l'attributo B della tabella esterna deve essere soggetto a un vincolo UNIQUE (o PRIMARY KEY)

Nota:

L'attributo (o insieme di attributi) B può anche non essere la chiave primaria, ma deve essere identificante per le tuple della tabella esterna.

REFERENCES e **FOREIGN KEY** permettono di definire vincoli di integrità referenziale.

Per singoli attributi, basta utilizzare il costrutto **REFERENCES**.

```

1 CREATE TABLE Impiegato (
2   Matricola CHARACTER(6) PRIMARY KEY,
3   Nome CHARACTER VARYING(20) NOT NULL,
4   Cognome CHARACTER VARYING(20) NOT NULL,
5   Dipart CHARACTER VARYING(15),
6   REFERENCES Dipartimento(NomeDip),  --attributo chiave (singolo) di
   tabella esterna
7   Ufficio NUMERIC(3),
8   Stipendio NUMERIC(9) DEFAULT 0,
9   UNIQUE(Cognome, Nome)

```


10)

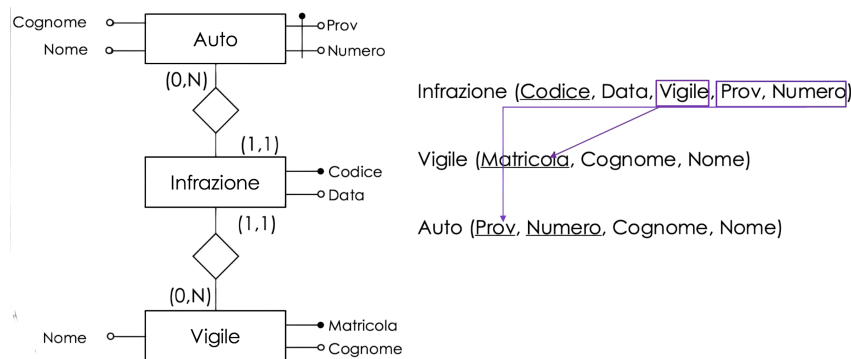
Per attributi multipli, si usa il costrutto **FOREIGN KEY** alla fine della definizione degli attributi, seguito da **REFERENCES**.

```

1 CREATE TABLE Impiegato (
2   Matricola CHARACTER(6) PRIMARY KEY,
3   Nome CHARACTER VARYING(20) NOT NULL,
4   Cognome CHARACTER VARYING(20) NOT NULL,
5   Dipart CHARACTER VARYING(15),
6   REFERENCES Dipartimento(NomeDip),
7   Ufficio NUMERIC(3),
8   Stipendio NUMERIC(9) default 0,
9   FOREIGN KEY(Nome, Cognome)
10  REFERENCES Anagrafica (Nome, Cognome) --tabella esterna con
11  attributi chiave

```

Vediamo un esempio più concreto:



```

1 CREATE TABLE Infrazione(
2   Codice CHAR(6) PRIMARY KEY,
3   Data DATE NOT NULL,
4   Vigile INTEGER NOT NULL
5   REFERENCES Vigile(Matricola),
6   Provincia CHAR(2),
7   Numero CHAR(6) ,
8   FOREIGN KEY(Provincia, Numero)
9   REFERENCES Auto(Provincia, Numero)
10 )

```

6.2 Modifica degli schemi

- ▷ ALTER: effettua modifiche (a domini, tabelle...), ad esempio aggiungere una colonna a una relazione
- ▷ DROP DOMAIN: permette di rimuovere i domini
- ▷ DROP TABLE: cancella una tabella

6.2.1 Comando ALTER TABLE

Il costrutto ALTER permette di:

- aggiungere un nuovo attributo (ADD COLUMN):

```
1 ALTER TABLE tabella
2 ADD COLUMN attributo tipo;
3
4 ALTER TABLE impiegato
5 ADD COLUMN stipendio NUMERIC(8,2);
```

- rimuovere un attributo (DROP COLUMN):

```
1 ALTER TABLE tabella
2 DROP COLUMN attributo;
3
4 ALTER TABLE impiegato
5 DROP COLUMN stipendio;
```

- modificare valore di default di un attributo (ALTER COLUMN):

```
1 ALTER TABLE tabella
2 ALTER COLUMN attributo { SET DEFAULT valore | DROP DEFAULT };
3
4 ALTER TABLE impiegato
5 ALTER COLUMN stipendio SET DEFAULT 1000.00;
```

6.3 Comando DELETE e DROP TABLE

Le tuple di una tabella vengono cancellate con il comando DELETE:

```
1 DELETE FROM tabella [ WHERE condizione ];
```

dove:

- ▷ condizione: espressione booleana che seleziona quali tuple cancellare

Nota

Se non viene specificata la condizione WHERE, verranno cancellate tutte le tuple.

```
1 DELETE FROM impiegato WHERE matricola = 'A001';
```

Una tabella viene cancellata con il comando DROP TABLE:

```
1 DROP TABLE tabella;
```

6.4 Comando INSERT INTO e UPDATE

Il comando INSERT INTO permette di inserire valori all'interno di una tupla:

```

1 INSERT INTO Tabella [ ( Attributi ) ]
2 VALUES( Valori )
3
4 --oppure
5
6 INSERT INTO Tabella [ ( Attributi )]
7 (SELECT ...)
```

Utilizzo di INSERT INTO

```

1 INSERT INTO Museo(nome, citta, indirizzo, numeroTelefono,
   giornoChiusura, prezzo)
2 VALUES('Arena', 'Verona', 'piazza Bra', '045 8003204', 'MARTEDI',
   20);
```

Il comando UPDATE permette di aggiornare i valori all'interno di una tupla:

```

1 UPDATE NomeTabella
2 SET Attributo = < Espressione | SELECT ... | NULL | DEFAULT >
3 [ WHERE Condizione ]
```

Utilizzo di INSERT INTO

```

1 UPDATE Museo SET sitoInternet = 'www.ciao.it'
2 WHERE citta = 'Verona';
```

6.5 Selezione

Struttura base

```

1 SELECT [ DISTINCT ]
2 [ * | expression [[ AS] output_name ] [,
3 ...] ]
4 FROM from_item [, ...]
5 [ WHERE condition ]
6 [ GROUP BY grouping_element [, ...] ]
7 [ HAVING condition [, ...] ]
8 [ { UNION | INTERSECT | EXCEPT } [ DISTINCT
9 ] other_select ]
10 [ ORDER BY expression [ ASC | DESC | USING
11 operator ]]
```

6.5.1 Clausola SELECT

```
1 SELECT [ DISTINCT ]
2 [{* | expression [[ AS] output_name ]] [, ...]]
```

***** è un'abbreviazione per indicare tutti gli attributi delle tabelle.

expression è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).

output_name è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione **expression** nella relazione risultato.

DISTINCT: se presente richiede l'eliminazione delle tuple duplicate.

L'esecuzione di una **SELECT** produce una relazione che:

- ha come schema tutti gli attributi elencati nella clausola **SELECT**;
- ha come contenuto tutte le tuple ottenute proiettando sugli attributi indicati nella **SELECT** provenienti dalle tabelle del **FROM** (prodotto cartesiano), limitate a quelle che soddisfano le eventuali condizioni specificate nelle clausole **WHERE**, **GROUP BY** e **HAVING**

6.5.1.1 Target List

```
1 SELECT *
2 FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

```
1 SELECT Attributo1,
2        Attributo2
3 FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

Esempio

```
1 SELECT Nome, Reddito
2 FROM Persone
3 WHERE Eta < 30
```

Nome	Reddito
Andrea	21
Aldo	15
Filipo	30

Figura 6.1: selezionati solo le tuple dove è rispettata la condizione

6.5.1.2 Concatenazione stringhe

L'operatore di concatenazione **||** è un operatore scalare (o operatore di riga).

Lavora su una singola riga alla volta. Se la tua tabella ha 100 righe, il costrutto `SELECT titolo || ' - ' || città` calcolerà e restituirà 100 risultati testuali separati, uno per ogni riga.

nota

Non è un'esclusiva di `SELECT`, può infatti essere utilizzato anche nella clausola `WHERE` e nella clausola `ORDER BY`

Concatenazione in clausola `WHERE` (per filtrare i dati)

```
1 -- Trova la riga in cui l'unione di nome e cognome da esattamente '
  Mario Rossi'
2 WHERE nome || ' ' || cognome = 'Mario Rossi'
```

Concatenazione in clausola `ORDER BY` (per ordinare i risultati)

```
1 -- Ordina i risultati basandosi sull'unione testuale dei due campi
2 ORDER BY titolo || città
```

6.5.2 Clausola `FROM`

```
1 FROM from_item [, ...]
```

1. `table_name` `[[AS] alias [(column_alias [, ...])]]`
 - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nel `SELECT` questi attributi sono identificati antepoendo il nome della tabella e un `'.'`: `nomeTabella.nomeAttributo`
2. `(other_select) [AS] nomeRisultato [(column_alias [,...])]`
 - è una `SELECT` innestata. Il risultato della `SELECT` interna è lo schema con nome `nomeRisultato` su cui fare la `SELECT` corrente.
3. `from_item [NATURAL] join_type from_item [ON condition]`
 - è la clausola `JOIN`.
 - Metodo più sofisticato di 1) che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. `join_type` specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti

6.5.2.1 Uso di alias

```
1 SELECT P.Reddito, R.Telefono
2 FROM Reparti as R, Persona ad P
3 WHERE ... ;
```

Utile quando è necessario specificare la provenienza dell'attributo, perchè nella base di dati sono presenti più attributi con lo stesso nome

6.5.2.2 Target List

```

1 SELECT P.Nome as NPersona, P.Reddito as RPersona
2 FROM Persone as P
3 WHERE P.Eta < 30

```

6.5.3 Clausola WHERE

```

1 WHERE condition

```

condition è un'espressione booleana combinando condizioni semplici C_i con gli operatori AND, OR, NOT.

1. $C_i := \text{expr} [\theta \{ \text{expr} \mid \text{const} \}]$ dove
 - $\text{expr} :=$ espressione che contiene riferimenti agli attributi delle tabelle che compaiono nella clausola FROM.
 - $\theta \in \{=, <, >, <=, >=, <>\}$
 - $\text{const} :=$ valore dei domini di base.
2. Una riga soddisfa **condition** se l'espressione è vera quando valutata con i valori della riga.
3. Una riga che non soddisfa **condition** è scartata dal risultato finale

```

1 SELECT Nome, Reddito
2 FROM Persone
3 WHERE Eta < 30;

```

```

SELECT Nome, Reddito FROM Persone
WHERE Eta < 30 OR Reddito > 50;

```

6.5.3.1 Operatore LIKE

```

1 WHERE attributo [NOT] LIKE 'pattern';

```

Nella clausola WHERE può apparire l'operatore LIKE per il confronto di stringhe. LIKE è un operatore di pattern matching. I pattern si costruiscono con i caratteri speciali `_` e `%`:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.

```

1 SELECT Nome, Reddito
2 FROM Persona
3 WHERE Nome LIKE 'A\%a'

```

Nome	Reddito
Andrea	21
Anna	35

6.5.3.2 Operatore SIMILAR TO

L'operatore **SIMILAR TO** è un **LIKE** più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL). Esempi di componenti di espressioni regolari:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.
- `*` = ripetizione del precedente match 0 o più volte.
- `+` = ripetizione del precedente match UNA o più volte.
- `{n,m}` = ripetizione del precedente match almeno n e non più di m volte.
- `[ ... ]` = ... è un elenco di caratteri ammissibili

```
1 SELECT *
2 FROM Persone
3 WHERE Nome SIMILAR TO '[AFO]'
```

Per trovare nomi che iniziano per A,F o O

6.5.3.3 Operatore BETWEEN

```
1 WHERE attributo [NOT] BETWEEN valore_iniziale AND valore_finale;
```

Nella clausola **WHERE** può apparire l'operatore **BETWEEN** per testare l'appartenenza di un valore ad un intervallo (non limitato ai valori numerici, usato anche con date e stringhe)

```
1 SELECT *
2 FROM Persone
3 WHERE Eta BETWEEN 20 AND 40
```

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Filipo	26	30
Olga	30	41

6.5.3.4 Operatore IN

```
1 WHERE attributo [NOT] IN ( valore [, ...] );
```

Nella clausola **WHERE** può apparire l'operatore **[NOT] IN** per testare l'appartenenza di un valore ad un insieme.

```
1 SELECT *
2 FROM Persone
3 WHERE Nome NOT IN ('Andrea', 'Franco')
```

6.5.3.5 Operatore IS NULL

```
1 WHERE attributo IS [ NOT ] NULL;
```

Nella clausola WHERE può apparire l'operatore **IS [NOT] NULL** per testare se un valore è **NOT KNOWN (NULL)** o no. Il valore NULL non rappresenta lo zero o una stringa vuota, ma l'assenza di dati o un valore sconosciuto. A partire dalle versioni SQL-2 è stata adottata una logica a tre valori (true, false, unknown) e i valori nulli non sono considerati né veri, né falsi.

NON SI PUÒ usare '=' o '<>' con il valore NULL!

```
1 SELECT *
2 FROM Persone
3 WHERE Reddito IS NOT NULL
```

6.5.3.6 Operatore ORDER BY

```
1 ORDER BY attributo [{ ASC | DESC }] [, ... ];
```

La clausola ORDER BY ordina le tuple del risultato in ordine rispetto agli attributi specificati, ASCendente è lo standard.

```
1 SELECT Nome
2 FROM Persone
3 ORDER BY Nome DESC
```

6.5.3.7 Operatori di aggregazione

Sono operatori che permettono di determinare un valore considerando i valori ottenuti da una SELECT. Due tipi principali:

- COUNT.
- MAX, MIN, AVG, SUM.

```
1 SELECT MAX(Reddito)
2 FROM Persone;
```

Nota

Quando si usano gli operatori aggregati, dopo la SELECT non possono comparire espressioni che usano i valori presenti nelle singole tuple perché il risultato è sempre e solo una tupla

1. Count


```
1 COUNT ({ * | expr | ALLexpr | DISTINCTexpr })
```

Restituisce il numero di tuple significative nel risultato dell'interrogazione. Tre casi comuni:

- **COUNT(*)** ritorna il numero di tuple nel risultato dell'interrogazione.
- **COUNT(expr)** ritorna il numero di tuple in ciascuna delle quali il valore expr è non nullo
- **COUNT(ALL expr)** include duplicati e valori nulli.
- **COUNT(DISTINCT expr)** come con **COUNT(expr)** ma con l'ulteriore condizione che i valori di expr siano distinti

2. SUM/MAX/MIN/AVG

```
1 op ({ expr | DISTINCTexpr })
```

Determinano un valore numerico (SUM/AVG) o alfanumerico (MAX/MIN) considerando le tuple significative nel risultato dell'interrogazione.

- op = SUM | AVG | MAX | MIN;
- expr è un'espressione che usa uno o più attributi e funzioni di attributi.
- DISTINCT considero solo i valori distinti.

```
1 SELECT AVG(Eta) AS Media\_Eta
2 FROM Persone;
```

6.6 Esempi di interrogazioni

6.6.1 Interrogazione con raggruppamento: Group by

Un raggruppamento è un insieme di tuple che hanno medesimi valori su uno o più attributi caratteristici del raggruppamento.

La clausola GROUP BY attr [, ...] permette di determinare tutti i raggruppamenti delle tuple della relazione risultato (tuple selezionate con la clausola WHERE) in funzione degli attributi dati.

In una interrogazione che usa GROUP BY, la clausola SELECT contiene solamente gli attributi (da 0 a tutti) utilizzati per il raggruppamento ed eventuali funzioni di aggregazione sugli altri attributi.

Visualizzare tutte le città raggruppate della tabella Studente

```
1 SELECT città
2 FROM studente
3 GROUP BY città
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	Via nobile	Verona	27.50
IN0002	Paoli	Paolo	Via dritta	Padova	28.6666
IN0003	Bianchi	Dante	Via storta	VERONA	18.345
IN0004	Rossi	Matteo	Via nobile	Verona	24.567
IN0005	Verdi	Luca	Via nuova	Va	28.6666
IN0006	Nocasa	Caio			

città
Padova
Verona
VERONA
Va

Visualizzare tutte le città e i cognomi raggruppati

```
1 SELECT cognome, LOWER(città)
2 FROM Studente
3 GROUP BY cognome, LOWER(città);
```

cognome	città
Verdi	Va
Rossi	verona
Paoli	padova
Nocasa	
Bianchi	verona

Nota

Nel GROUP BY non si possono usare espressioni con operatori di aggregazione.

Nota

Nella SELECT con GROUP BY non si possono specificare attributi che non sono raggruppati.

- La clausola WHERE permette di selezionare le righe che devono far parte del risultato.
- La clausola HAVING permette di selezionare i raggruppamenti che devono far parte del risultato.
- La sintassi è HAVING bool_expr, dove bool_expr è un'espressione booleana che può usare gli attributi usati nel GROUP BY e/o gli altri attributi mediante operatori di aggregazione.

Visualizzare tutte le città raggruppate che iniziano con 'V' della tabella Studente.

```
1 SELECT città
2 FROM Studente
3 GROUP BY città
4 HAVING città LIKE 'V%';
```

Città
Verona
VERONA
Va

Visualizzare tutte le città e i cognomi raggruppati con almeno due studenti con lo stesso cognome..

```
1 SELECT cognome, LOWER(città)
2 FROM Studente
3 GROUP BY cognome, LOWER(città)
4 HAVING COUNT ( cognome) >1;
```

cognome	città
Rossi	verona

Si possono specificare espressioni con operatori di aggregazione su attributi non raggruppati anche nella clausola HAVING.

```

1 SELECT LOWER(città) AS "Città",
   AVG (media)::DECIMAL(5 ,2) AS "
   mediaArr."
2 FROM Studente
3 GROUP BY LOWER(città)
4 HAVING AVG(media) >23.47 ;

```

Città	mediaArr
padova	28.67
va	28.67
Verona	23.47

6.6.2 Interrogazione con JOIN

```

1 TABLE1 join_type TABLE2 ON join_condition[, ...].

```

INNER JOIN, **LEFT [OUTER] JOIN**, **RIGHT [OUTER] JOIN** e **FULL [OUTER] JOIN**. Dove **join_condition** è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola **WHERE**. Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**.

Interrogazioni con INNER JOIN

```

1 table1 [INNER] JOIN table2 ON join_condition[, ...]

```

Rappresenta il tradizionale θ join dell'algebra relazionale. Combina ciascuna riga r_1 di `table1` con ciascuna riga di `table2` che soddisfa la condizione della clausola ON.

6.6.3 Interrogazioni con LEFT OUTER JOIN

```

1 table1 LEFT[OUTER] JOIN table2 ON join_condition[, ...].

```

Si esegue un INNER JOIN. Poi, per ciascuna riga r_1 di `table1` che non soddisfa la condizione con qualsiasi riga di `table2`, si aggiunge una riga al risultato con i valori di r_1 e assegnando NULL agli altri attributi..

Il LEFT [OUTER] JOIN non è simmetrico! Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

6.6.4 Interrogazioni con RIGHT OUTER JOIN

```

1 table1 RIGHT[OUTER] JOIN table2 ON join_condition[, ...].

```

Si esegue un INNER JOIN. Poi, per ciascuna riga r_2 di `table2` che non soddisfa la condizione con qualsiasi riga di `table1`, si aggiunge una riga al risultato con i valori di r_2 e assegnando NULL agli altri attributi..

6.6.5 Interrogazioni con FULL OUTER JOIN

```
1 table1 FULL[OUTER] JOIN table2 ON join\_condition[, ...].
```

- È equivalente a: INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.
- Non è equivalente a CROSS JOIN!

Quando usare left join rispetto a right join?

- **JOIN**: Solo i record che fanno "match" in entrambe le tabelle.
- **LEFT JOIN**: Tutto ciò che sta nella prima tabella scritta, con i NULL se manca il pezzo della seconda.
- **RIGHT JOIN**: Tutto ciò che sta nella seconda tabella scritta, con i NULL se manca il pezzo della prima.
- **FULL JOIN**: Tutti i dati di entrambe le tabelle, allineando ciò che combacia e mettendo NULL ovunque manchi una corrispondenza.

6.7 Interrogazioni nidificate e insiemistiche

Interrogazioni nidificate

SQL permette la costruzione di interrogazioni che presentano al loro interno altre interrogazioni. La nidificazione può avvenire nelle clausole **SELECT**, **FROM**, **WHERE**.

L'uso più comune è la nidificazione nella clausola WHERE

6.7.1 Clausola FROM

Con l'utilizzo di questa opzione, è possibile comporre catene di interrogazioni, in cui ciascuna interrogazione utilizza il risultato della precedente come se fosse una tabella di base

```
1 SELECT titolo, prezzoIntero
2 FROM Mostra, (
3     SELECT MAX ( prezzoIntero )
4     FROM Mostra
5 ) AS T( prezzoMax )
6 WHERE prezzoIntero = prezzoMax;
```

dove:

- **T**: alias della tabella. La query interna ha generato una tabellina "anonima", noi la chiamiamo temporaneamente con la lettera T
- **(prezzoMax)**: alias della colonna

6.7.2 Clausola WHERE

Il predicato (predicato complesso) nella clausola WHERE confronta un valore (ottenuto come espressione valutata sulla singola riga), con il risultato dell'esecuzione di un'interrogazione

SQL (in generale un insieme di valori). Occorre estendere gli operatori di confronto per poter realizzare la comparazione tra un valore e un insieme di valori

Soluzione offerta da SQL: estensione degli operatori di confronto (>, <, =, <=, >=, <>) con ANY/SOME e ALL (IN, NOT IN).

- **ANY/SOME**: specifica che la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo per la riga e **almeno uno** degli elementi restituiti dall'interrogazione.
- **ALL**: specifica che la riga soddisfa la condizione solo se **tutti gli elementi** restituiti dall'interrogazione nidificata rendono vero il confronto.
- **IN** e **NOT IN**: usati per rappresentare l'appartenenza o l'esclusione rispetto ad un insieme (identici a = ANY o <>ALL)

6.7.2.1 Operatore ANY/SOME

```
1 expression operator ANY( subquery )
2 expression operator SOME( subquery )
```

- **expression** è un'espressione che coinvolge attributi della SELECT principale.
- (**subquery**) è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ANY**: è vero se la condizione è vera per almeno un valore della subquery.
- **SOME** è uno sinonimo di **ANY**

Visualizzare il nome degli insegnamenti che hanno un numero di crediti inferiore alla media dell'ateneo di un qualsiasi anno accademico.

```
1 SELECT DISTINCT I.nomeins , IE.crediti
2 FROM Insegn I
3 JOIN InsErogato IE ON I.id=IE.id_insegn
4 WHERE IE.crediti < ANY (
5     SELECT AVG ( crediti )
6     FROM InsErogato
7     WHERE modulo =0
8     GROUP BY annoaccademico
9 );
```

6.7.2.2 Operatore ALL

```
1 expression operator ALL( subquery )
```

- **expression** è un'espressione che coinvolge attributi della SELECT principale.
- (**subquery**) è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ALL**: La condizione è vera solo se è vera per TUTTI i valori della subquery.

Trovare il nome degli insegnamenti con almeno un docente e crediti maggiori rispetto ai crediti di ciascun insegnamento del corso di

laurea con id=6. (si considerano solo occorrenze insegnamento genitore)

```

1 SELECT DISTINCT I. nomeins , IE. crediti
2 FROM Insegn I
3     JOIN InsErogato IE ON I.id = IE. id_insegn
4     JOIN Docenza D ON IE.id = D. id_inserogato
5 WHERE IE. modulo = 0 AND IE. crediti > ALL (
6     SELECT crediti
7     FROM InsErogato
8     WHERE id_corsostudi = 6 AND modulo =0
9 );

```

6.7.2.3 Operatore IN

E' un operatore utilizzabile nei predicati complessi.

```

1 ROW ( expr [ ,...]) IN ( subquery )

```

- **expr** è un'espressione costruita con un attributo della query principale. Ci possono essere una o più espressioni.
- La (**subquery**) deve restituire un numero di colonne pari al numero di espressioni in (**expr [,...]**).
- I valori dell'espressioni vengono confrontati con i valori di ciascuna riga del risultato di (**subquery**).
- Il confronto ritorna vero se i valori sono uguali ai valori di almeno una riga della subquery.

Trovare gli impiegati che hanno stesso nome e cognome di qualcuno in un'altra azienda.

```

1 SELECT I.nome , I.cognome
2 FROM Impiegato I
3 WHERE ROW( I.nome , I.cognome ) IN (
4 SELECT I1.nome, I1.cognome FROM ImpiegatoAltraAzienda I1)

```

6.7.2.4 Operatore EXIST

E' una clausola utilizzabile nei predicati complessi

```

1 EXISTS(subquery)

```

- (**subquery**) è una **SELECT**.
- **EXISTS** ritorna falso se (**subquery**) non contiene righe; altrimenti vero.
- **EXISTS** è significativo quando nella (**subquery**) si selezionano righe usando i valori della riga corrente nella **SELECT** principale: **data binding**. Vale a dire se **subquery** è un'interrogazione dipendente dalla query esterna

Determinare i nomi degli impiegati che sono diversi tra loro ma di pari lunghezza

```

1 SELECT I. nome
2 FROM Impiegato I
3 WHERE EXISTS (
4     SELECT 1
5     FROM Impiegato I1
6     WHERE I.nome <> I1. nome AND CHAR_LENGTH( I.nome ) = CHAR_LENGTH( I1
7         . nome )
8 )

```

I.nome nella subquery è il valore di nome nella riga corrente della SELECT principale

Nota

Se si usano attributi esterni nella subquery (data binding), la subquery deve essere valutata per ogni riga della SELECT principale

Visualizzare il nome e il cognome dei docenti, escludendo i coordinatori, che hanno tenuto almeno due insegnamenti (o moduli) con più di 24 crediti nel 2010/2011

```

1 SELECT P.nome , P.cognome
2 FROM Persona P
3 WHERE EXISTS (
4     SELECT 1
5     FROM Docenza D
6     JOIN InsErogato IE ON D. id_inserogato = IE.id
7     WHERE IE. crediti > 24 AND D. coordinatore = '0'
8     AND IE. annoaccademico = '2010/2011' AND D. id_persona = P.id
9     GROUP BY D. id_persona
10    HAVING COUNT (*) >=2
11 );

```

Visualizzare il nome dei corsi di studio che nel 2006/2007 non hanno erogato insegnamenti il cui nome contiene la sottostringa 'Info'.

```

1 SELECT CS. nome FROM CorsoStudi CS
2 WHERE NOT EXISTS (
3     SELECT 1
4     FROM InsErogato IE
5     JOIN Insegn I ON IE. id_insegn = I.id
6     WHERE I.nomeins LIKE '%Info%'
7     AND IE.annoaccademico = '2006/2007'
8     AND IE.id_corsostudi = CS.id
9 )

```

Interrogazioni di tipo insiemistico

```
1 query_1 { UNION | INTERSECT | EXCEPT } [ALL] query_2
```

SQL mette a disposizione anche degli operatori insiemistici UNION, INTERSECT e EXCEPT per manipolare i risultati di più query. Gli operatori insiemistici si possono utilizzare solo al livello più esterno di una query, operando sul risultato di due o più clausole SELECT. Si possono avere sequenze di UNION/INTERSECT/EXCEPT

Gli operatori si possono applicare solo quando `query1` e `query2` producono risultati con lo stesso numero di colonne e di tipo compatibile fra loro. Tutti gli operatori eliminano i duplicati dal risultato a meno che ALL non sia stato specificato.

- **UNION** aggiunge il risultato di `query2` a quello di `query1`.
- **INTERSECT** restituisce le righe che sono presenti sia nel risultato di `query1` sia in quello di `query2`
- **EXCEPT** restituisce le righe di `query1` che non sono presenti nel risultato di `query2`. In pratica esegue la differenza insiemistica

Visualizzare i nomi degli insegnamenti e i nomi dei corsi di laurea che non iniziano per 'A' mantenendo i duplicati

```
1 SELECT nomeins
2 FROM Insegn
3 WHERE NOT nomeins LIKE 'A%'
4 UNION ALL
5 SELECT nome
6 FROM CorsoStudi
7 WHERE NOT nome LIKE 'A%'
```

Visualizzare i nomi degli insegnamenti che sono anche nomi di corsi di laurea.

```
1 SELECT nomeins
2 FROM Insegn
3 INTERSECT ALL
4 SELECT nome
5 FROM CorsoStudi;
```

Visualizzare i nomi degli insegnamenti che NON sono anche nomi di corsi di laurea.

```
1 SELECT nomeins
2 FROM Insegn
3 EXCEPT
4 SELECT nome
5 FROM CorsoStudi;
```


6.7.3 Le viste

Le viste sono tabelle virtuali il cui contenuto dipende dal contenuto delle altre tabelle della base di dati. In SQL le viste vengono definite associando un nome ed una lista di attributi al risultato dell'esecuzione di un'interrogazione. **Ogni volta che si usa una vista, si esegue la query che la definisce.** Nell'interrogazione che definisce la vista possono comparire anche altre viste. SQL **non ammette** però

- dipendenze immediate (definire una vista in termini di se stessa) o ricorsive (definire una interrogazione di base e una interrogazione ricorsiva);
- dipendenze transitive circolari (V1 definita usando V2, V2 usando V3, . . . , Vn usando V1).

```
1 CREATE [ TEMP ] VIEW nome [( col_name [, ...]) ] AS query
```

TEMP: la vista è temporanea. Quando ci si sconnette, la vista viene distrutta. È un'estensione di PostgreSQL. Nella base di dati did2014 si possono fare solo viste temporanee

- **column_name**: nomi delle colonne che compongono la vista; se non vengono specificati, sono ricavati dai nomi delle colonne restituiti dalla query, eventualmente tramite alias.
- **query** deve restituire un insieme di attributi pari e nel medesimo ordine a quello specificato con (**column_name** [, ...]) se presente.
- SQL permette che una vista sia aggiornabile solo quando ciascuna tupla della vista corrisponde a una sola tupla di ciascuna tabella di base.

Definire la vista che contiene gli insegnamenti erogati completi di nomeins e codiceins presi dalla tabella Insegn.

```
1 CREATE TEMP VIEW InsErogatiCompleti AS
2 SELECT I. nomeins , I. codiceins , IE .*
3 FROM InsErogato IE
4 JOIN Insegn I ON IE. id_insegn = I.id;
```

Si vuole determinare quali sono i corsi di studi con il massimo numero di nome di insegnamenti diversi

Prima si crea una vista:

```
1 CREATE TEMP VIEW InsCorsoStudi (Nome , NumIns) AS
2 SELECT CS.nome , COUNT ( DISTINCT I. nomeins )
3 FROM CorsoStudi CS
4 JOIN InsErogato IE ON CS.id=IE. id_corsostudi
5 JOIN Insegn I ON IE. id_insegn = I.id
6 GROUP BY CS.nome;
```

Poi la si usa:

```
1 SELECT Nome , NumIns
2 FROM InsCorsoStudi
3 WHERE NumIns = ANY (
4 SELECT MAX ( NumIns ) FROM InsCorsoStudi);
```

Laurea specialistica in Medicina e Chirurgia: 320

6.8 Indici

Gli indici sono delle **strutture dati ausiliare** che permettono di accedere ai dati di una tabella in maniera più efficiente. Essendo una struttura dati ausiliaria, deve essere sempre mantenuta aggiornata in base al contenuto della tabella in cui è definita. Il costo di aggiornamento può essere significativo quando ci sono molti indici definiti sulla medesima tabella. Gli indici quindi devono essere definiti considerando la loro utilità.

Si consideri il seguente schema fisico

```

1  InsErogato(id, id_insegn, id_corsostudi, id_discriminante,
    annoaccademico, id_facolta, modulo, nomemodulo, discriminantemodulo
    , crediti, programma)
2  Insegn(id, nomeins, codiceins)
3  Discriminante(id, nome, descrizione)
4  CorsoStudi(id, nomecorsostudi, sede, durataAnni)
5  CorsoInFacolta(id, id_corsostudi, id_facolta)
6  Facolta(id, nome)

```

Query SENZA indici

Si consideri la query:

```

1  SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';

```

Il DBMS deve fare una scansione sequenziale della tabella Insegn rispetto nomeins per estrarre le righe con valore uguale a 'Algoritmi';

Il comando \timing attiva/disattiva la visualizzazione del tempo di pianificazione + calcolo di una query. Il tempo è visualizzato subito dopo la visualizzazione del risultato della query. \timing dà un'idea del tempo necessario per eseguire una query.

```

1  =>\timing
2  Timing is ON.
3  => SELECT id , nomeins
4  FROM Insegn
5  WHERE nomeins='Algoritmi';
6
7
8  id | nomeins
9  ----+-----
10  5441 | Algoritmi
11
12  (1 row)
13  TIME: 2.437 ms

```

Il tempo è di 2 millisecondi circa

6.8.1 Creazione indice

```

1 CREATE INDEX [ nome] ON nomeTabella [ USING method ]
2 ({ nomeAttr | ( expression ) } [ ASC | DESC ] [, ...])
3 CREATE INDEX Nome Indice ON NomeTabella (ListaAttributi)

```

- **method** è il tipo di indice;
- **nomeAttr** o **expression** indicano su quali espressione di attributi si deve creare indice.
- **ASC/DESC** specifica se l'indice è ordinato in modo ascendente o discendente.
- **ALTER INDEX** e **DROP INDEX** permettono di modificare/rimuovere indici creati precedentemente.

Un DBMS crea gli indici in modo automatico solo per attributi dichiarati PRIMARY KEY (**indice primario**).

Per tutti gli altri attributi si deve fare una dichiarazione esplicita di **CREATE INDEX**.

```

1 ANALYZE nomeTabella;

```

Il comando ANALYZE[nomeTabella] aggiorna le statistiche circa il contenuto delle tabelle (e indici). È eseguito in modo automatico dal sistema a intervalli regolari. L'ottimizzatore di query usa queste statistiche per decidere quando usare gli indici.

Query CON indici

Per rendere più veloce l'esecuzione della query:

```

1 SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';

```

un indice sull'attributo nomeins potrebbe essere utile perché nomeins è l'attributo usato per selezionare le righe. Query CON

```

1 => CREATE INDEX nomeins_index ON Insegn( nomeins);
2 => ANALYZE Insegn;
3 => SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';
4
5 id | nomeins
6 ----+-----
7 5441 | Algoritmi
8 TIME: 0.587 ms

```

Il tempo di esecuzione è passato da 2.4337 ms a 0.587 ms.

Nota

Nota: Non è garantito che il tempo di esecuzione sia sempre uguale perché il server è multitask. I tempi di queste slide sono stati determinati con il server completamente dedicato

6.8.2 Caso pratico

Si consideri la seguente query senza Indici

```

1 SELECT DISTINCT I.nomeins
2 FROM CorsoStudi CS
3 JOIN InsErogato IE ON CS.id = IE.id_corsostudi
4 JOIN Insegn I ON I.id = IE.id_insegn
5 WHERE IE.annoaccademico = '2009/2010'
6 AND CS.nome = 'Laurea in Informatica';
7 ...
8 (26 ROWS )
9 TIME : 40.975 ms

```

Quanto è possibile rendere più veloce la query definendo degli indici?

Una possibilità è definire gli indici su tutti gli attributi usati per fare i JOIN o le selezioni

```

1 CREATE INDEX cs_id ON CorsoStudi(id);
2 CREATE INDEX i_id ON Insegn(id);

```

Nota

Gli indici sugli attributi PRIMARY KEY (id nel nostro caso) solitamente sono già presenti! In did2014small sono stati tolti per fare gli esperimenti

Si consideri la creazione degli indici:

```

1 CREATE INDEX ie_id_corsostudi ON Inserogato( id_corsostudi);
2 CREATE INDEX ie_id_insegn ON Inserogato( id_insegn);
3 CREATE INDEX ie_aa ON Inserogato( annoaccademico);
4 CREATE INDEX cs_nome ON Corsostudi( nome );
5 ANALYZE Corsostudi;
6 ANALYZE Inserogato;
7 ANALYZE Insegn;

```

Non tutti gli indici sono necessari, ma per ora accettiamo il costo di crearli tutti. Importante! Forziamo il DBMS ad aggiornare le statistiche con il comando ANALYZE dopo la creazione degli indici.

Con la creazione degli indici, la query richiede un tempo inferiore:

```

1 SELECT DISTINCT I.nomeins
2 FROM Corso StudiCS
3 JOIN InsErogato IE ON CS.id= IE.id_corsostudi
4 JOIN InsegnI ON I.id= IE.id_insegn
5 WHERE IE.annoaccademico= '2009/2010'
6 AND CS.nome= 'Laurea in Informatica';
7 ...
8 (26 ROWS )
9 TIME : 5.201 ms

```

Il tempo di esecuzione è passato da 40 ms a 5 ms.

6.8.3 Tipi di indice

PostgreSQL ammette diversi tipi di indici: B-tree, hash, GiST, SP-GiST, GIN e BRIN. Sintassi per specificare anche il tipo:

```
1 CREATE INDEX nome ON nomeTab USING type(COLUMN);
```

6.8.3.1 B-tree

Se nel CREATE INDEX non si specifica il tipo di indice, il sistema crea un indice di tipo B-tree. L'ottimizzatore di query considera un indice B-tree ogni volta che l'attributo indicizzato è coinvolto in un confronto usando uno degli operatori: <, <=, =, >=, >. Un indice B-tree può essere considerato anche quando ci sono **BETWEEN**, **IN**, **IS NULL**, **IS NOT NULL** e **LIKE** 'abc%'.

Con la localizzazione (it_IT.UTF-8), la comparazione di stringhe con operatori diversi da '=' e con **LIKE** ha regole diverse (minuscole/maiuscole) rispetto a UTF8. Un indice su attributo VARCHAR in una base di dati con LC_LOCALE = it_IT.UTF-8 (come le basi di dati in dbserver) dovrebbe essere dichiarato come:

```
1 CREATE INDEX nome ON nomeTab(nomeAttrvarchar_pattern_ops);
```

se si vuole che l'indice sia usato anche quando il confronto usa <,>,<>, LIKE. La parola chiave **varchar_pattern_ops** abilita l'uso degli operatori rispettando le regole imposte dal locale (it_IT.UTF-8).

6.8.3.2 Hash

Il tipo **hash** ha un uso limitato perché può essere usato solo quando i confronti sonodi eguaglianza. La sua gestione poi è più complicata in basi di dati replicate o in caso di crash: il gruppo di PostgreSQL ne sconsiglia l'uso. Gli altri tipi di indici sono indicati per particolari strutture dati. Per esempio, GiST è indicato quando un attributo è di tipo geometrico bi-dimensionale(tipo non standard).

6.8.3.3 Multi-attributo

Se si hanno query che hanno condizioni su coppie (terne, ecc) di attributi di una tabella, un indice multi-attributo definito usando la coppia (terna, ecc) di attributi potrebbe essere più utile rispetto gli indici sui singoli attributi.

Creazione indice multi-attributo

```
1 CREATE INDEX ie_aa_idcs ON
2
3 Inserogato( annoaccademico, id_corsostudi);
4 ANALYZE Inserogato;
5
6 SELECT I.nomeins, I.codiceins
7 FROM InsegnI JOINInserogatoIE ONI.id= IE. id_insegn
8 WHERE IE. annoaccademico= '2006/2007 ';
```

```

9 ANDIE. id_corsostudi= 4;
10
11 ... (46 ROWS )
12 TIME : 1.910 ms

```

NON sempre gli indici multi-attributo possono essere usati: per esempio in espressioni con OR non è possibile.

```

1 SELECT I. nomeins, I. codiceins
2 FROM InsegnI JOIN InsErogatoIE ON I.id= IE. id_insegn
3 WHERE IE.annoaccademico= '2006/2007' OR IE.id_corsostudi= 4;
4 ... (5334 ROWS )

```

- Non c'è il tempo di esecuzione perché il risultato è diverso e quindi non comparabile.
- In questa query l'indice `ie_aa_idcs` NON è usato perché la disgiunzione **OR** rende impossibile di fatto il suo uso.
- Nemmeno gli indici singoli `ie_id_corsostudi` e `ie_aa` sono usati.
- L'ottimizzatore esegue una scansione di tutta la tabella `Insegn`.

6.8.3.4 Indici di espressioni

È possibile definire indici anche usando espressioni di attributi.

```

1 CREATE INDEX nome ON nomeTabella(expression);

```

dove `expression` è una espressione su uno o più attributi.

Query con indici definiti usando espressioni:

```

1 CREATE INDEX ins_lower_nonins ON Insegn
2 ( LOWER( nomeins) varchar_pattern_ops);
3 ANALYZE Insegn;
4 SELECT nomeins FROM Insegn
5 WHERE LOWER(nomeins) LIKE 'algoritmi %';
6 ... (4 ROWS )
7 TIME : 1.796 ms

```

6.8.4 Costo e Pratiche d'uso

Gli indici costano tempo e memoria. Se si considera solo il tempo, il costo maggiore è mantenere gli indici aggiornati. Per ogni operazione di INSERT, UPDATE, DELETE su una tabella con indici, il DBMS deve aggiornare anche gli indici della tabella.

Regola pratica: definire gli indici in base alle query più frequenti.

Il comando PostgreSQL `\di` visualizza l'elenco degli indici definiti in una base di dati.

Il comando PostgreSQL `\d nomeTabella` visualizza la definizione di tabella e degli indici associati.

Perchè questo?

Un DBMS ha un ottimizzatore di query che determina un piano per eseguire una data query nel minor tempo possibile. Il comando EXPLAIN permette di vedere il piano di esecuzione di una query che l'ottimizzatore determina senza eseguire la query. Un piano di esecuzione di una query è un albero di nodi di esecuzione.

Le foglie sono nodi di scansione: l'esecuzione di questi nodi restituiscono indirizzi di righe della tabella.

Struttura

Esistono differenti tipi di scansioni: **sequenziali**, **indicizzate** e **mappate su bit**.

- Se una query contiene **JOIN**, **GROUP BY** o **ORDER BY** o altre operazioni sulle righe, **allora ci saranno altri nodi di esecuzione sopra le foglie nell'albero**.
- L'output di EXPLAIN ha una riga per ciascun nodo nell'albero di esecuzione dove viene indicato il tipo di operazione e una stima del costo di esecuzione.
- Ulteriori proprietà del nodo possono essere mostrate con una riga indentata subito sotto.
- La prima riga dell'output ha la stima del costo totale di esecuzione della query. Questo è il costo che l'ottimizzatore cerca di minimizzare.

```

1 EXPLAIN SELECT* FROM Insegn;
2 QUERY PLAN
3 -----
4 SeqScan ON insegn(cost=0.0..185.69 ROWS=8169 width=63)

```

- 0.0 è il costo iniziale, per produrre la prima riga(in termini di numero di accessi alla memoria secondaria).
- 185.69 è il costo totale, per produrre tutte le righe.
- 8169 è il numero totale di righe del risultato(Il numero totale di righe non è sempre il numero totale di righe valutate dall'esecutore).
- 63 è la dimensione, in byte, di ciascuna riga.

6.8.4.1 Explain con due nodi

```

1 EXPLAIN SELECT* FROM Insegn WHERE id < 1000;
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=18.60..132.79 ROWS=815...)
5 Recheck Cond: (id < 1000)
6 -> Bitmap INDEX Scan ON i1 (cost=0..18.39 ROWS=815 w =0)
7 INDEX Cond: (id < 1000)

```

Prima viene eseguito il nodo foglia (Bitmap INDEX Scan): grazie all'indice B-tree, l'ottimizzatore determina un vettore di indirizzi di righe da considerare (in un B-tree la chiave cercata k non è pari a nessuna chiave K_i ma è compresa tra K_i e K_{i+1} , può essere presente nel sottoalbero P_i). Tale vettore viene poi passato al nodo padre (Bitmap Heap Scan) che carica le righe ed esegue la selezione finale ricontrollando che $id < 1000$.

6.8.4.2 Explain con due nodi – AND con un solo indice

```

1 DROP INDEX ins_nomeins;
2 EXPLAIN SELECT* FROM Insegn WHERE id <1000 AND nomeins LIKE 'A1%';
3 QUERY PLAN
4 -----
5 Bitmap Heap Scan ON Insegn(cost=18.40..134.62 ROWS=1...)
6 Recheck Cond: (id < 1000)
7 Filter: (( nomeins)::TEXT~~ 'A1% '::TEXT)
8 -> Bitmap INDEX Scan ON i1 (cost=0.00..18.39 ROWS=815 width=0)
9 INDEX Cond: (id < 1000)

```

Il nodo Bitmap Heap Scan carica le righe indicate dall'indice e, per ciascuna, ricontrolla che $id < 1000$, e la seleziona se nomeins inizia con 'A1'.

6.8.4.3 Esempi d'uso

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

```

1 CREATE INDEX nomeIdx ON Insegn(nomeins varchar_pattern_ops);
2 ANALYZE Insegn;
3 EXPLAIN SELECT* FROM Insegn WHERE id<1000 AND nomeins LIKE 'A1%';
4 QUERY PLAN
5 -----
6 Bitmap Heap Scan ON insegn(cost=15.30..22.58 ROWS=4 width=63)
7 Recheck Cond: (id < 1000)
8 Filter: (( nomeins)::TEXT~~ 'A1% '::TEXT)
9 -> BitmapAnd(cost=15.30..15.30 ROWS=4 width=0)
10 -> Bitmap INDEX Scan ON nomeidx(cost=0..2.65 ROWS=37 width=0)
11 INDEX Cond: (((nomeins)::TEXT ~>=~ 'A1'::TEXT) AND
12 ((nomeins)::TEXT~<~ 'Am'::TEXT))
13 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
14 INDEX Cond: (id < 1000)

```

- Ciascuna foglia scansiona un indice. Il padre delle foglie (BitmapAnd) fa l'intersezione degli indirizzi delle righe trovate.
- La root (Bitmap Heap Scan) carica le righe ed ricontrolla che le condizioni siano soddisfatte.

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

```

1 EXPLAIN SELECT* FROM Insegn WHERE id <1000 OR nomeins LIKE 'A1%';
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=15.47..132.25 ROWS=850 width=63)
5 Recheck Cond: ((id < 1000) OR((nomeins)::TEXT~~ 'A1% '::TEXT))
6 Filter : ((id < 1000) OR((nomeins)::TEXT~~ 'A1% '::TEXT))
7 -> BitmapOr(cost=15.47..15.47 ROWS=852 w=0)
8 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
9 INDEX Cond: (id < 1000)
10 -> Bitmap INDEX Scan ON nomeidx(cost=0.00..2.65 ROWS=37 width=0)
11 INDEX Cond: (((nomeins)::TEXT~>=~ 'A1'::TEXT) AND
12 ((nomeins)::TEXT~<~ 'Am'::TEXT))

```


Il padre delle foglie (BitmapOr) esegue l'unione degli indirizzi delle righe trovate.

Esempio di EXPLAIN con planner che decide diversamente cambiando condizione

```

1 EXPLAIN SELECT* FROM Insegn WHERE id <1000 AND nomeins LIKE '%A1';
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=12.40..128.62 ROWS=1 width=63)
5 Recheck Cond: (id < 1000)
6 Filter: ((nomeins)::TEXT~~ '%A1'::TEXT)
7 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
8 INDEX Cond: (id < 1000)

```

La condizione nomeins LIKE '%A1' rende impossibile l'uso dell'indice.

Esempio di EXPLAIN di un JOIN: join mediante hash table

```

1 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie.id_insegn= i.id
   WHERE ie.annoaccademico= '2013/2014';
2 QUERY PLAN
3 -----
4 Hash JOIN(cost=287.80..6328.90 ROWS=5155 width=641)
5 Hash Cond: (ie. id_insegn= i.id)
6 -> Seq Scan ON inserogatoie(cost =0.00..5970.21 ROWS =5155 w =578)
7 Filter : ((annoaccademico)::TEXT= '2013/2014'::TEXT)
8 -> Hash (cost =185.69..185.69 ROWS =8169 width=63)
9 -> Seq Scan ON insegni (cost =0.00..185.69 ROWS =8169 w =63)

```

Esempio di EXPLAIN di un JOIN: join mediante nestedloop

```

1 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie.id_insegn> i.id
   WHERE ie. annoaccademico= '2013/2014';
2 QUERY PLAN
3 -----
4 Nested Loop (cost=0.28..393703.79ROWS=14037065 width=641)
5 -> SeqScanONinserogatoie(cost=0.00..5970.21 ROWS=5155 w=578)
6 Filter: (( annoaccademico)::TEXT= '2013/2014':: TEXT)
7 -> INDEX Scan USING i1 ON insegni (cost =0.28..47.99 ROWS=2723 w=63)
8 INDEX Cond: (ie. id_insegn> id)

```

Ottimizzazione della query precedente:

```

1 CREATE INDEX ie_aa ON Inserogato(annoaccademico varchar_pattern_ops);
2 ANALYZE Inserogato;
3 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie. id_insegn> i.id
   WHERE ie.annoaccademico= '2013/2014';
4 QUERY PLAN
5 -----
6 Nested Loop ( cost =82.65.. 391932.20ROWS =14037065 width =641)
7 -> Bitmap Heap Scan ON inserogatoie(cost=82..4198 rows=5155 w=578)
8 Recheck Cond: ((annoaccademico)::TEXT= '2013/2014'::TEXT)
9 -> Bitmap INDEX Scan ON ie_aa(cost=0.00..81.08 ROWS=5155 w=0)
10 INDEX Cond: ((annoaccademico)::TEXT= '2013/2014':: TEXT)
11 -> INDEX Scan USING i1 ON insegni(cost =0.28..48 ROWS =2723 w =63)

```

```
12 INDEX Cond : (ie. id_insegn > id)
```

Altro tipo di JOIN: join mediante merge join

```
1 EXPLAIN SELECT *FROM t1, t2
2 WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2 ;
3 QUERY PLAN
4 -----
5 Merge JOIN (cost=198.11..268.19 ROWS=10 width=488)
6 Merge Cond: (t1.unique2 = t2.unique2 )
7 -> INDEX Scan USING t1_unique2 ON t1 (cost =0..656 ROWS =101..)
8 Filter : ( unique1 < 100)
9 -> Sort(cost =197.83..200.33 ROWS =1000..)
10 Sort KEY: t2.unique2
11 -> SeqScanON t2 (cost =0.00..148.00 ROWS =1000..)
```

6.8.5 Explain versione avanzata

Il comando **EXPLAIN ANALYZE** mostra il piano di esecuzione, esegue la query senza registrare eventuali modifiche e mostra, infine, una stima verosimile dei tempi di esecuzione

Explain con esecuzione

```
EXPLAIN ANALYZE SELECT *FROM Insegn WHERE id <1000 AND nomeins LIKE 'Algo %';
```

QUERY PLAN

```
-----
Bitmap Heap Scan ON insegn (cost=18.40..134.62 ROWS=1..)
  (actual TIME=1.001..1.605 ROWS=1 loops=1)
  Recheck Cond: (id < 1000)
  Filter: (nomeins ~~ 'Algo %')
  ROWS Removed BY Filter: 814
  Heap Blocks: exact=90
  -> Bitmap INDEX Scan ON insegn_pkey (cost=0..18 ROWS=815)
    (actual TIME=0.928..0.928 ROWS=815 loops=1)
    INDEX Cond: (id < 1000)
Planning TIME: 1.243 ms
Execution TIME: 2.234 ms
```

- Per ogni nodo del piano: dettaglio righe rimosse, memoria usata e tempi di esecuzione in ms.
- Alla fine: stima tempo di pianificazione ed di esecuzione.

Explain con esecuzione di una query pesante

```
EXPLAIN ANALYZE SELECT * FROM insegn I, interrogato IE
WHERE IE.annoaccademico = '2013/2014' AND IE.id_insegn > I.id;
```

QUERY PLAN

```
-----
Nested Loop (cost=0..397132 ROWS=14148708 width=638)
  (actual TIME=0.111..18713.811 ROWS=24457113 loops=1)
-> Seq Scan ON interrogato (cost=0..6059 ROWS=5196)
  (actual TIME=0.070..41.651 ROWS=5155 loops=1)
  Filter: (annoaccademico = '2013/2014')
  ROWS Removed BY Filter: 62862
-> INDEX Scan USING insegn_pkey ON insegn (cost=0..48.03 ROWS=2723)
  (actual TIME=0.010..1.790 ROWS=4744 loops=5155)
  INDEX Cond: (IE. id_insegn > id)
Planning TIME: 0.612 ms
Execution TIME: 21471.997 ms
```

- Il valore di loops indica per quante volte viene eseguito il nodo.
- Spesso il numero di righe dopo cost è diverso dal numero di righe dopo actualTIME. Quest'ultimo è più corretto.

6.9 Introduzione al controllo di concorrenza in SQL

Una transazione SQL è una sequenza di istruzioni SQL che può essere eseguita in concorrenza con altre transazioni in modo isolato. Se si implementasse un isolamento completo, il grado di parallelismo sarebbe limitato. Si accettano quindi diversi livelli di isolamento per favorire un maggior grado di parallelismo. I diversi livelli di isolamento possono determinare delle anomalie di esecuzioni:

- **perdita di aggiornamento** (lost update)
- **lettura sporca** (dirty read)
- **letture inconsistenti** (non-repeatable read)
- **aggiornamento fantasma** (ghost update)
- **inserimento fantasma** (phantom update)

In PostgreSQL è prevista un'altra anomalia che nel testo di teoria non è considerata: **mancata serializzazione** (serialization anomaly). Il risultato di un gruppo di transazioni (nessun abort) è inconsistente con tutti gli ordini di esecuzioni seriali delle stesse. Di seguito viene mostrato come si introducono i livelli di isolamento implementati in PostgreSQL e come si possono attivare

6.9.1 Transazioni

In PostgreSQL una transazione ben formata è così composta:

- inizia con il comando **BEGIN TRANSACTION** o **START TRANSACTION**;
 - termina con un **END TRANSACTION**;
 - nel mezzo vengono eseguiti con il comando **COMMIT** (per confermare tutte le istruzioni della transazione) o con il comando **ROLLBACK** (per annullare tutte le operazioni).
- Transazione passaggio importo da

Transazione: passaggio importo da un conto ad un altro

```

1 BEGIN;
2 UPDATE accounts SET balance = balance -100.0 WHERE name = 'Alice';
3 UPDATE accounts SET balance = balance +100.0 WHERE name = 'Bob';
4 COMMIT;

```

Transazione: passaggio importo da un conto ad un altro

```

1 BEGIN;
2 UPDATE accounts SET balance = balance -100.0 WHERE name = 'Alice';
3 UPDATE accounts SET balance = balance +100.0 WHERE name = 'Bob';
4 ABORT

```

Alice ha ancora l'importo originale nel suo conto!

6.9.1.1 Controllo concorrenza funzionamento

PostgreSQL mantiene la consistenza dei dati usando un modello multi-versione (Multiversion Concurrency Control (MVCC)):

Le transizioni in PostgreSQL sono gestite come:

- Ciascuna transazione vede un'istantanea della base di dati.
- Le letture su questa istantanea sono sempre possibili e non sono mai bloccate anche se ci sono altre transazioni che stanno modificando la base di dati.
- Le scritture possono essere sospese. Una scrittura è sospesa quando, in parallelo, un'altra transazione (non ancora chiusa) ha modificato la sorgente dei dati che si vuole aggiornare.
- Al COMMIT, il sistema registra sempre l'istantanea aggiornata come nuova base di dati.

In questo modo ci sono meno conflitti e una performance più ragionevole in ambito multiutente

Esempio di sessioni concorrenti

Si consideri la tabella *Studente* con il seguente contenuto:

```
SELECT matricola, cognome, città FROM Studente;
```

Matricola	Cognome	Città
IN0001	Rossi	Verona
IN0002	Paoli	Padova
IN0003	Bianchi	VERONA
IN0004	Rossi	Verona
IN0005	Verdi	Va
IN0006	Nocasa	

Esempio di sessioni concorrenti			
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	SELECT * FROM STUDENTE; ... -- <i>Tabella iniziale</i>		
3	UPDATE Studente SET città = 'Vicenza' WHERE matricola = 'IN0006'; UPDATE 1	SELECT città FROM STUDENTE 3 WHERE matricola = 'IN0006'; città -----	3
4	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa Vicenza	UPDATE Studente SET città = 'VI' WHERE matricola = 'IN0006'; -- <i>esecuzione sospesa. La tabella è bloccata dall'istante 3 in T1</i>	4
5	COMMIT;		5
6	continua...	UPDATE 1 -- <i>il COMMIT di T1 ha sbloccato!</i>	6

Esempio di sessioni concorrenti			
Transazione 1		Transazione 2	
7	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa Vicenza		7
8		COMMIT;	8
9	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa VI	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa VI	9

6.9.2 Livelli di isolamento

I 4 livelli di isolamento offerti da PostgreSQL 11.3 sono, in ordine decrescente di isolamento:

1. **Serializable**: Garantisce che un'intera transazione è eseguita in un qualche ordine sequenziale rispetto ad altre transazioni: completo isolamento da transazioni concorrenti.
 - Anomalie possibili: nessuna!
2. **Repeatable Read**: Garantisce che i dati letti durante la transazione non cambieranno a causa di altre transazioni: rifacendo la lettura dei medesimi dati, si ottengono sempre gli stessi.
 - Questo livello è più restrittivo del livello standard in SQL 'Repeatable Read'.

- Anomalie possibili: mancata serializzazione (rispetto a quanto visto a teoria questo livello di isolamento risolve anche l'anomalia di inserimento fantasma).
3. **Read Committed**: Garantisce che qualsiasi SELECT di una transazione veda solo i dati confermati (COMMITTED) prima che la SELECT inizi.
 - È il livello di isolamento di default usato da PostgreSQL.
 - Anomalie possibili: lettura inconsistente, aggiornamento fantasma, inserimento fantasma e mancata serializzazione.
 4. **Read Uncommitted**: In PostgreSQL (già da 11.3) è implementato come Read Committed. Quindi NON esiste questo livello di isolamento.

Il cambio di livello di isolamento si effettua con SET TRANSACTION appena dopo il BEGIN della transazione

6.9.2.1 Sintassi

```
1 SET TRANSACTION transaction_mode;  
2 // dove transaction_mode e' uno tra  
3 ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ | READ COMMITTED |  
4 READ UNCOMMITTED }  
5 | READ WRITE | READ
```

modo standard

```
1 BEGIN;  
2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
3 COMMIT;
```

modo compatto

```
1 BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;  
2  
3 COMMIT;
```

6.9.2.2 Livello di isolamento: Read Committed

- È il livello di default, quindi è sufficiente BEGIN;
 - SELECT vede solo i dati registrati (COMMITTED) in altre transazioni e quelli modificati da eventuali comandi precedenti nella medesima transazione.
 - UPDATE, DELETE vedono i dati come SELECT. Inoltre, se i dati che devono essere aggiornati sono stati modificati ma non registrati in transazioni concorrenti, il comando deve:
1. attendere il COMMIT o ROLLBACK della transazione dove è stato fatto il cambio
 2. riesaminare le righe selezionate per verificare che soddisfano ancora i criteri del comando.

Caso Lettura inconsistente		id (serial)	hit (integer)
Si assuma che esista la tabella W:		1	9
Si considerino le seguenti transazioni concorrenti		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	SELECT hit FROM W WHERE hit <=9; 9		2
3		UPDATE W SET hit =10 WHERE hit = 9;	3
4		COMMIT ;	
5	SELECT hit FROM W WHERE hit <=9; 9		
6	COMMIT;		

T1 ha due letture diverse della stessa tabella senza averla modificata.

Caso Aggiornamento Fantasma		id (serial)	hit (integer)
Si assuma la tabella W e che esista un vincolo di programma $SUM(hit) \leq 20$.		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	DO \$\$ DECLARE n INTEGER;		2
3	BEGIN		3
4	n := (SELECT hit FROM W WHERE id =1); -- n=9		4
5		UPDATE W SET hit = 8 WHERE id =1;	5
6		UPDATE W SET hit = 12 WHERE id =2;	6
7		COMMIT;	7
8	n := n + (SELECT hit FROM W WHERE id =2); -- n=12		8
9	IF n > 20 THEN RAISE 'La somma è %', n; END IF;		9

La base di dati soddisfa sempre il vincolo, ma per T1 no. T1 ha perso un aggiornamento!

Livello di isolamento Read Committed

Read Committed: caso più articolato		id (serial)	hit (integer)
Transazione 1		1	9
Transazione 2		2	10
1	BEGIN;		1
2	...		2
3	UPDATE W SET hit=hit +1; UPDATE 2		3
4	...		4
5	...		5
6	COMMIT ;		6
7			7

DELETE non cancella la tupla id=2 selezionata al passo 4!
 La tupla, infatti, viene aggiornata da UPDATE; UPDATE blocca DELETE fino al COMMIT.
 DELETE, quindi, deve riesaminare (solo) la tupla id=2.
 Al riesame, la clausola WHERE hit=10 non vale più. Tupla non cancellata.

6.9.2.3 Livello di isolamento Repeatable Read

Differisce da 'Read Committed' per il fatto che i comandi di una transazione vedono sempre gli stessi dati. PostgreSQL associa alla transazione una istantanea della base di dati all'esecuzione del suo primo comando.

- 'Repeatable Read' in PostgreSQL è più stringente di quanto richiesto dallo standard SQL.
- Due **SELECT** identiche successive vedono sempre gli stessi dati.
- **UPDATE e DELETE** vedono i dati come SELECT. Se i dati che devono essere aggiornati sono stati modificati ma non registrati in transazioni concorrenti, il comando deve attendere il **COMMIT/ROLLBACK** della transazione dove è stato fatto il cambio e riesaminare le righe selezionate.
- In caso di **ROLLBACK, UPDATE e DELETE** possono procedere. In caso di **COMMIT**, i dati sono cambiati, quindi UPDATE e DELETE vengono bloccati con l'errore **ERROR**: «could NOT serialize access due to concurrent UPDATE».

6.9.2.4 Livello di isolamento: Repeatable Read

'Repeatable Read': più restrittivo di 'Committed Read'		id (serial)	hit (integer)
Si assuma che esista la tabella W:		1	9
Si considerino le seguenti transazioni concorrenti		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	1
2	UPDATE W SET hit=hit +1; UPDATE 2		2
3		DELETE FROM W WHERE hit =10;	3
4		COMMIT ;	4
5	COMMIT;	ERROR : could NOT serialize access due to concurrent UPDATE	5

Cattura tutte anomalie tranne mancata serializzazione. Se si usa questo livello, si deve prevedere la possibilità di transazioni abortite per aggiornamenti concorrenti.

Repeatable Read: anomalia 'mancata serializzazione'		id (serial)	hit (integer)
Repeatable Read ammette l'anomalia mancata serializzazione.		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	1
2	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1		2
3		INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1	3
4	COMMIT;	COMMIT;	4

La tabella contiene due valori 11 e non 11 e 12. Il risultato ottenuto non è consistente con nessun ordine di esecuzione delle due transizioni.

Figura 6.2: anomalia: mancata serializzazione

È il più **restrittivo**: nessuna anomalia è permessa. Se si usa questo livello, si deve prevedere la possibilità di transazioni abortite per aggiornamenti concorrenti (come nel caso di 'Repeatable Read').

		id (serial)	hit (integer)
Serializable: anomalie di serializzazione non sono possibili!		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;	BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;	1
2	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1		2
3	COMMIT;	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1	3
4		COMMIT;	4
		ERROR : could NOT serialize access due to READ/WRITE dependencies among Transactions DETAIL : Reason code : Canceled ON identification AS a pivot, during COMMIT attempt. HINT: The TRANSACTION might succeed if retried.	

Un metodo pratico per decidere se è necessario il livello serializable è: verificare se la selezione delle righe di un UPDATE/INSERT/DELETE in una transizione può essere modificata in una transizione concorrente (che può aggiungere/togliere o modificare le righe di interesse).

6.9.2.5 Serializable: avvertenze

- L'uso di transazioni 'Serializable' rende più semplice lo sviluppo di programmi SQL: è sufficiente dimostrare che una transazione, da sola, determina sempre uno stato corretto per avere la garanzia che essa continuerà ad essere corretta anche in ambiente concorrente dichiarandola 'Serializable'.
- Dall'altra parte però dichiarare tutte le transazioni 'Serializable' determina, in generale, una limitazione alle prestazioni (throughput) di un DMBS.

6.9.2.6 Cosa fare in caso di failure:

- È fondamentale prevedere e gestire gli errori di concorrenza che possono occorrere con livello di isolamento 'Repeatable Read' o 'Serializable'.
- Se una transizione fallisce, il codice di fallimento è **SQLSTATE = '40001'**.
- SQLSTATE è una variabile di ambiente che si può leggere sia da programmi interni sia da programmi esterni.
- In caso di errore, si deve semplicemente ritentare l'esecuzione della transazione.
- Il costo computazionale di rieseguire una transazione è solitamente meno oneroso di quello che si avrebbe se si gestissero le transazioni usando i lock espliciti come, ad esempio, SELECT FOR UPDATE e SELECT FOR SHARE

6.9.3 Lock Espliciti

PostgreSQL permette di attivare lock espliciti di tabelle e anche di righe di tabelle. Lock espliciti dovrebbero essere gestiti a livello di applicazione quando il modello MVCC non garantisce il comportamento richiesto (soprattutto a livello di prestazioni). In questo corso i lock espliciti non sono considerati. Si rimanda al capitolo 13.3 del manuale di PostgreSQL per chi volesse approfondire l'argomento